

LECTURE 13
Memory Management (II)

Xuhao Chen

March 19, 2025



Survey

- Telerun
- Pacing/schedule of the homeworks and projects
- Lab session
- Projects group-based or solo?
- What is wanted in the writeups & the weekly status reports?

Outline

- Virtual Memory System
- Memory Allocation Strategies
 - ▣ Stacks
 - ▣ Heaps
- Garbage Collection
 - ▣ Reference Counting
 - ▣ Mark and Sweep
 - ▣ Stop-and-Copy
- Memory Hierarchy and Organization
 - ▣ Cache Hierarchy
 - ▣ Memory Level Parallelism

Different Ways of Garbage Collection

	Manual	Reference Counting	Mark and Sweep	Stop and Copy
Ease of Use	Bad			
Throughput	Good			
Latency	Good			
External Fragmentation	Bad			
Example	C malloc/free			

Garbage Collectors

Idea

- Free the programmer from freeing objects.
- A GC **identifies** and **recycles** the objects that the program can no longer access.
- GC can be built-in (Python, Java, Julia) or do-it-yourself.



GARBAGE COLLECTION BY REFERENCE COUNTING



Garbage Collection

Terminology: three types of memory objects

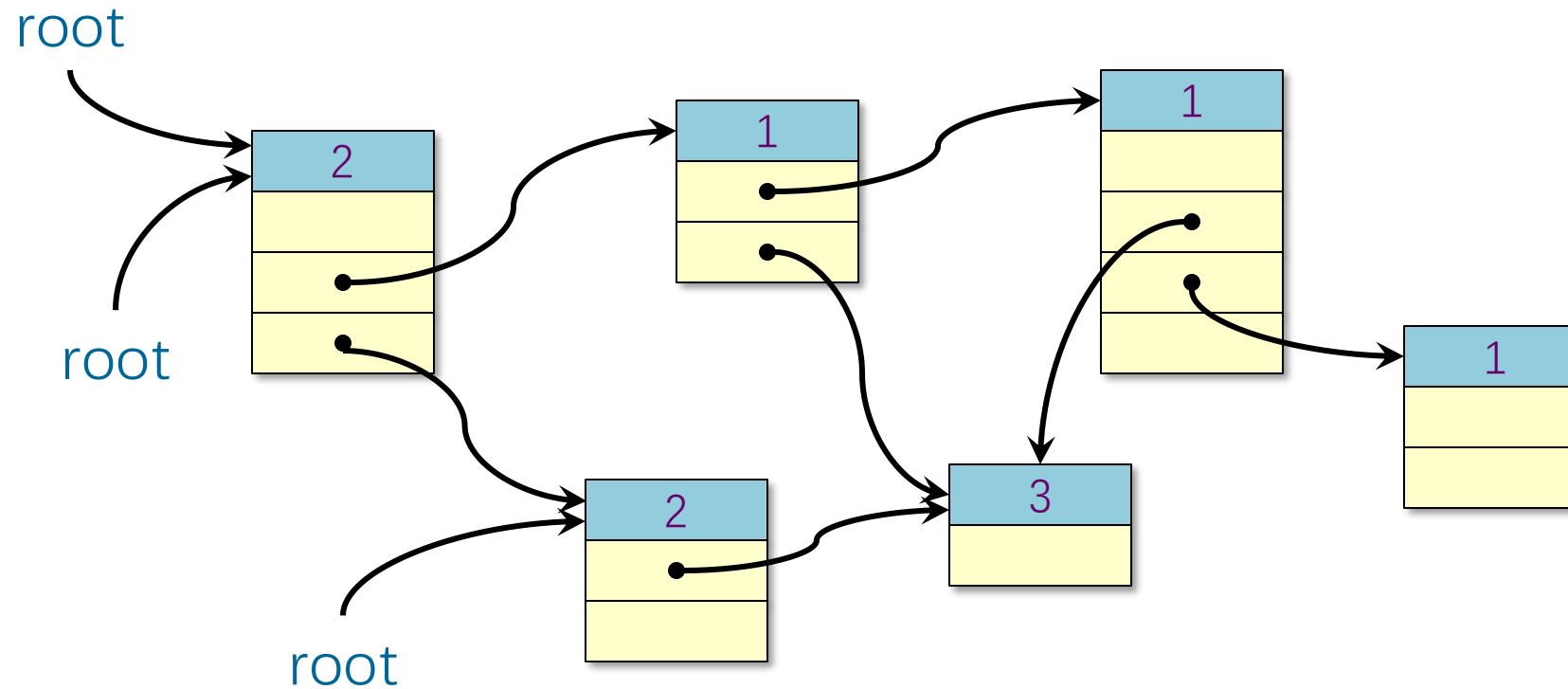
- **Roots** are objects directly accessible by the program (globals, stack, etc.).
- **Live** objects are reachable from the roots by following pointers.
- **Dead** objects are inaccessible and can be recycled.

How can the GC identify pointers?

- **Strong typing** — types are known at compile time (or at runtime with JIT).
- Prohibit **pointer arithmetic** (which may slow down some programs).

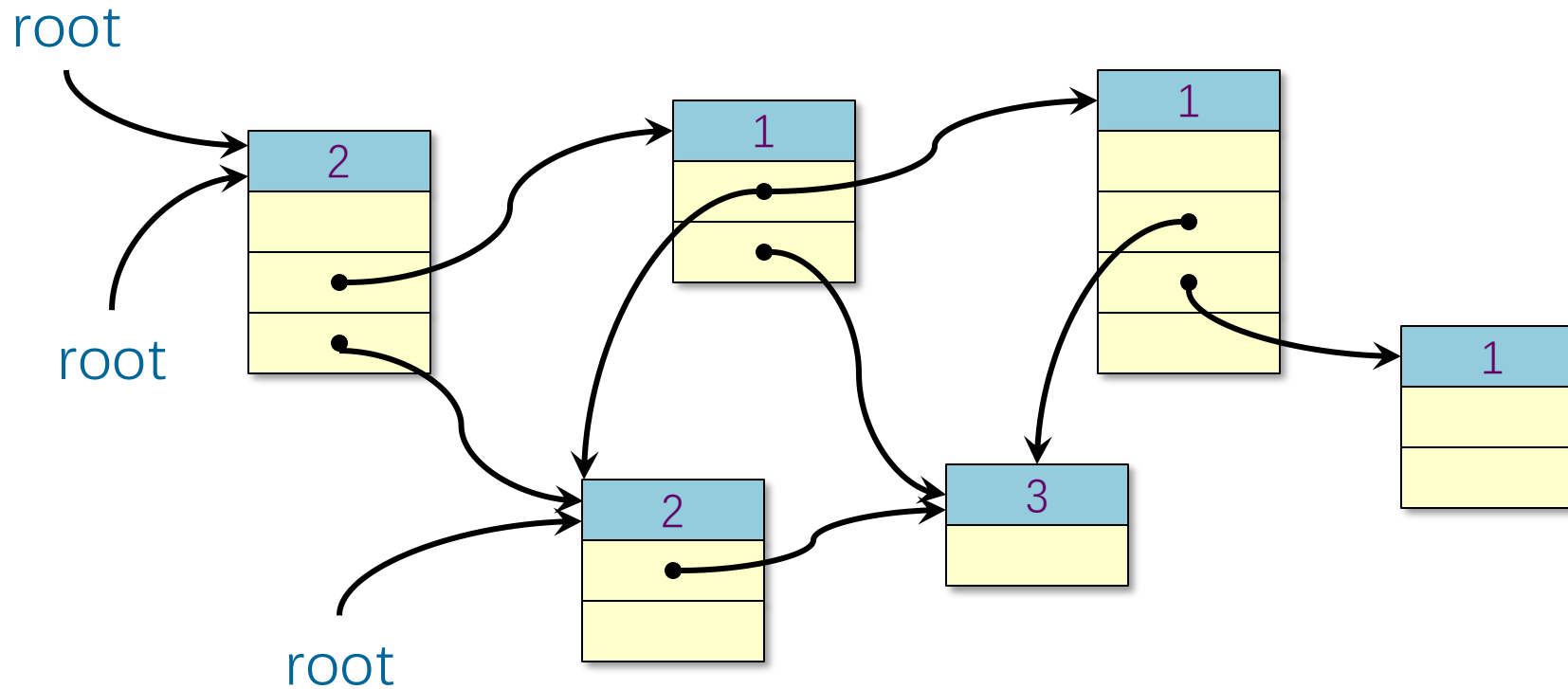
Reference Counting

Keep a count of the number of pointers referencing each object.
If the count drops to **0**, free the dead object.



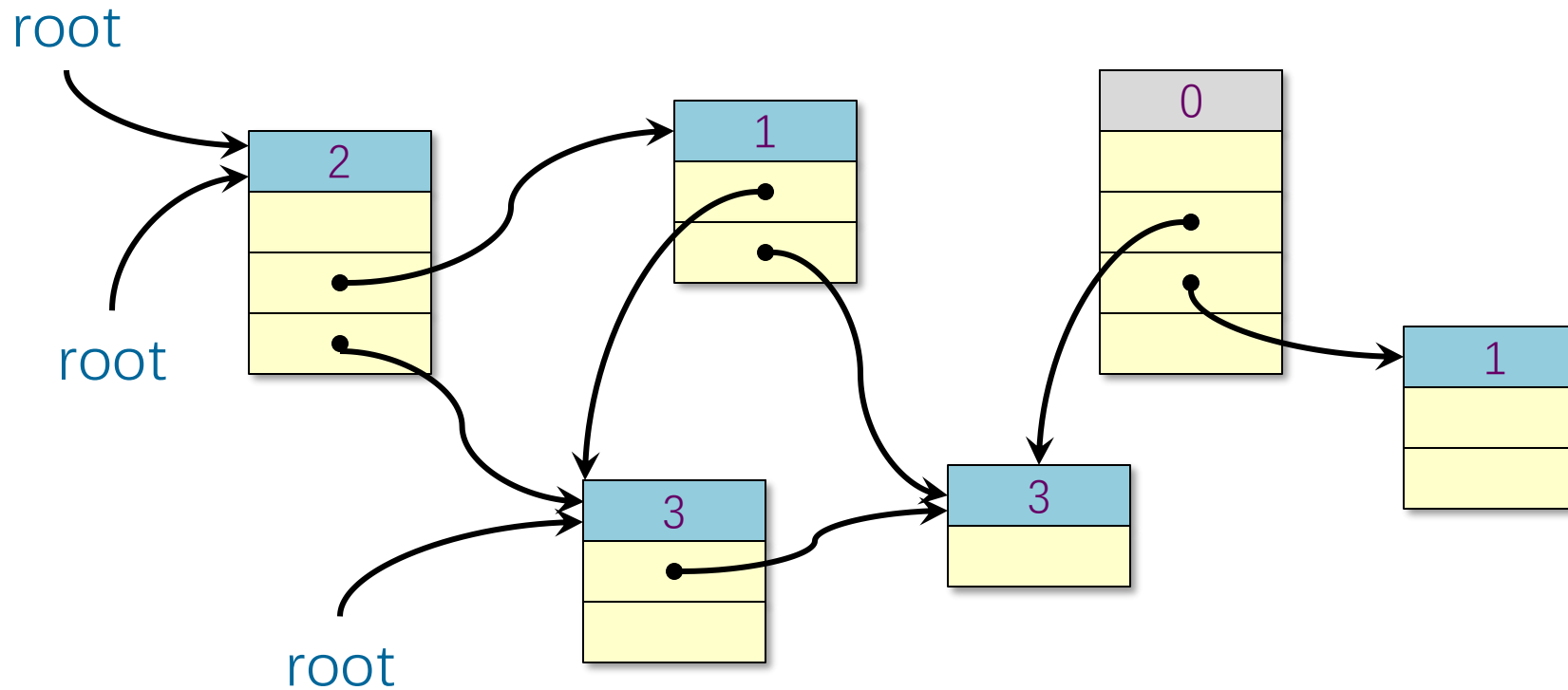
Reference Counting

Keep a count of the number of pointers referencing each object.
If the count drops to **0**, free the dead object.



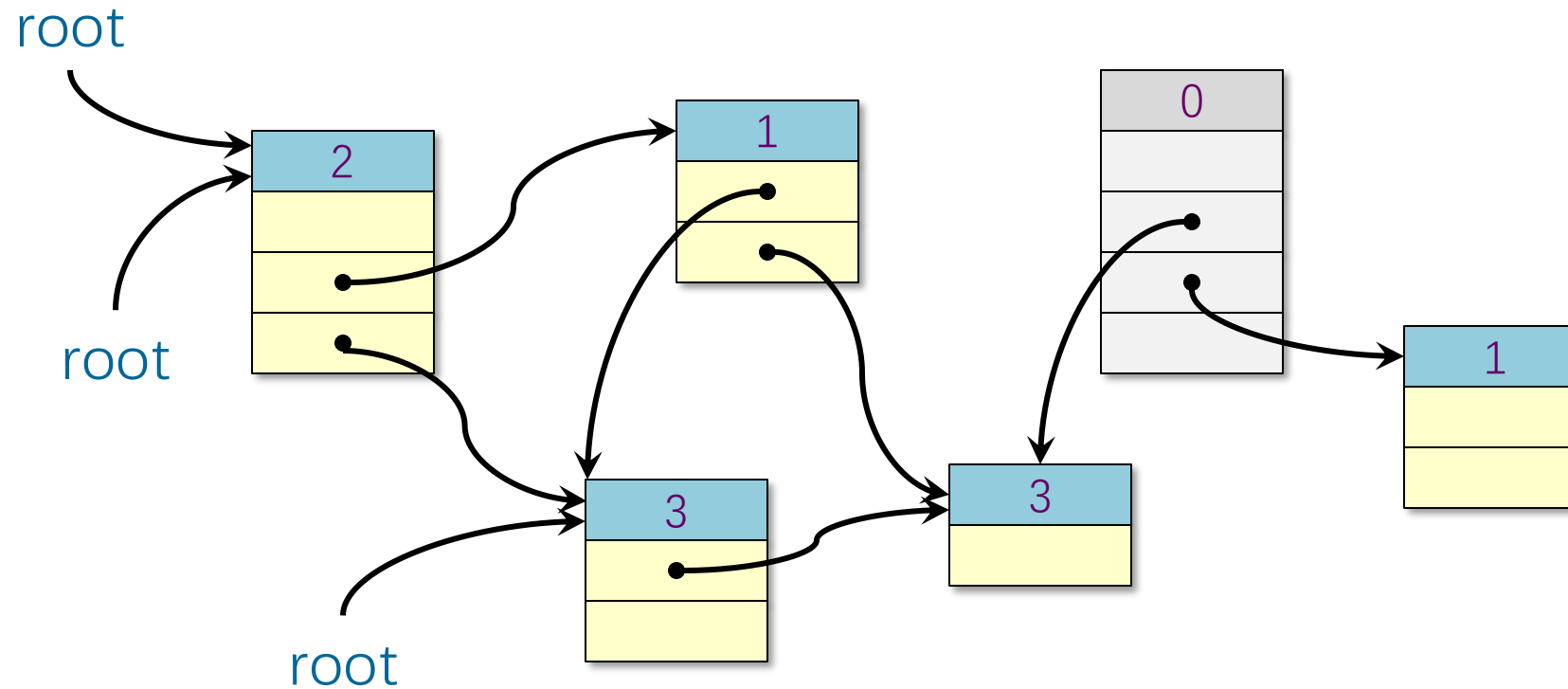
Reference Counting

Keep a count of the number of pointers referencing each object.
If the count drops to **0**, free the dead object.



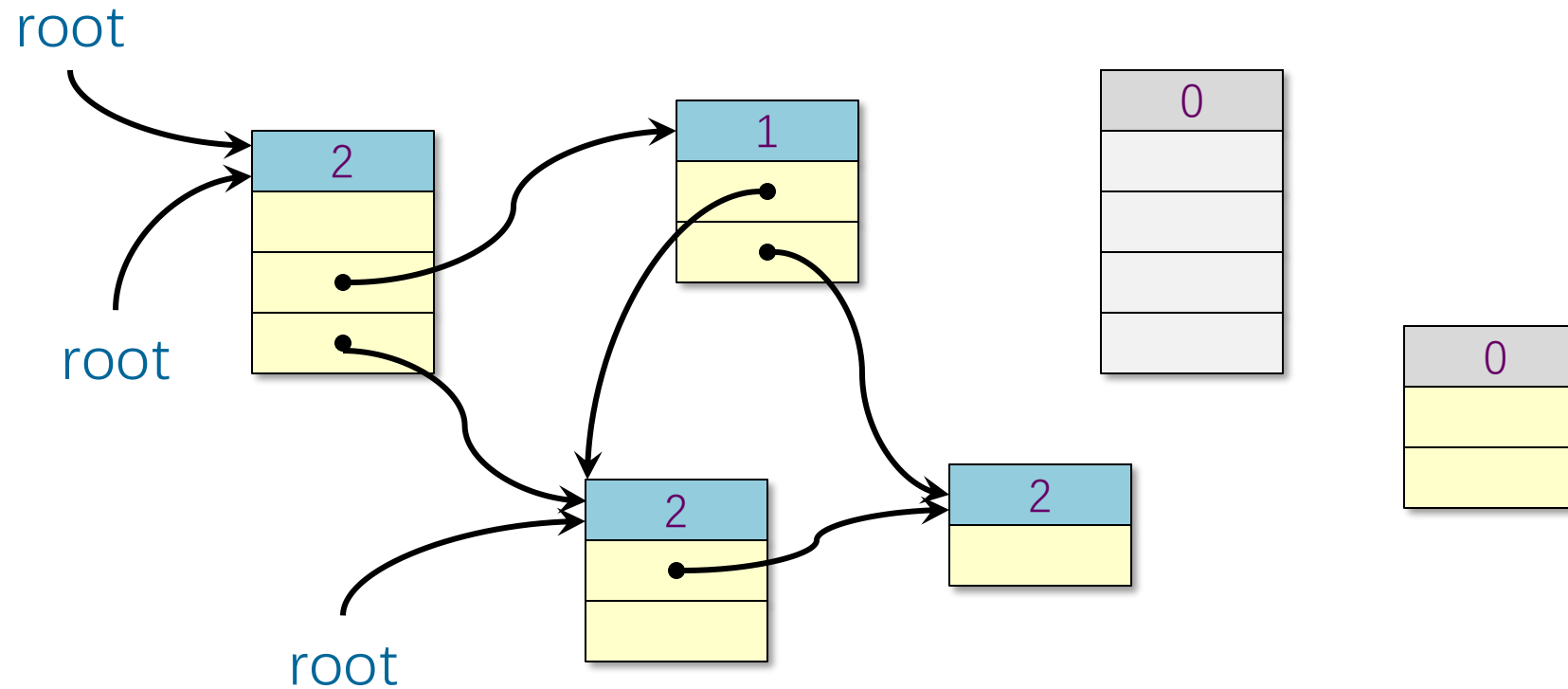
Reference Counting

Keep a count of the number of pointers referencing each object.
If the count drops to **0**, free the dead object.



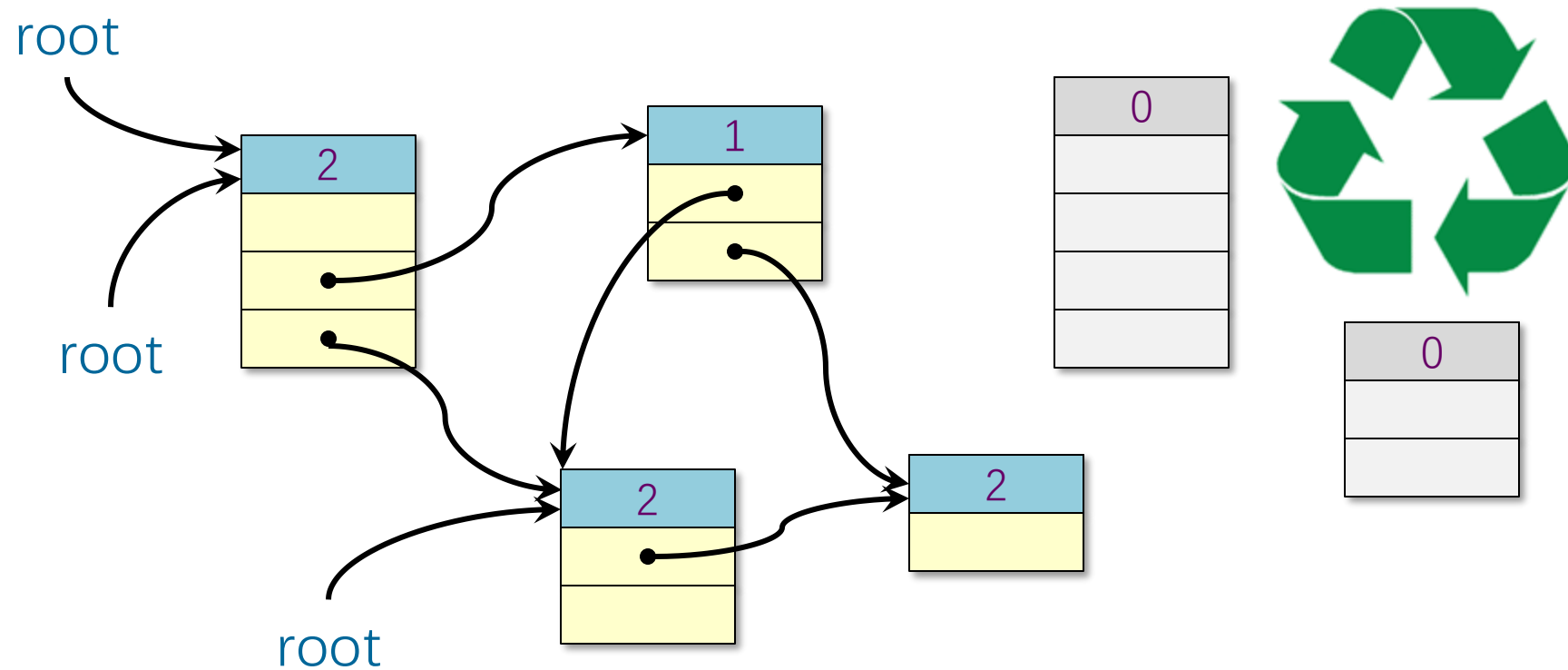
Reference Counting

Keep a count of the number of pointers referencing each object.
If the count drops to **0**, free the dead object.



Reference Counting

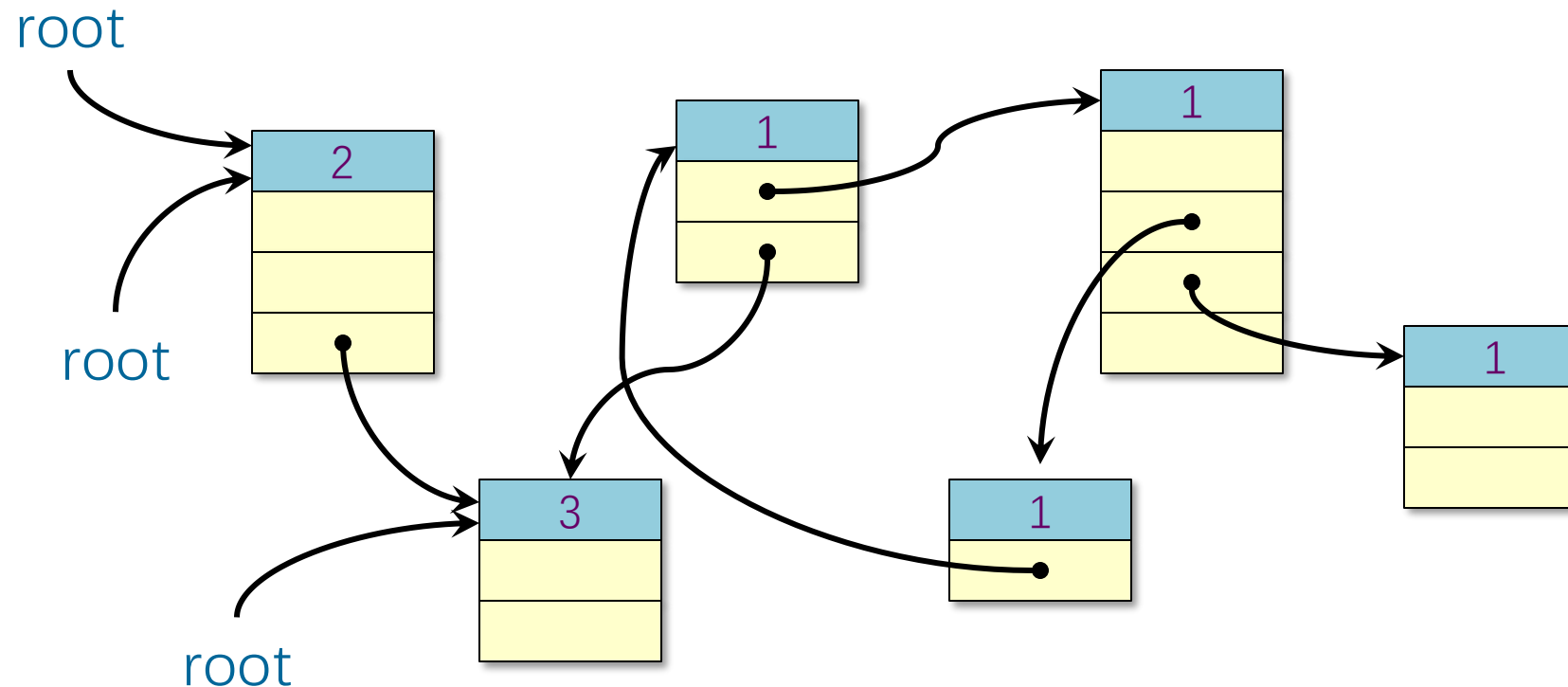
Keep a count of the number of pointers referencing each object.
If the count drops to **0**, free the dead object.



Limitation of Reference Counting

Problem

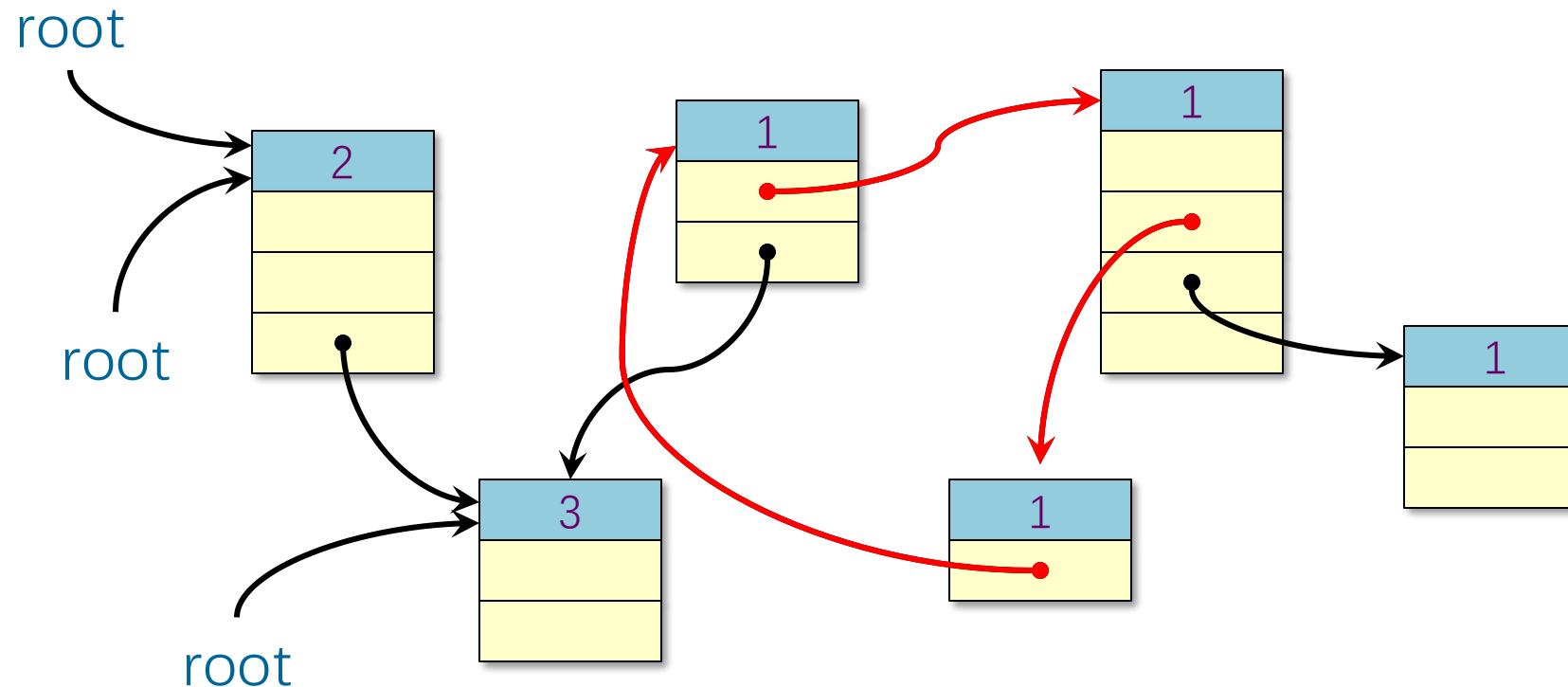
A cycle is never garbage collected!



Limitation of Reference Counting

Problem

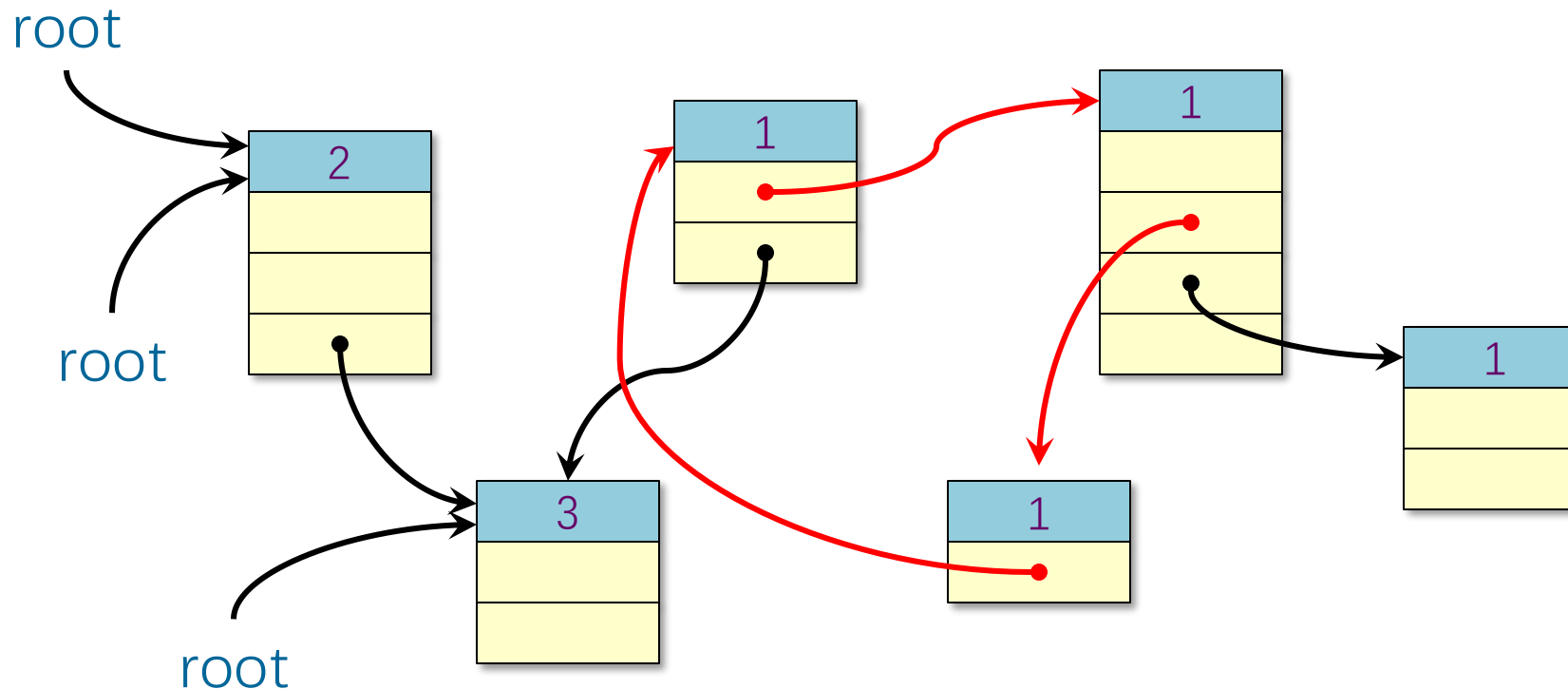
A cycle is never garbage collected!



Limitation of Reference Counting

Problem

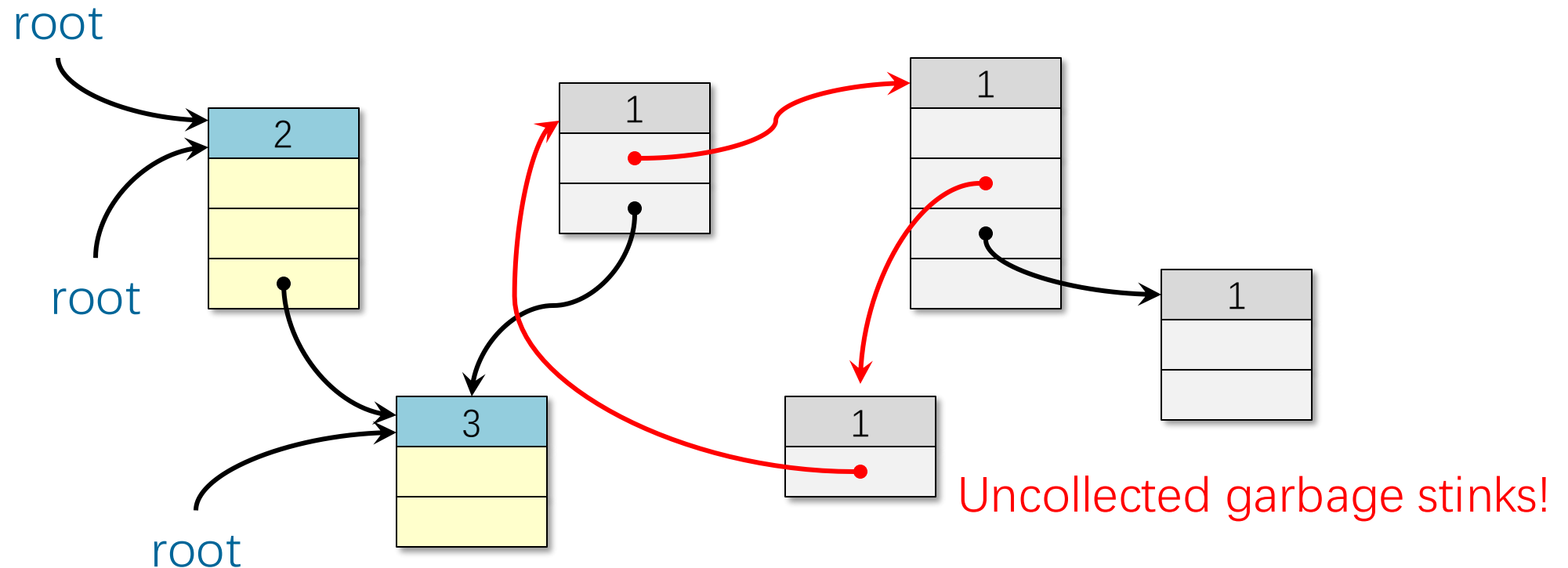
A cycle is never garbage collected!



Limitation of Reference Counting

Problem

A cycle is never garbage collected!



Nevertheless, reference counting works well for acyclic structures.

Summary

	Manual	Reference Counting		
Ease of Use	Bad	Medium		
Throughput	Good	Medium		
Latency	Good	Good		
External Fragmentation	Bad	Bad		
Example	C Malloc/Free	C++ std::shared_ptr		

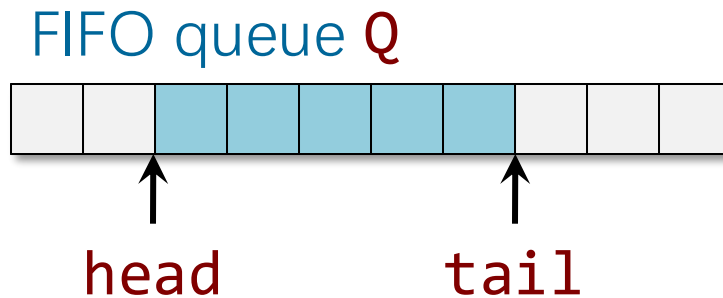
MARK-AND-SWEEP GARBAGE COLLECTION



Graph Abstraction

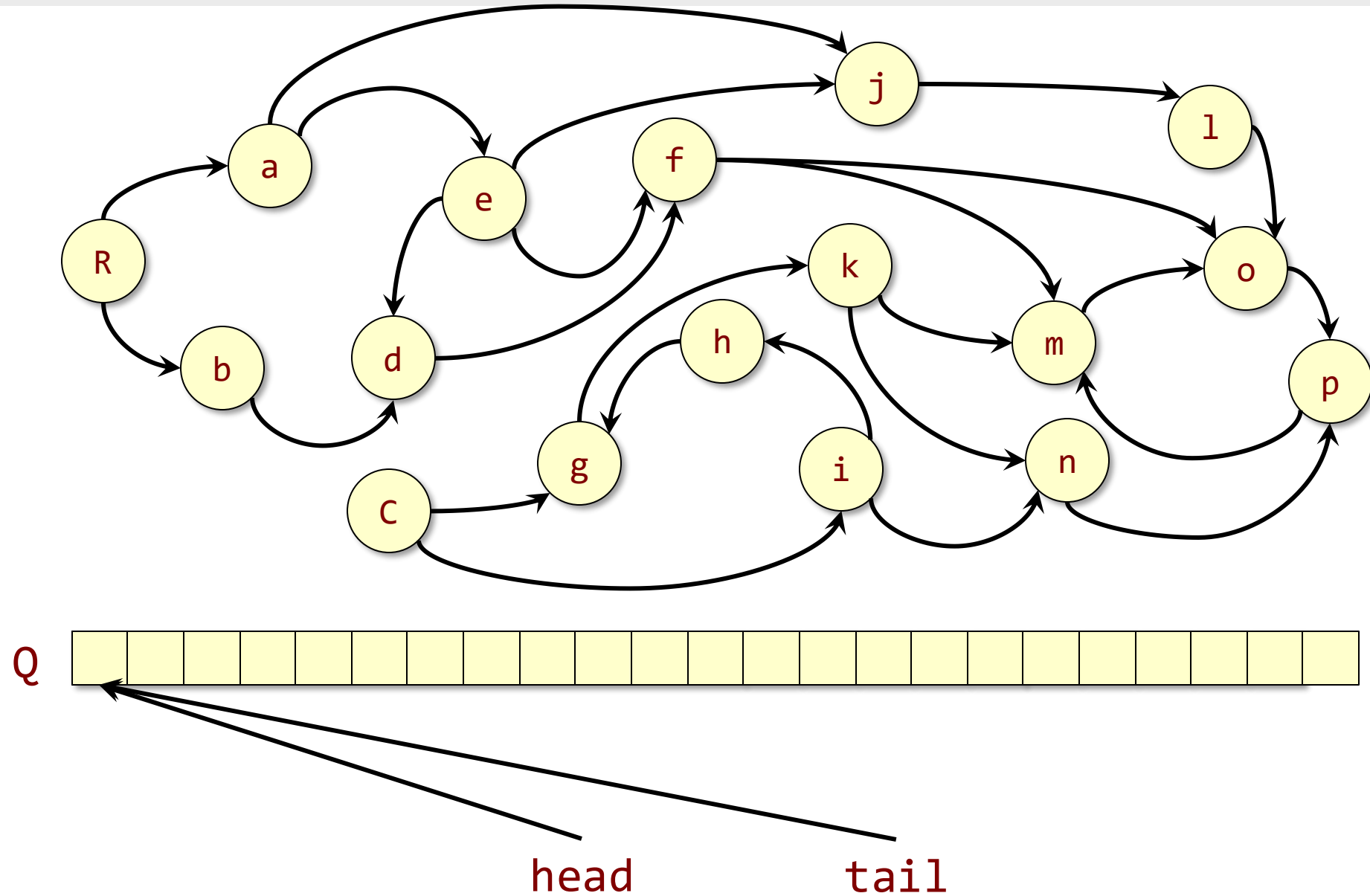
Idea

- Objects and pointers form a directed graph $G = (V, E)$.
- Live objects are reachable from roots
- How to find the live objects?
 - ▣ Breadth-first search (BFS)

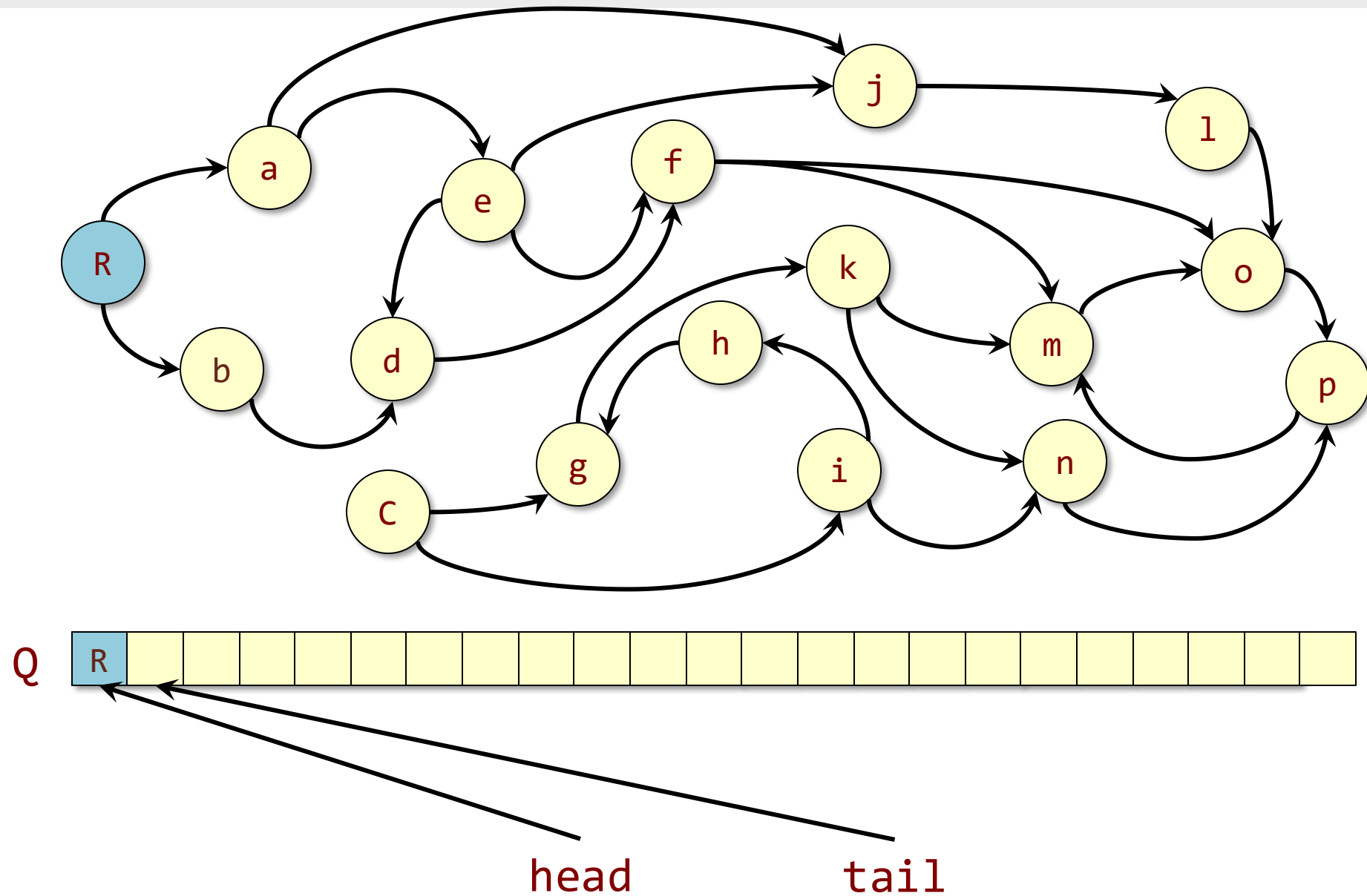


```
for (v ∈ V) {  
  if (root(v)) {  
    v.mark = 1;  
    enqueue(Q, v);  
  } else v.mark = 0;  
}  
while (Q != ∅) {  
  u = dequeue(Q);  
  for (v ∈ V such that (u,v) ∈ E) {  
    if (v.mark == 0) {  
      v.mark = 1;  
      enqueue(Q, v);  
    }  
  }  
}
```

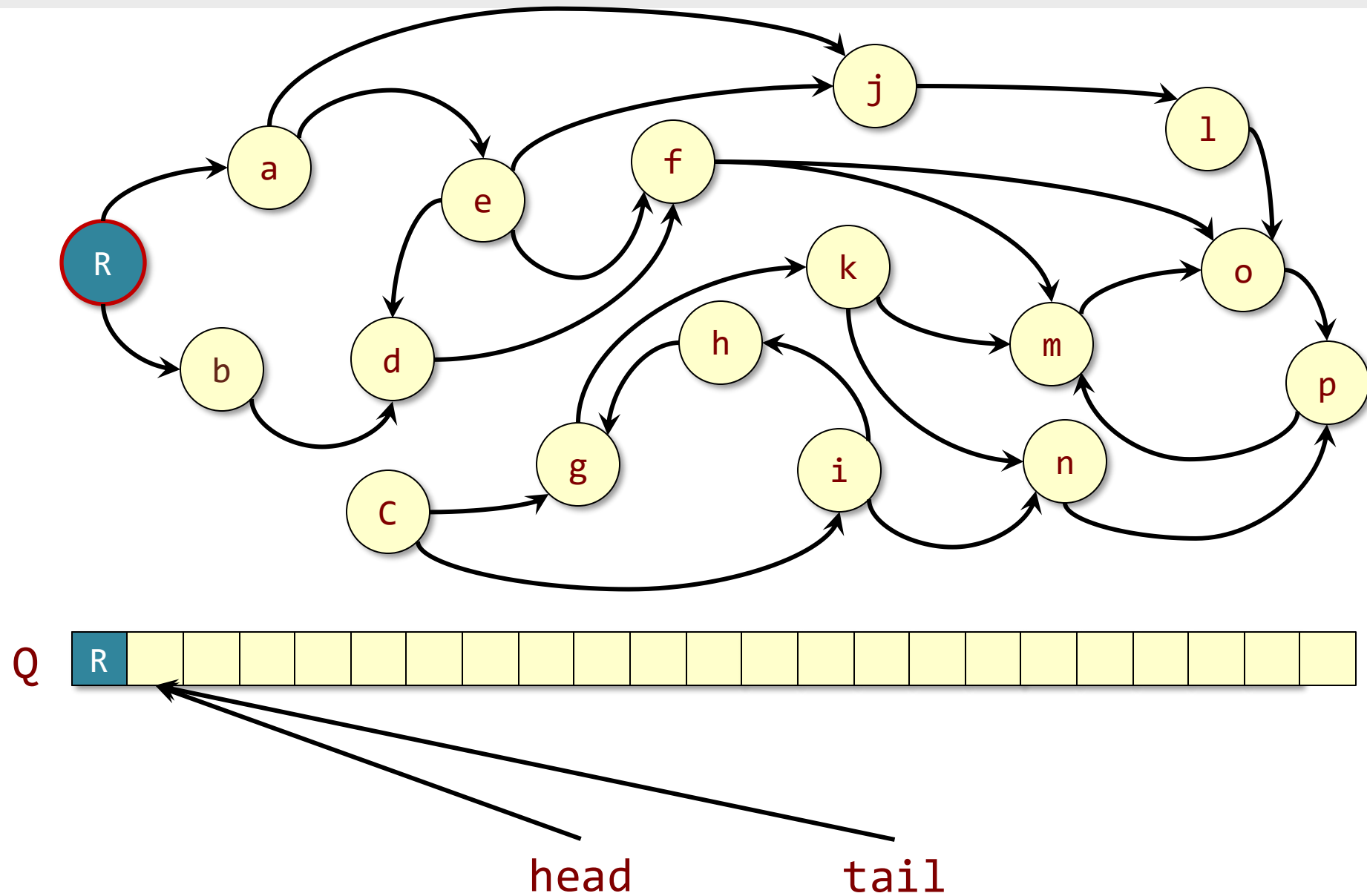
Breadth-First Search



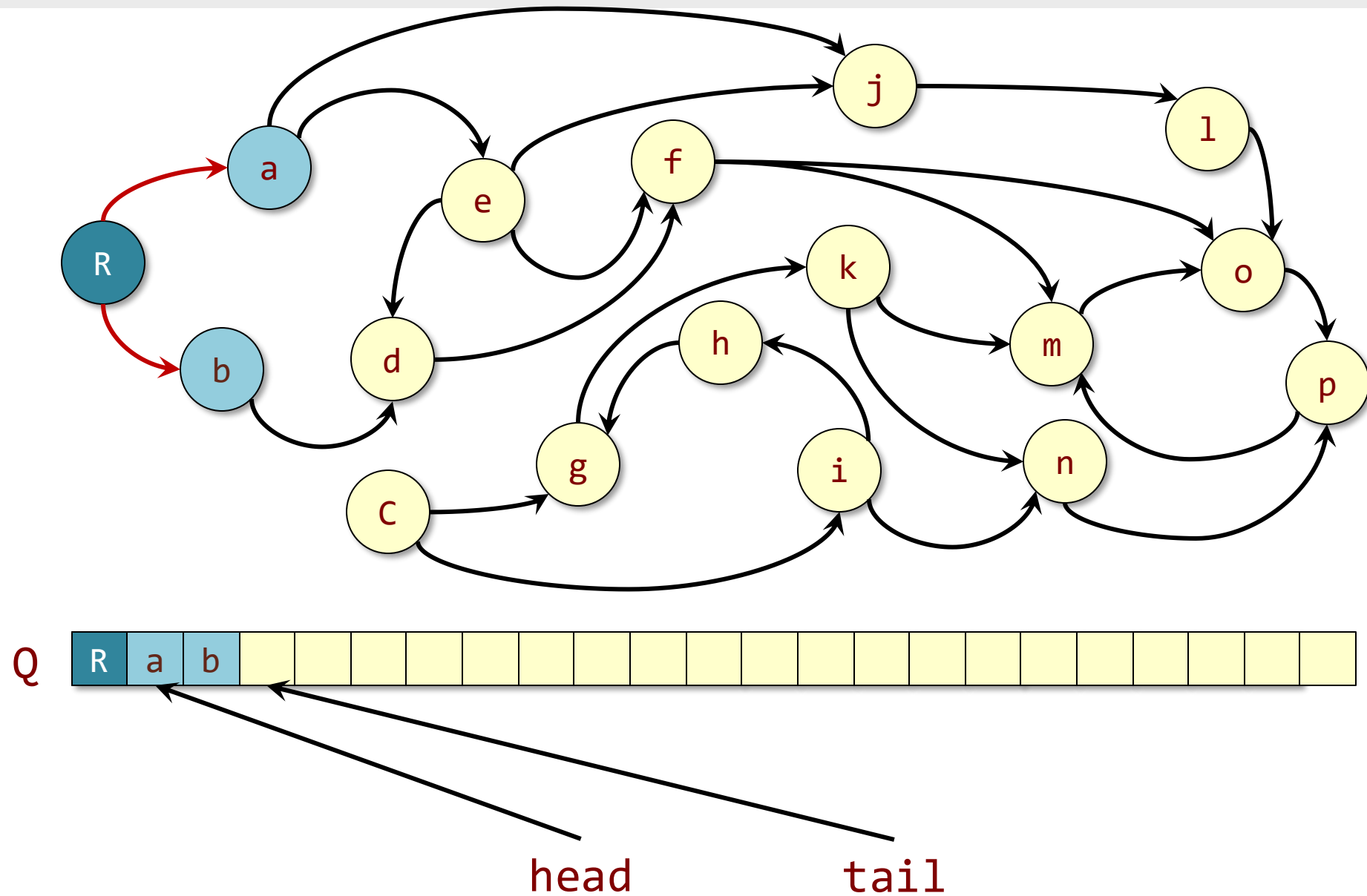
Breadth-First Search



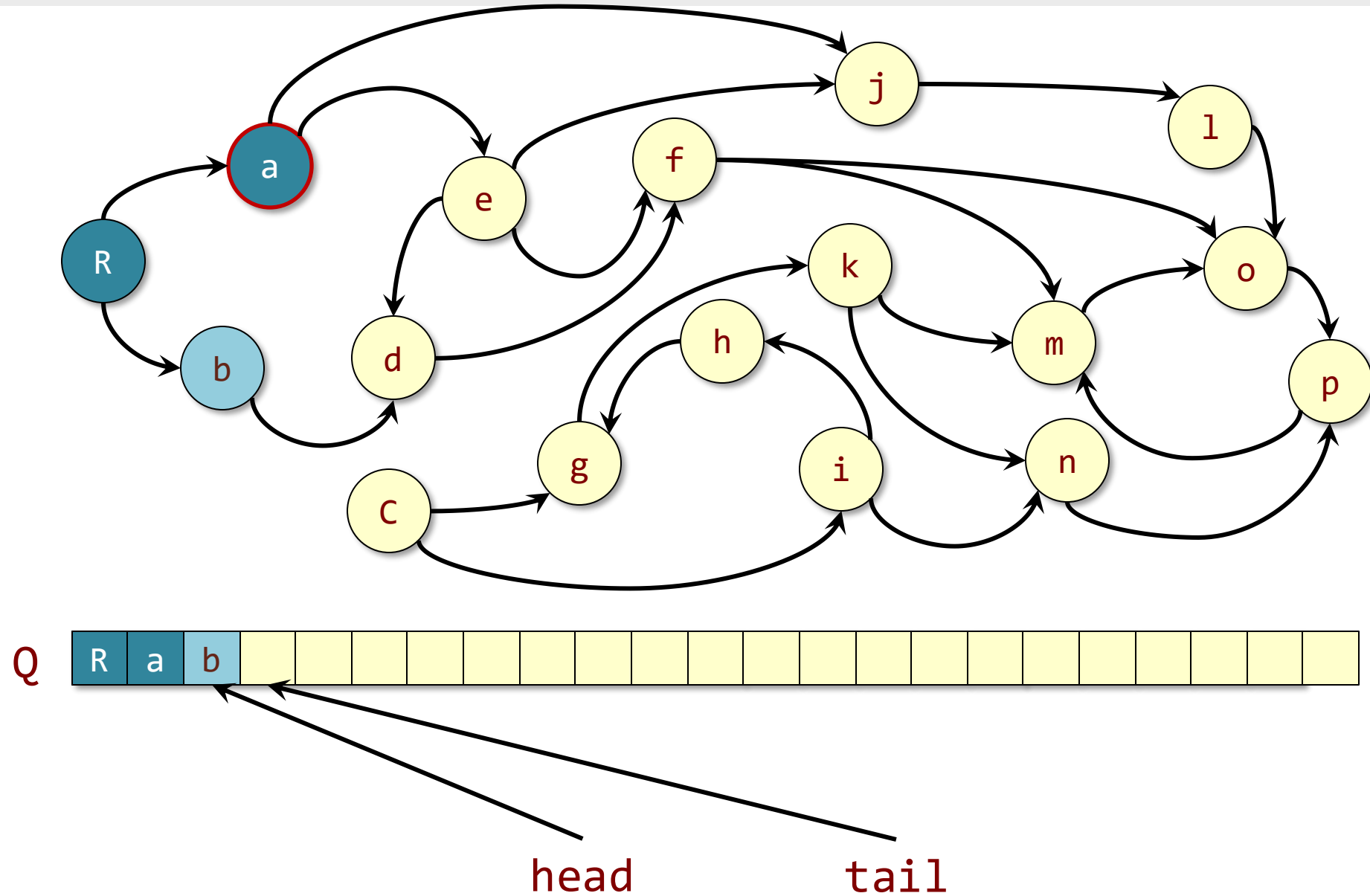
Breadth-First Search



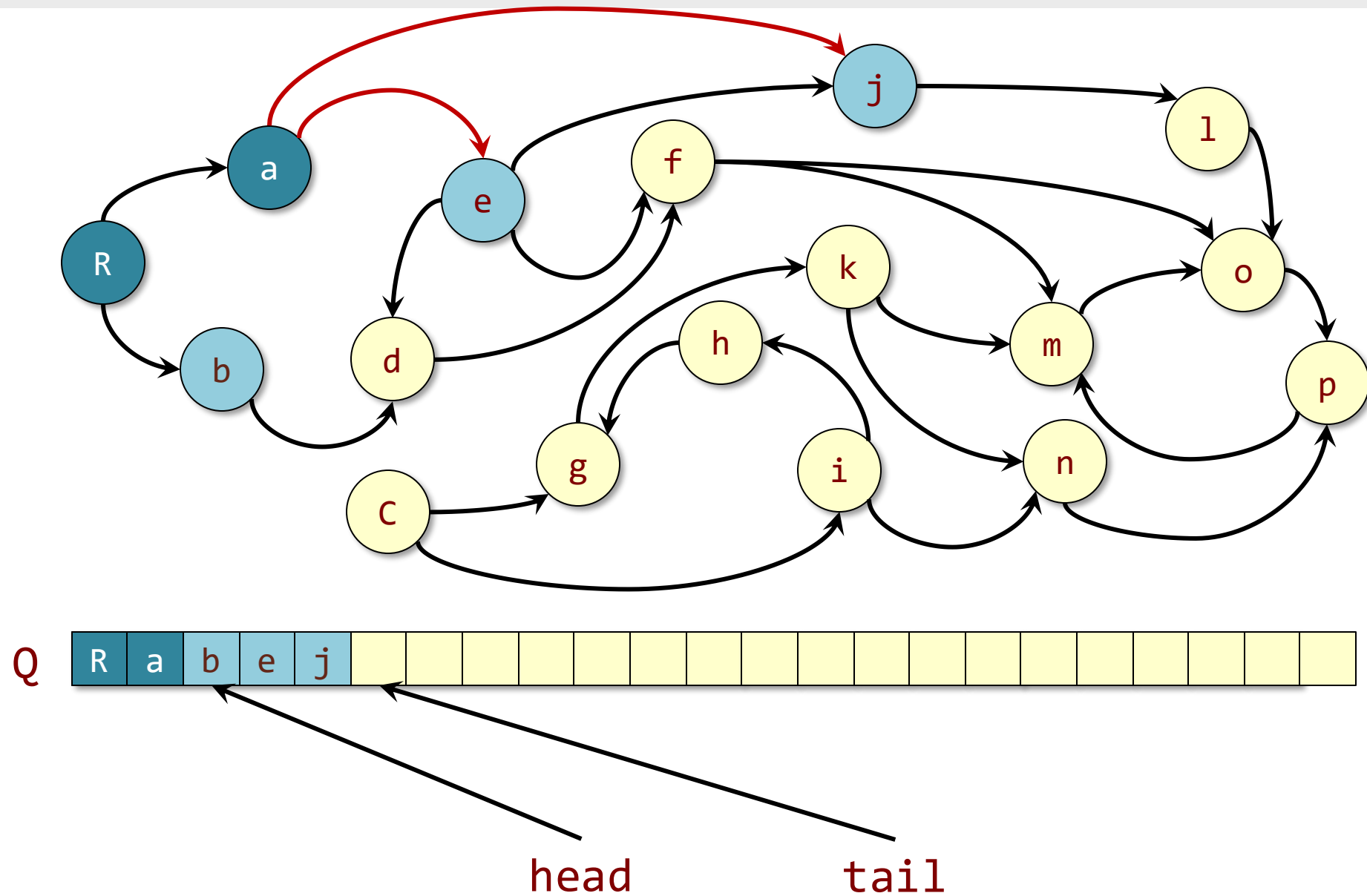
Breadth-First Search



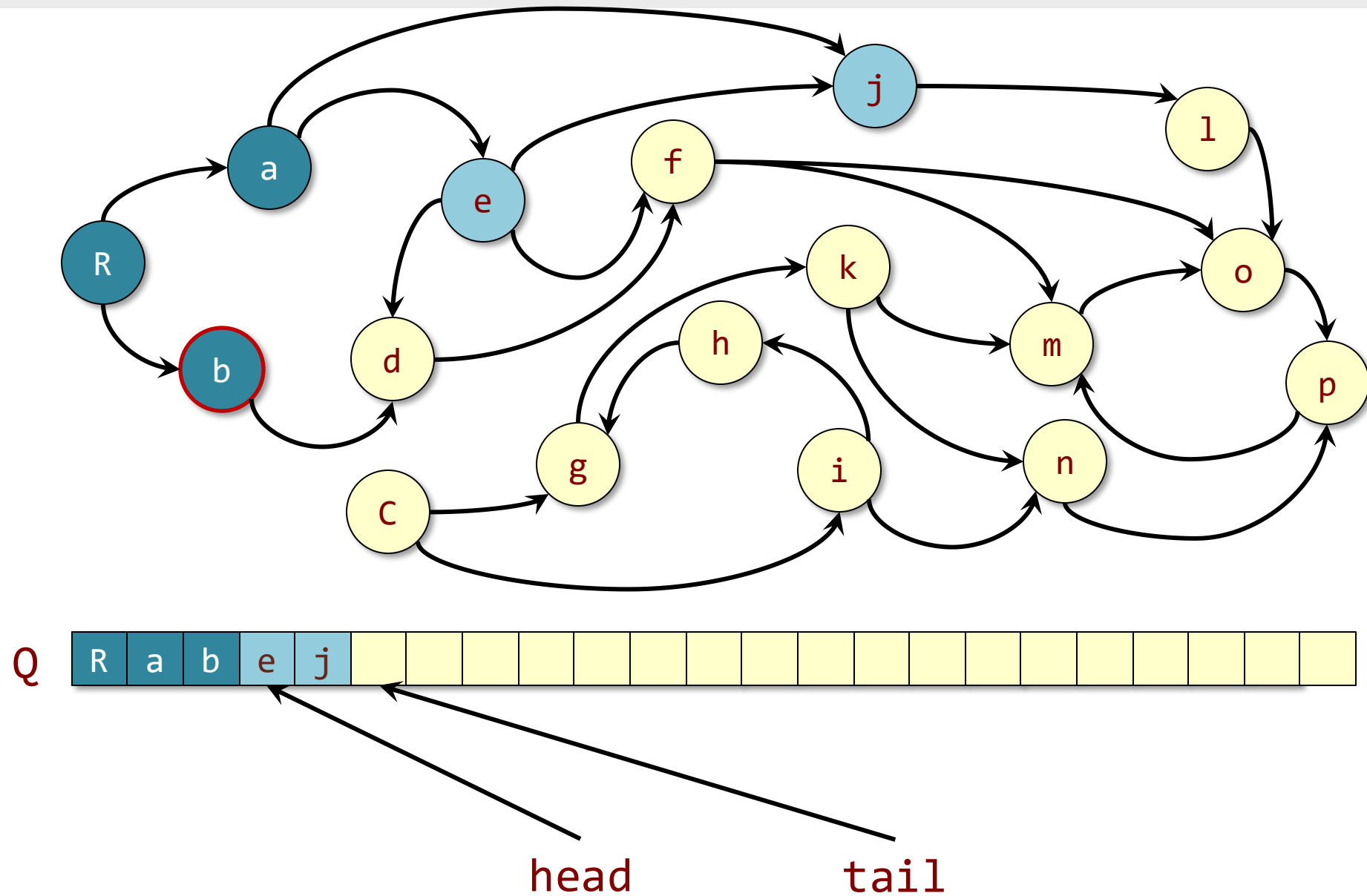
Breadth-First Search



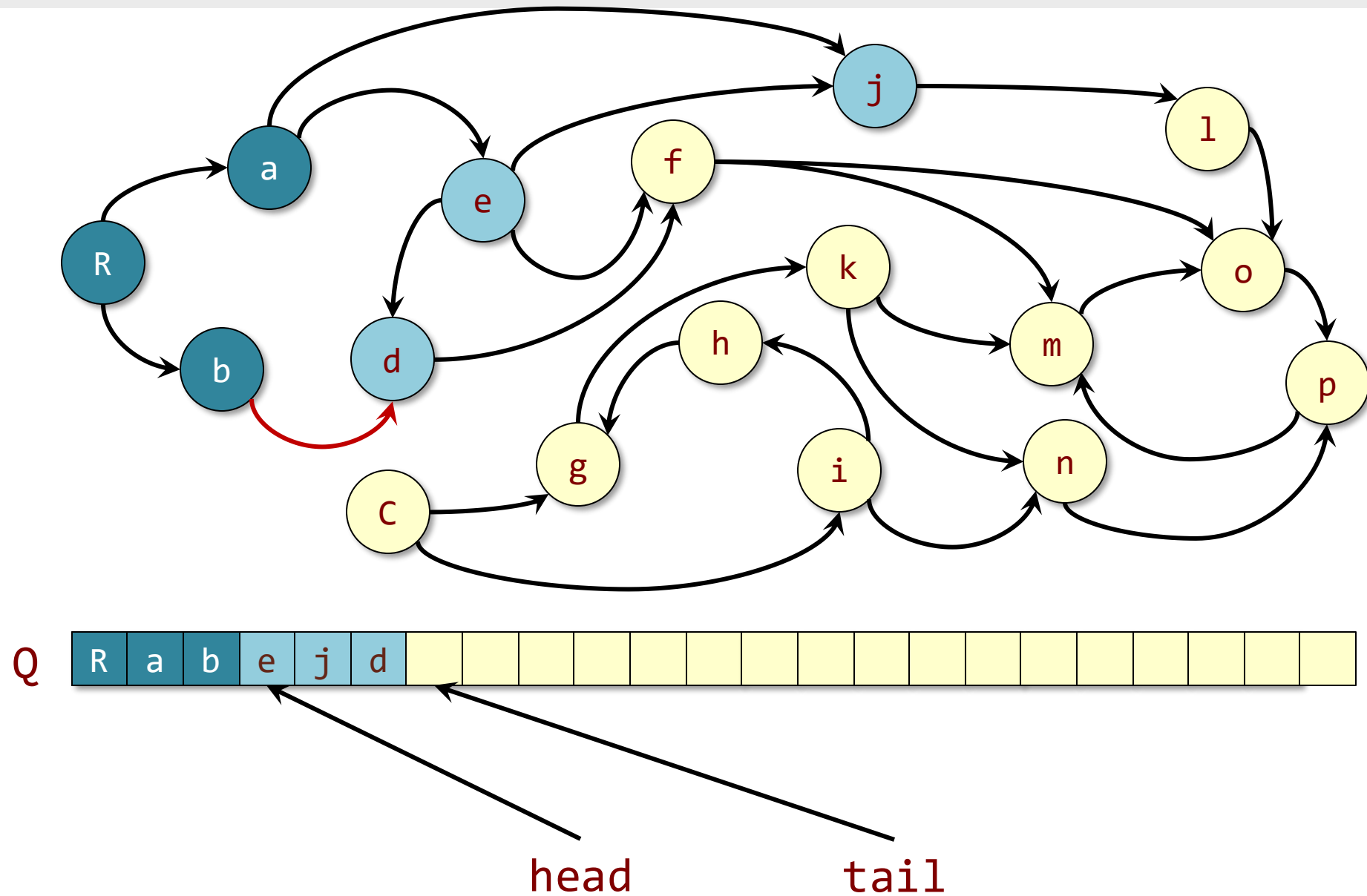
Breadth-First Search



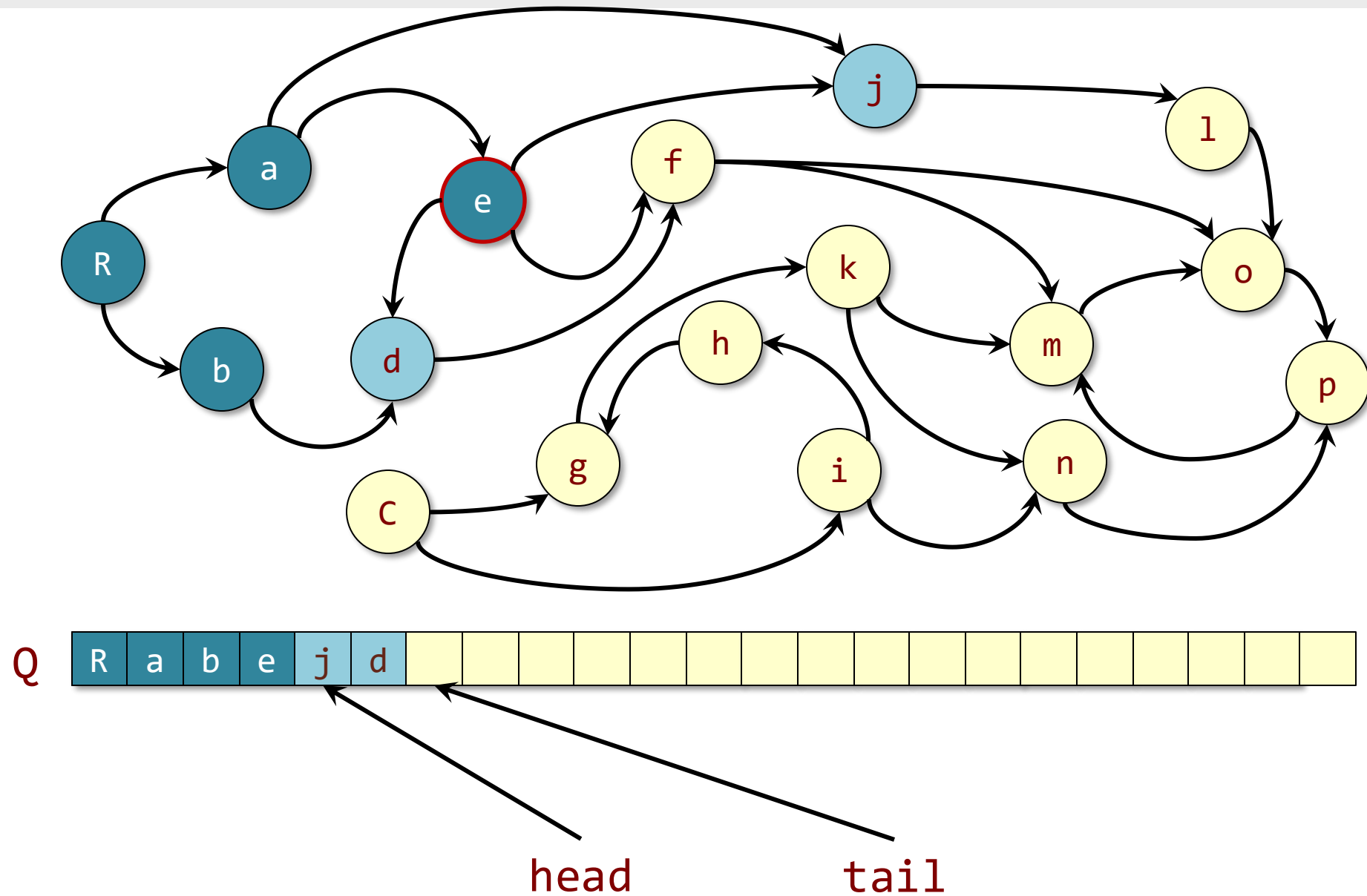
Breadth-First Search



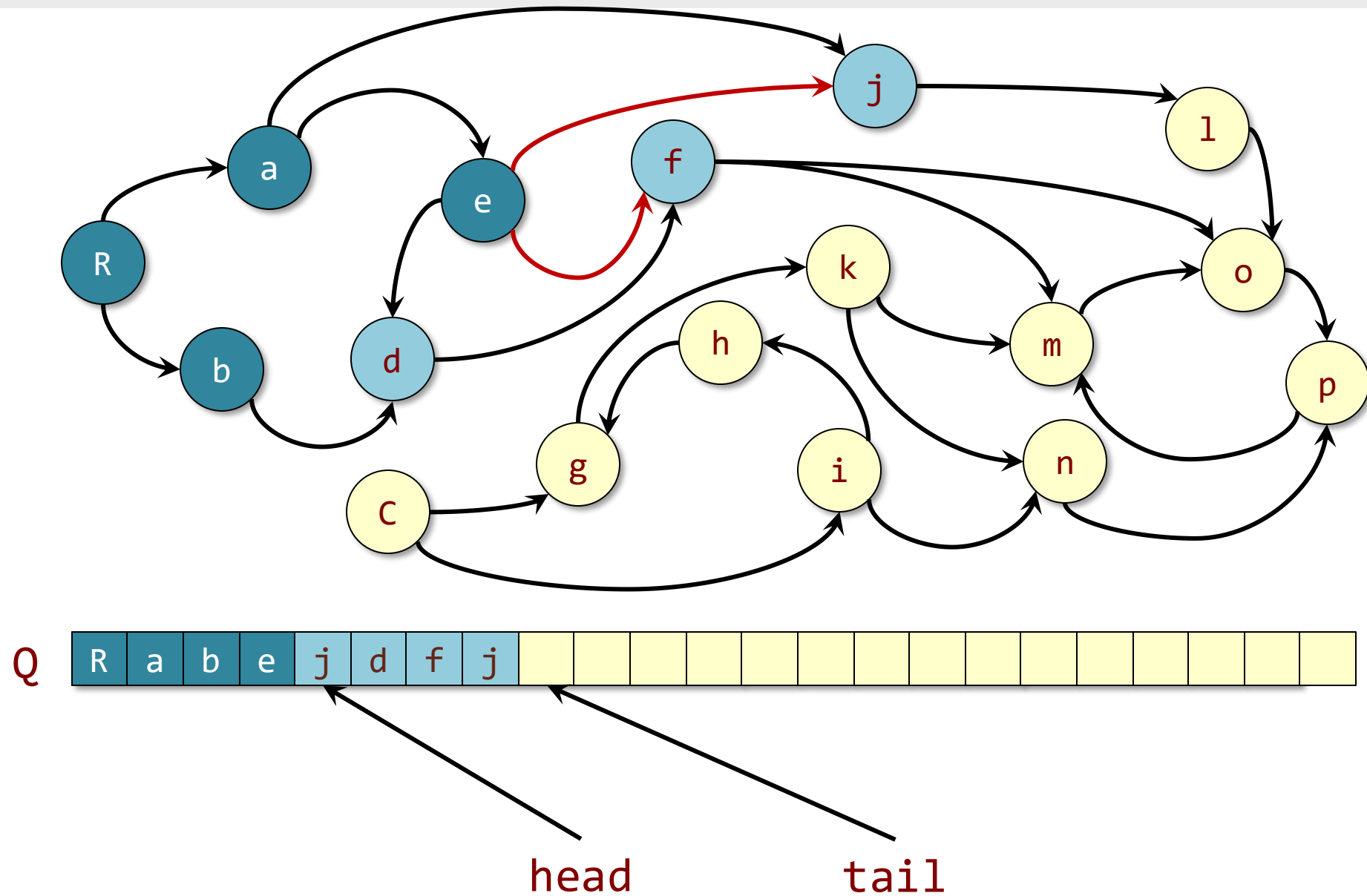
Breadth-First Search



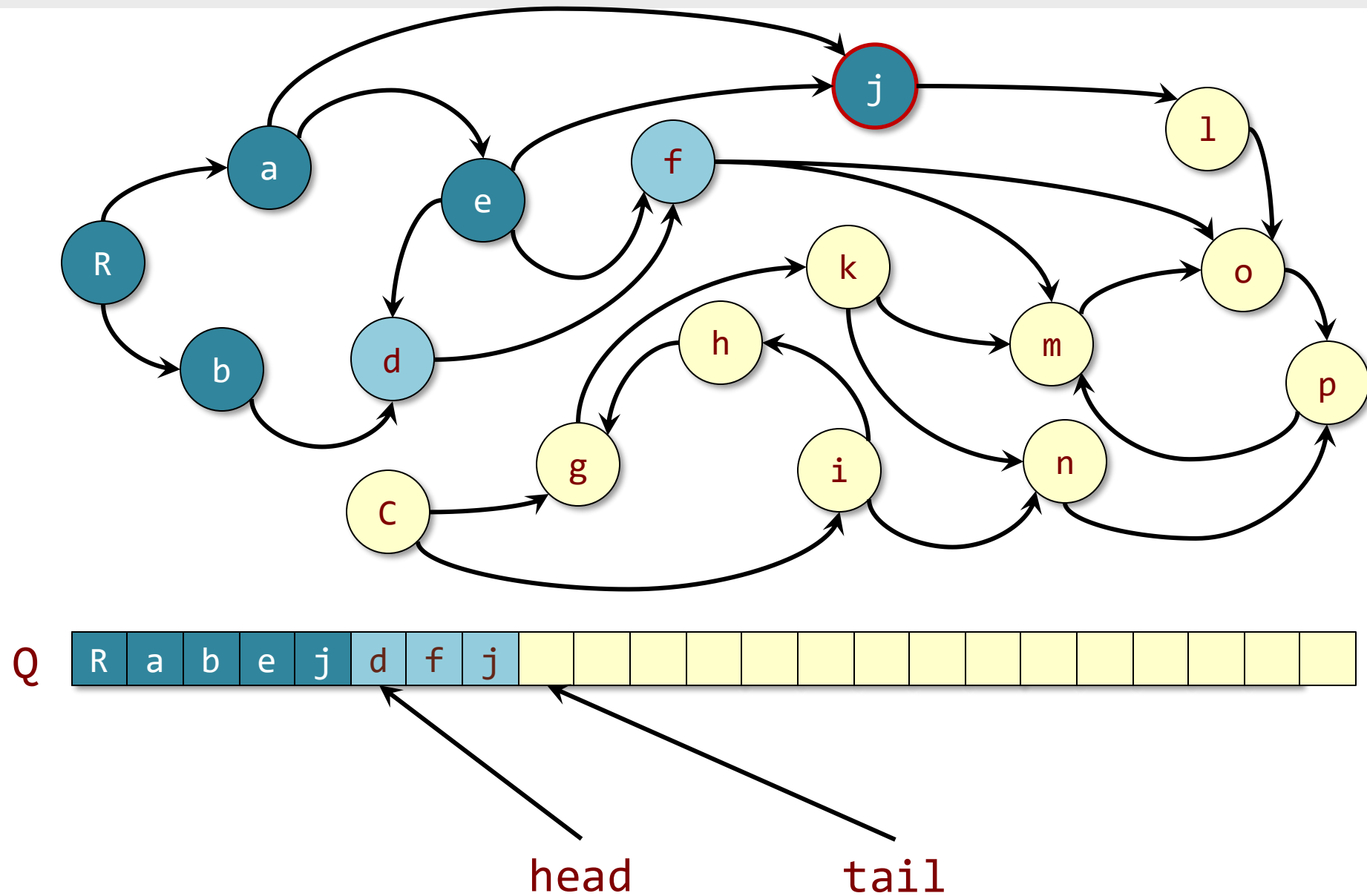
Breadth-First Search



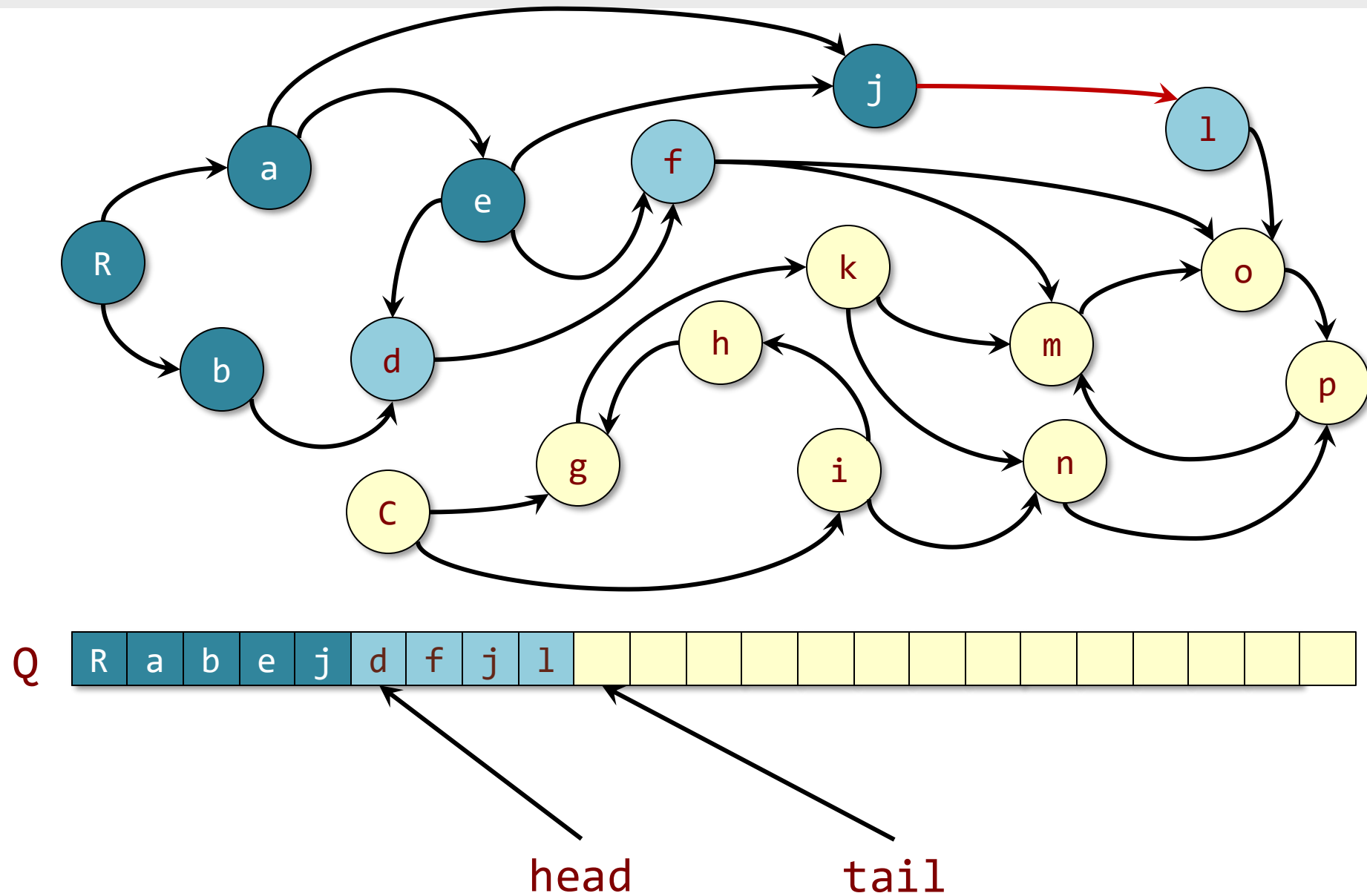
Breadth-First Search



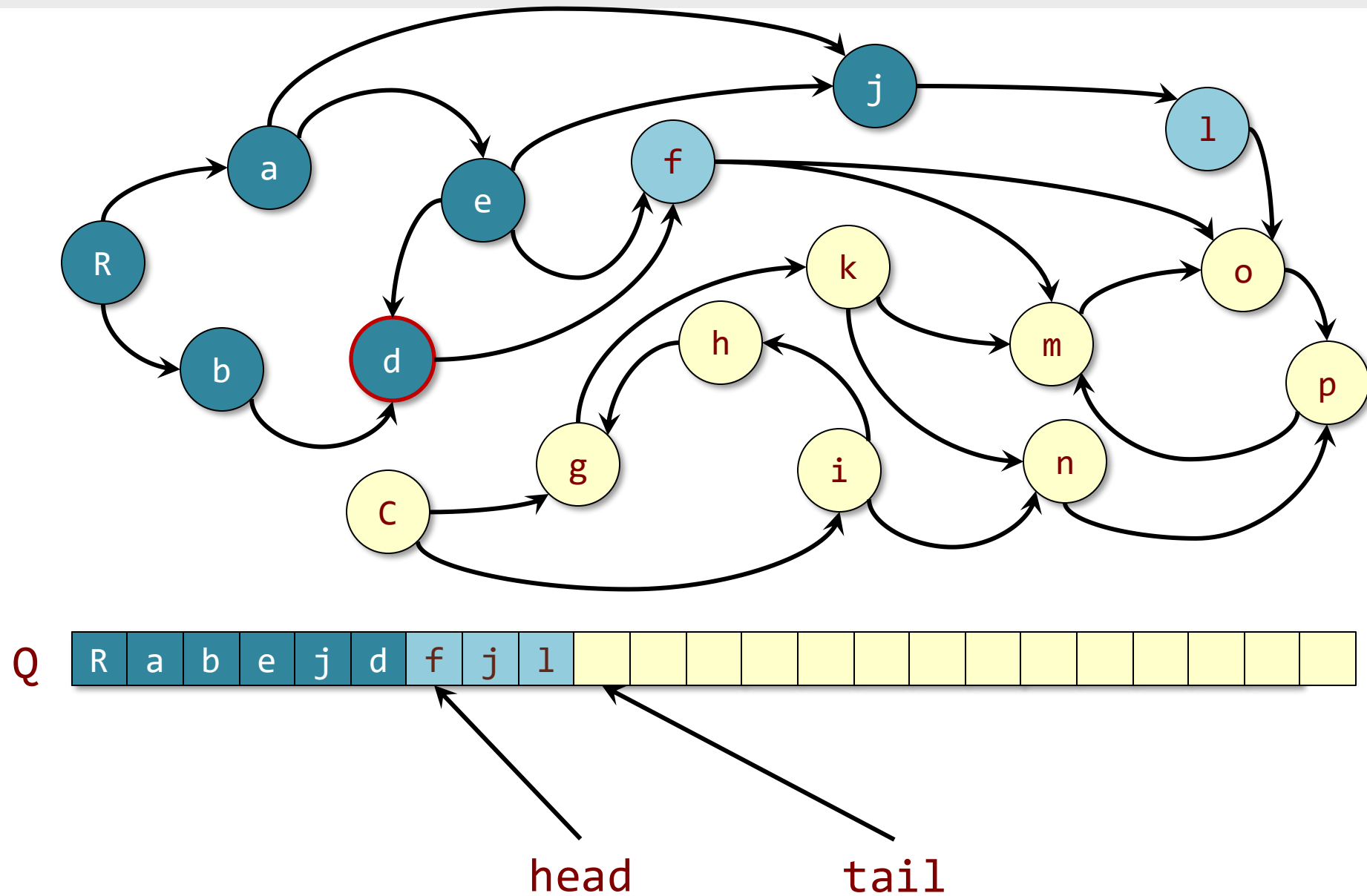
Breadth-First Search



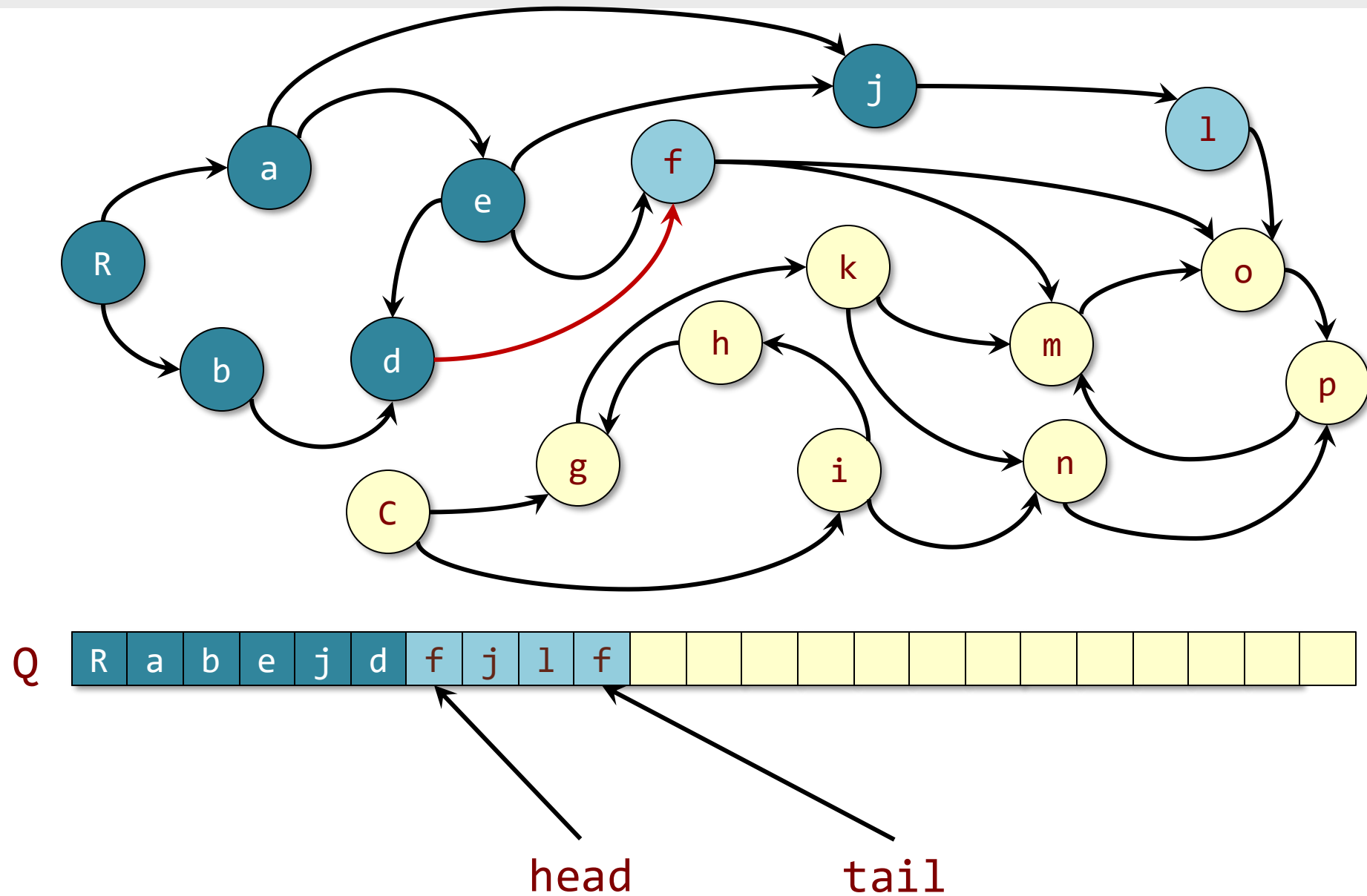
Breadth-First Search



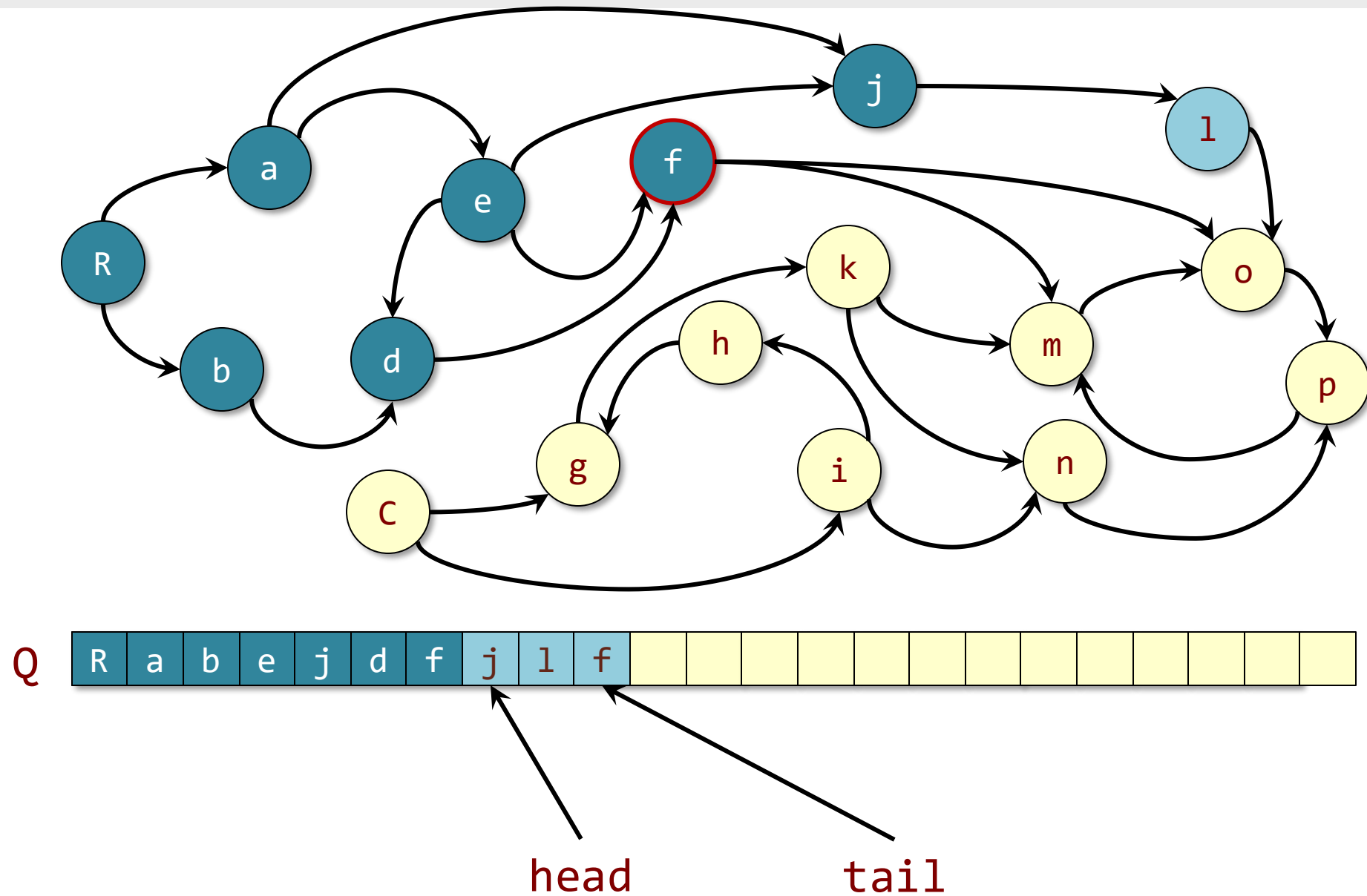
Breadth-First Search



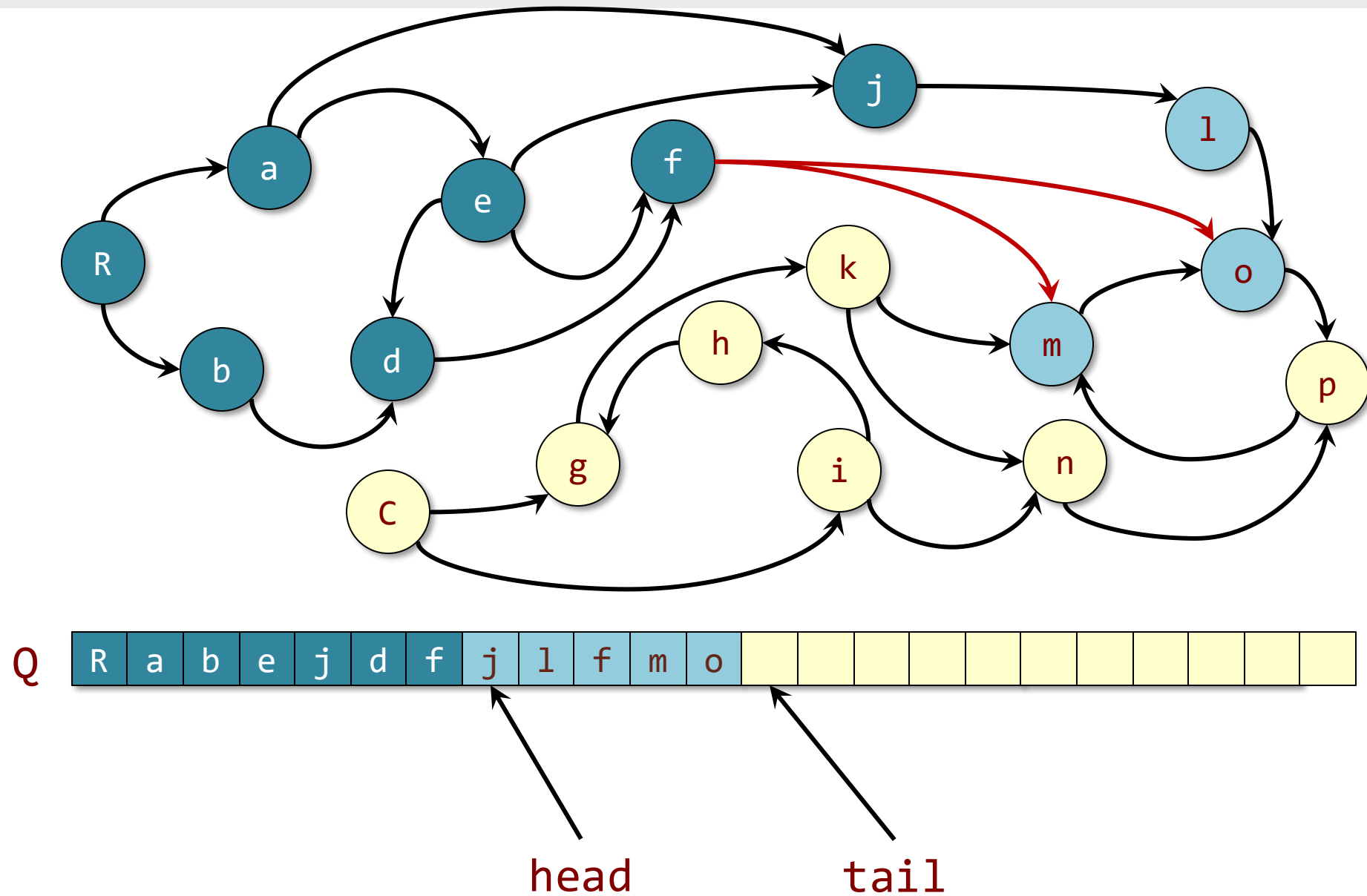
Breadth-First Search



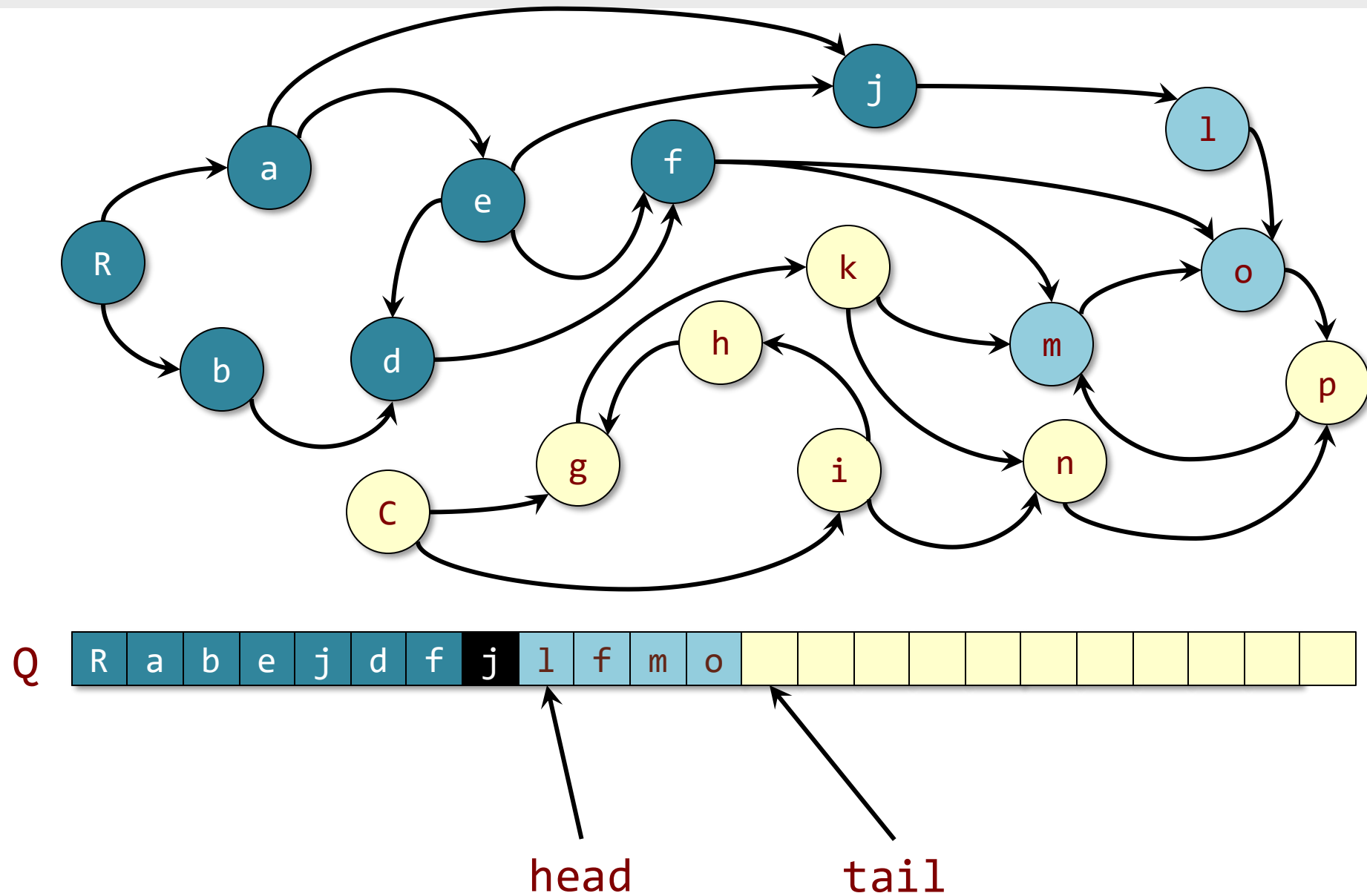
Breadth-First Search



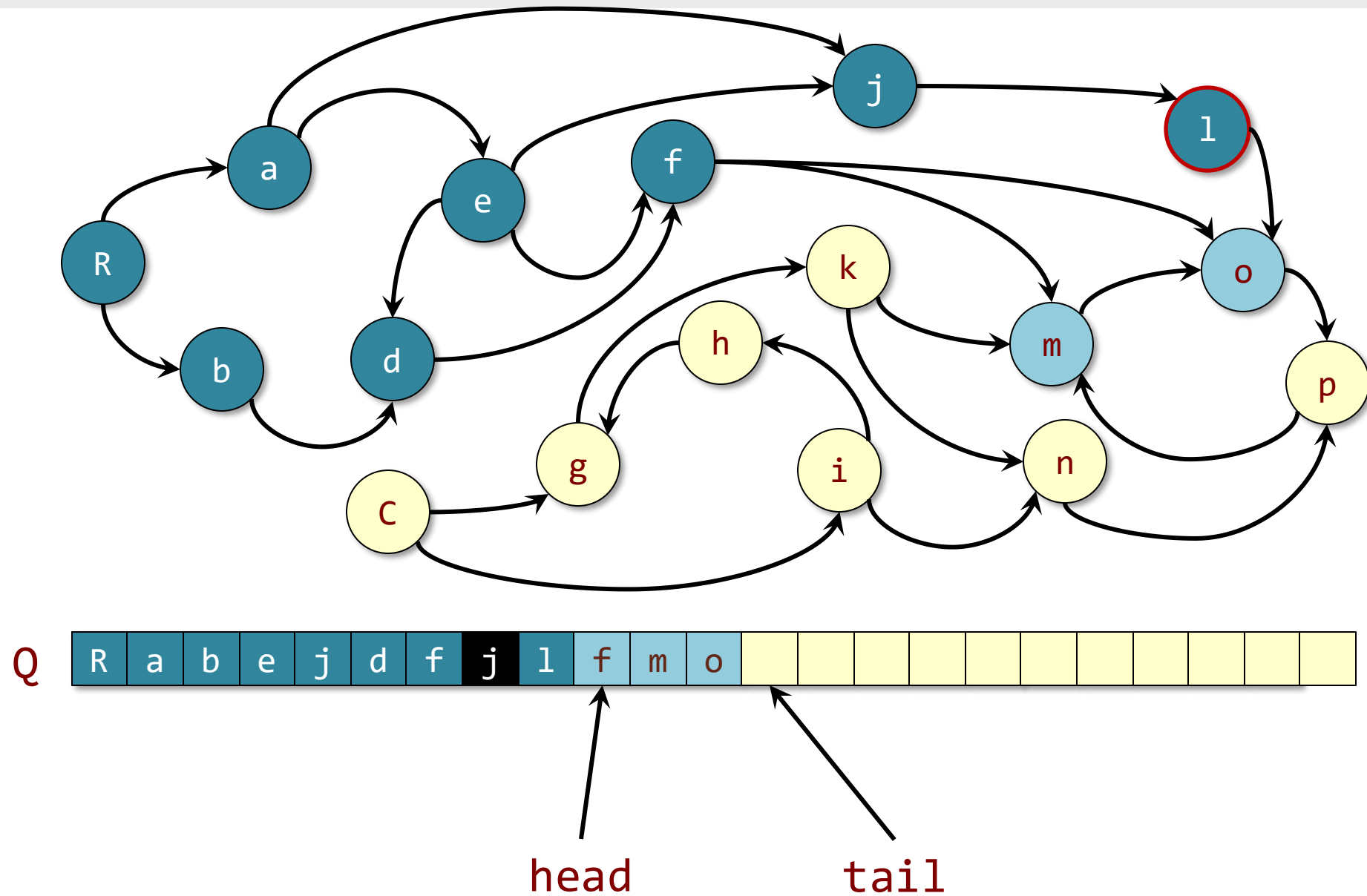
Breadth-First Search



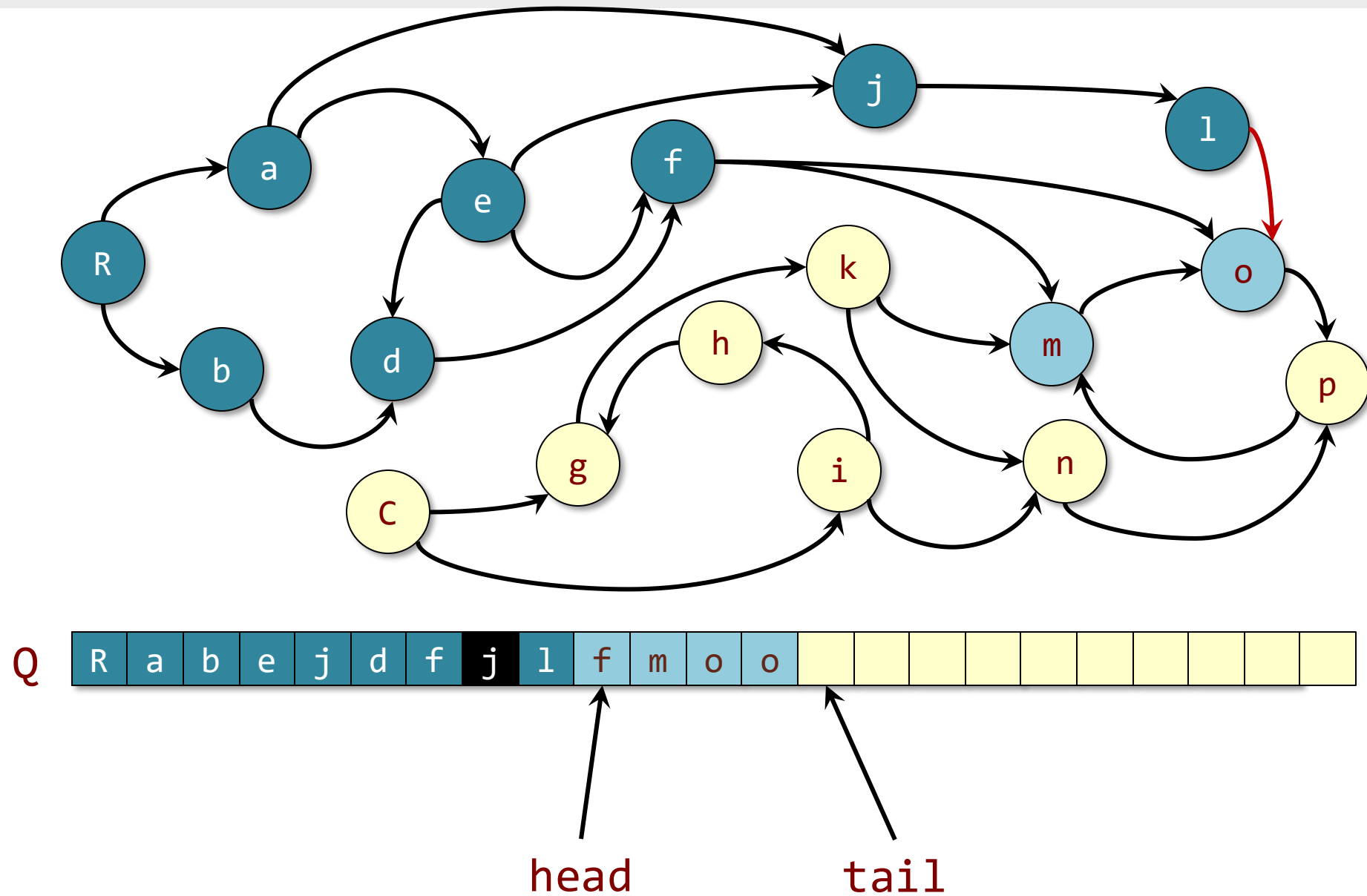
Breadth-First Search



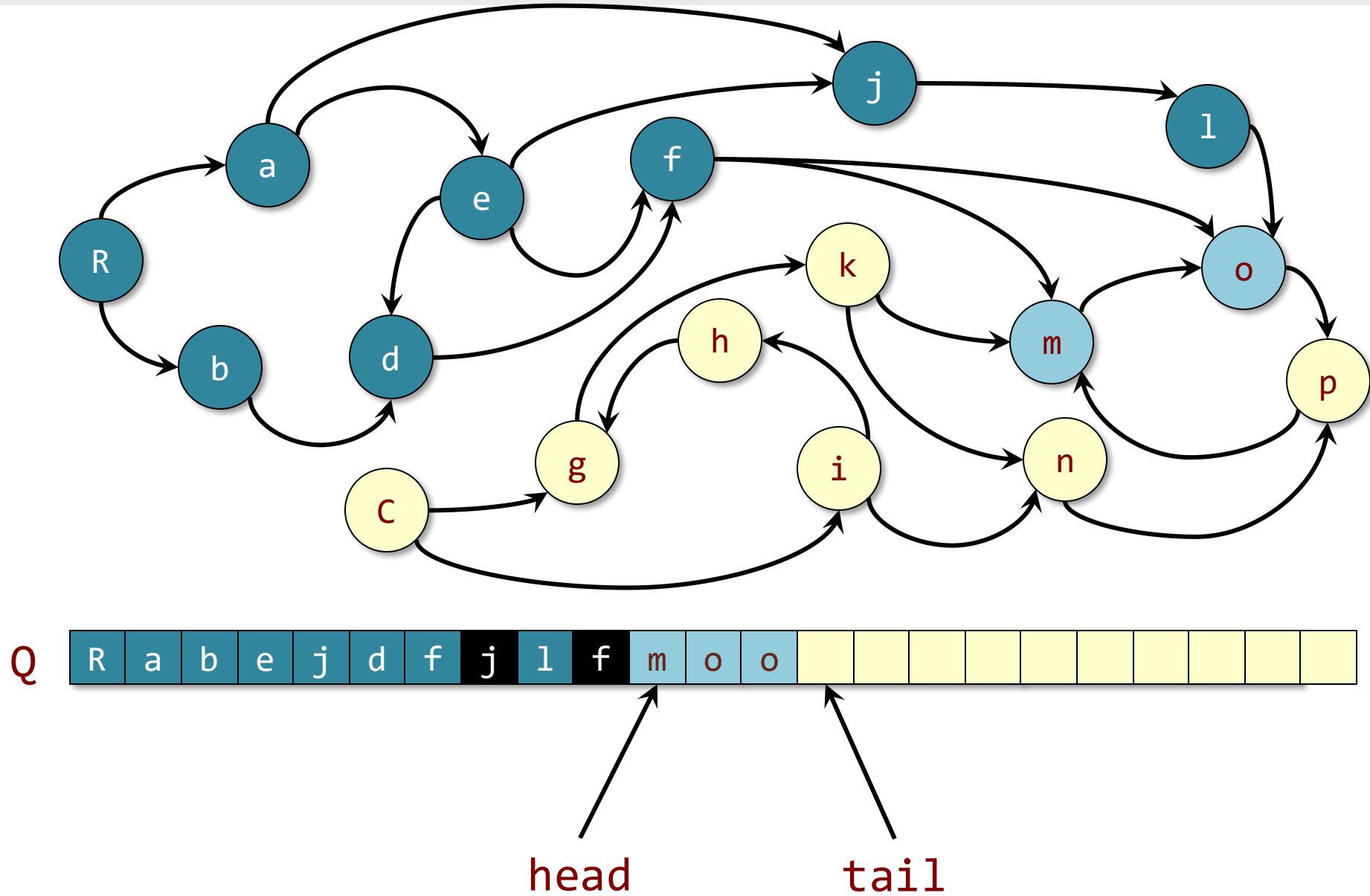
Breadth-First Search



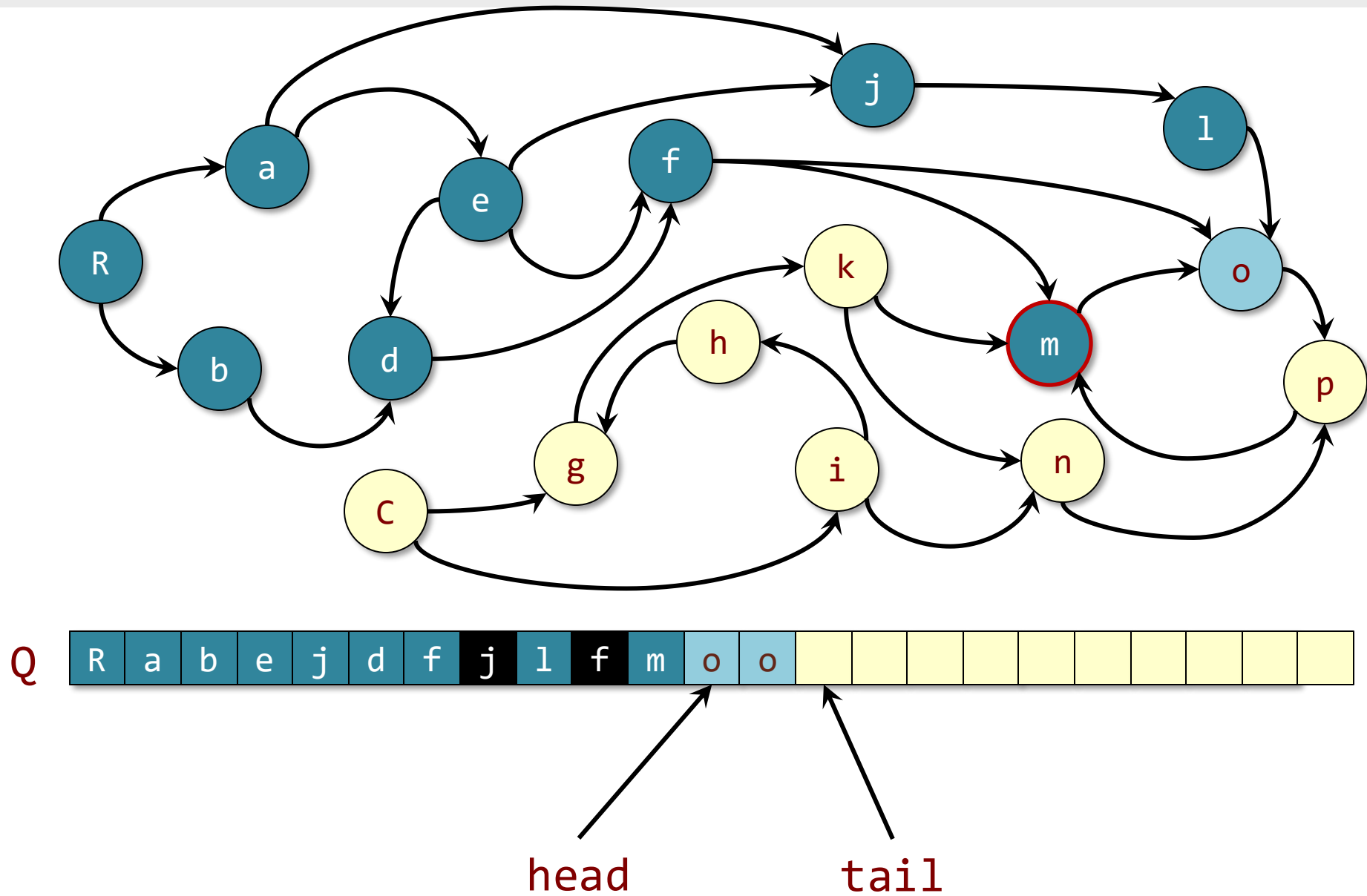
Breadth-First Search



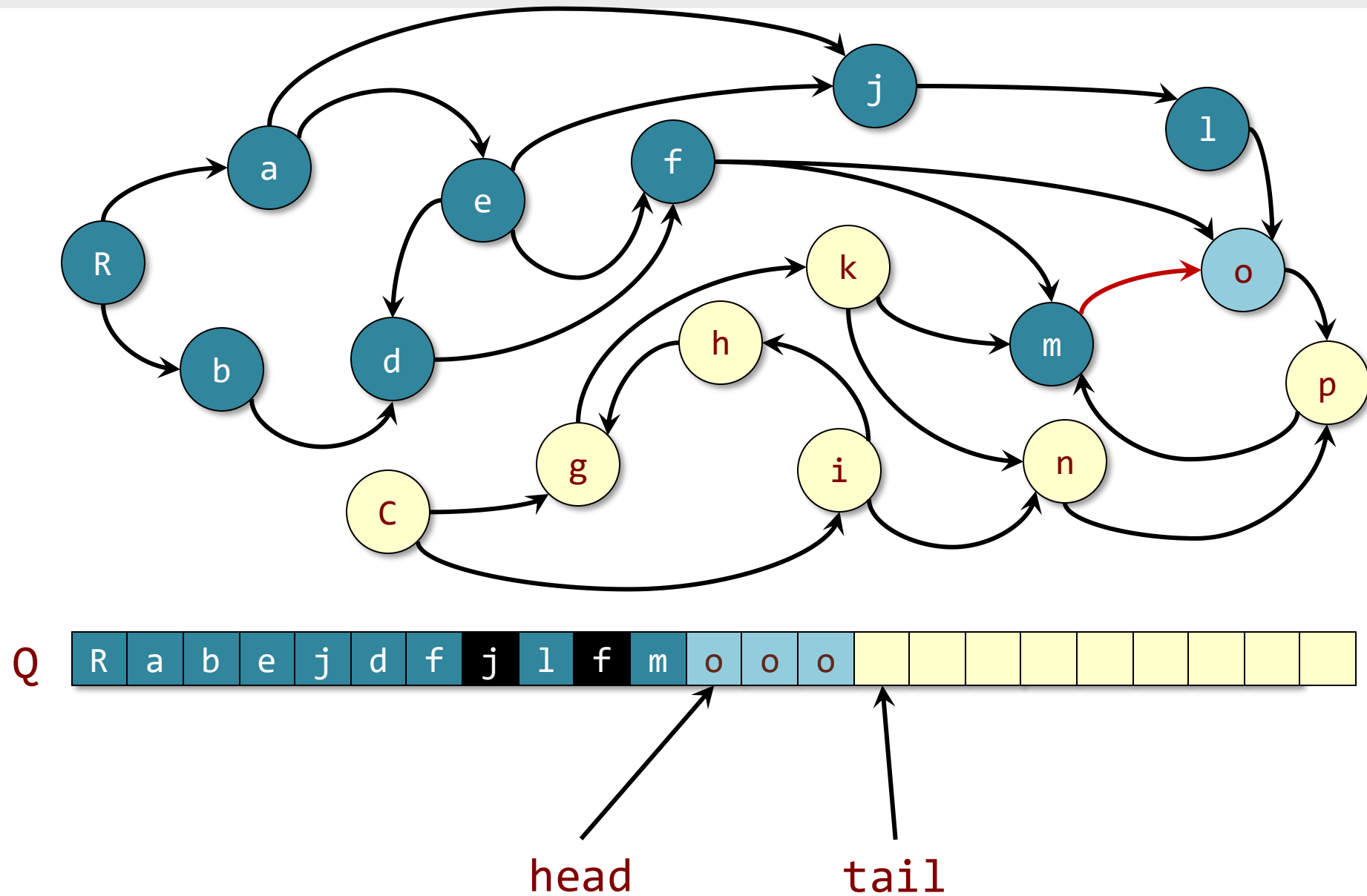
Breadth-First Search



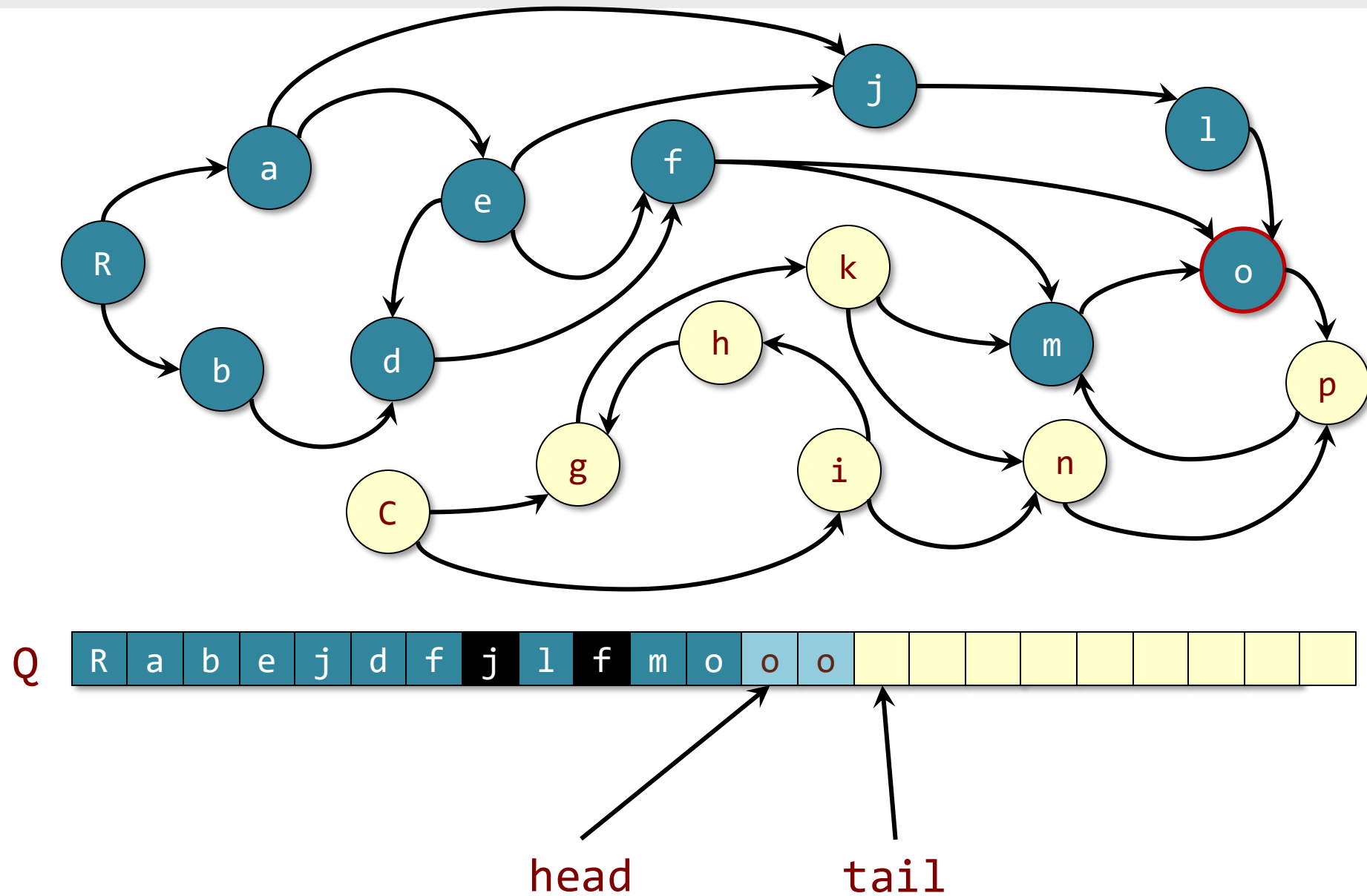
Breadth-First Search



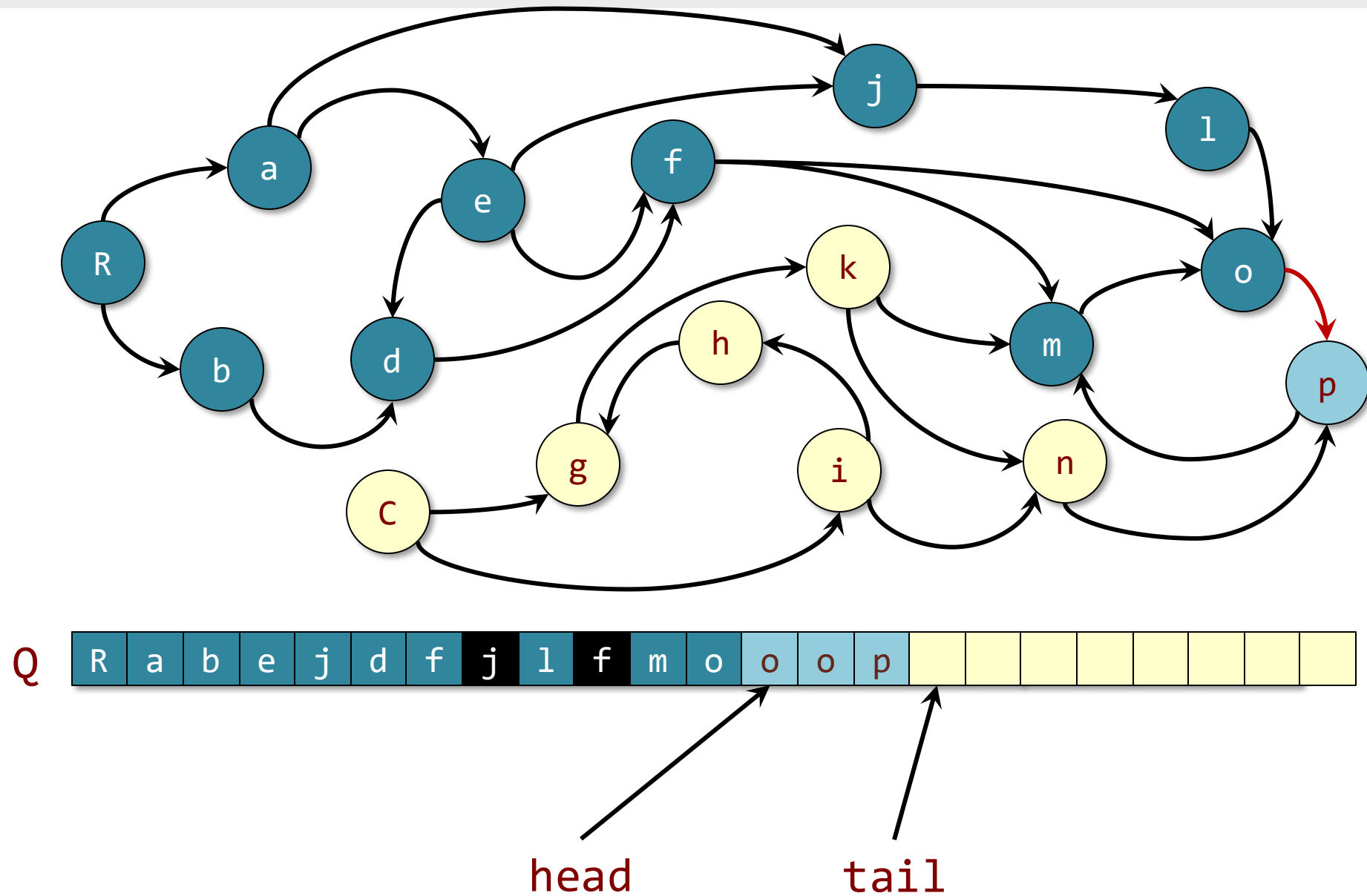
Breadth-First Search



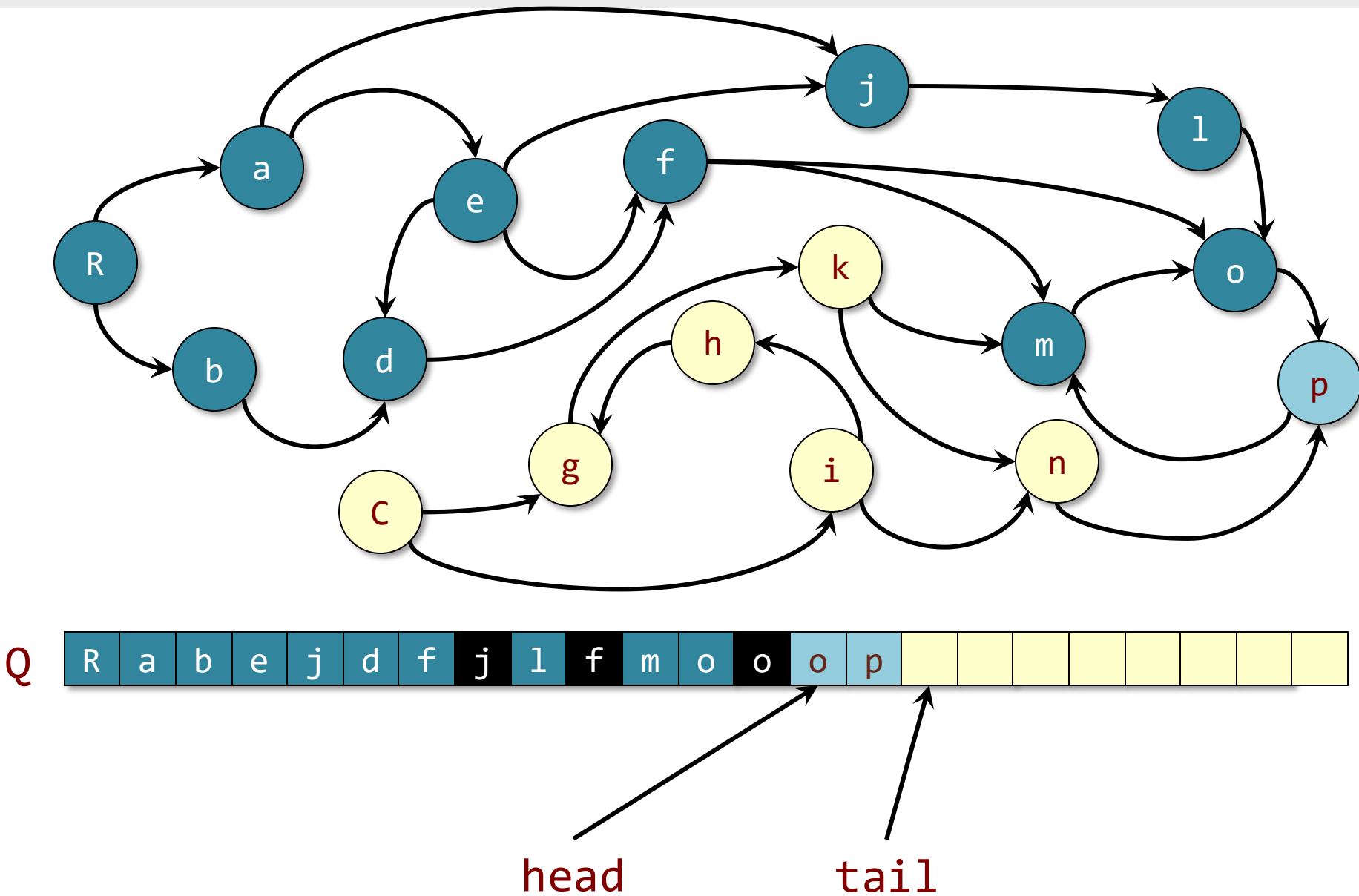
Breadth-First Search



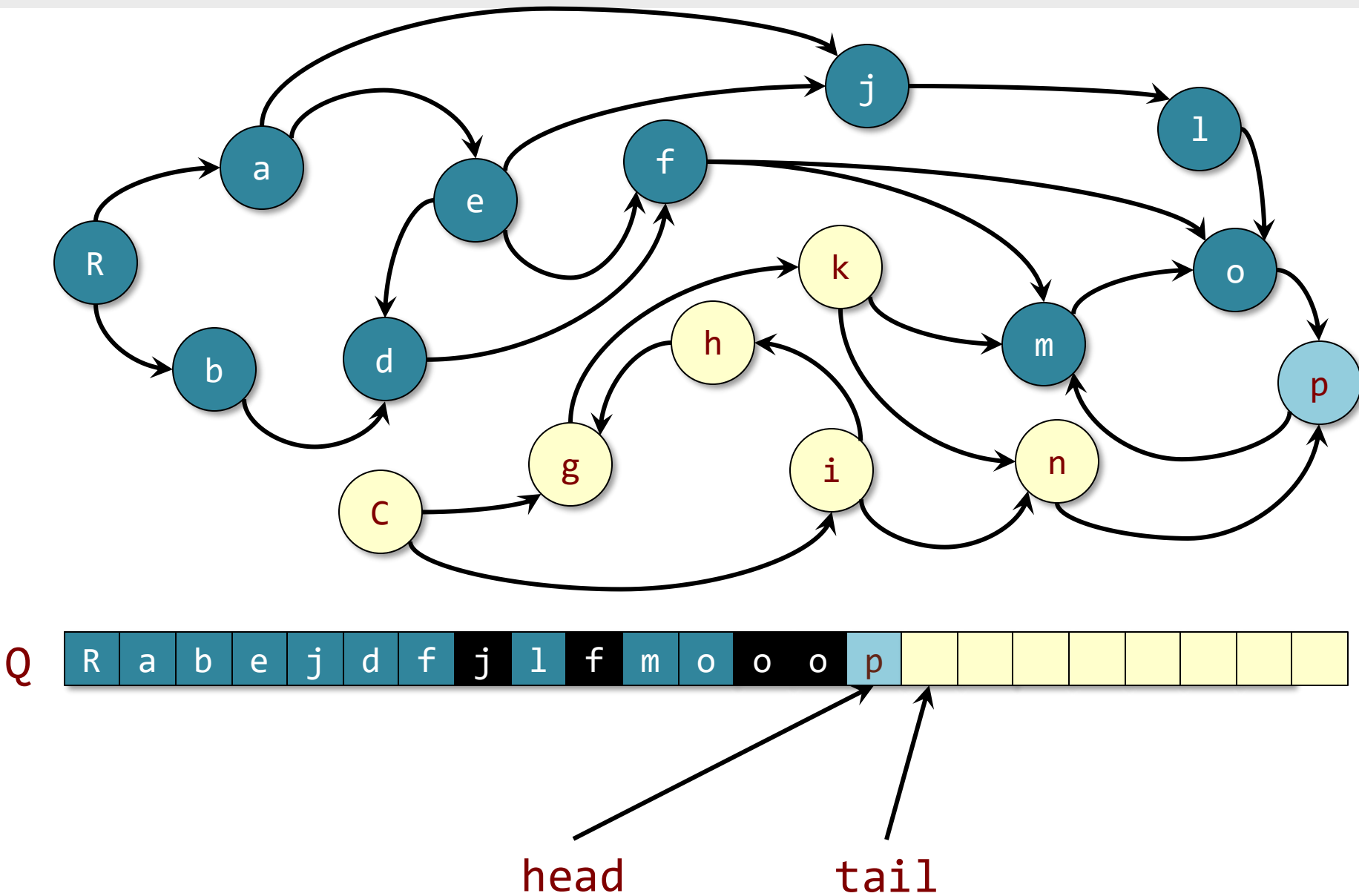
Breadth-First Search



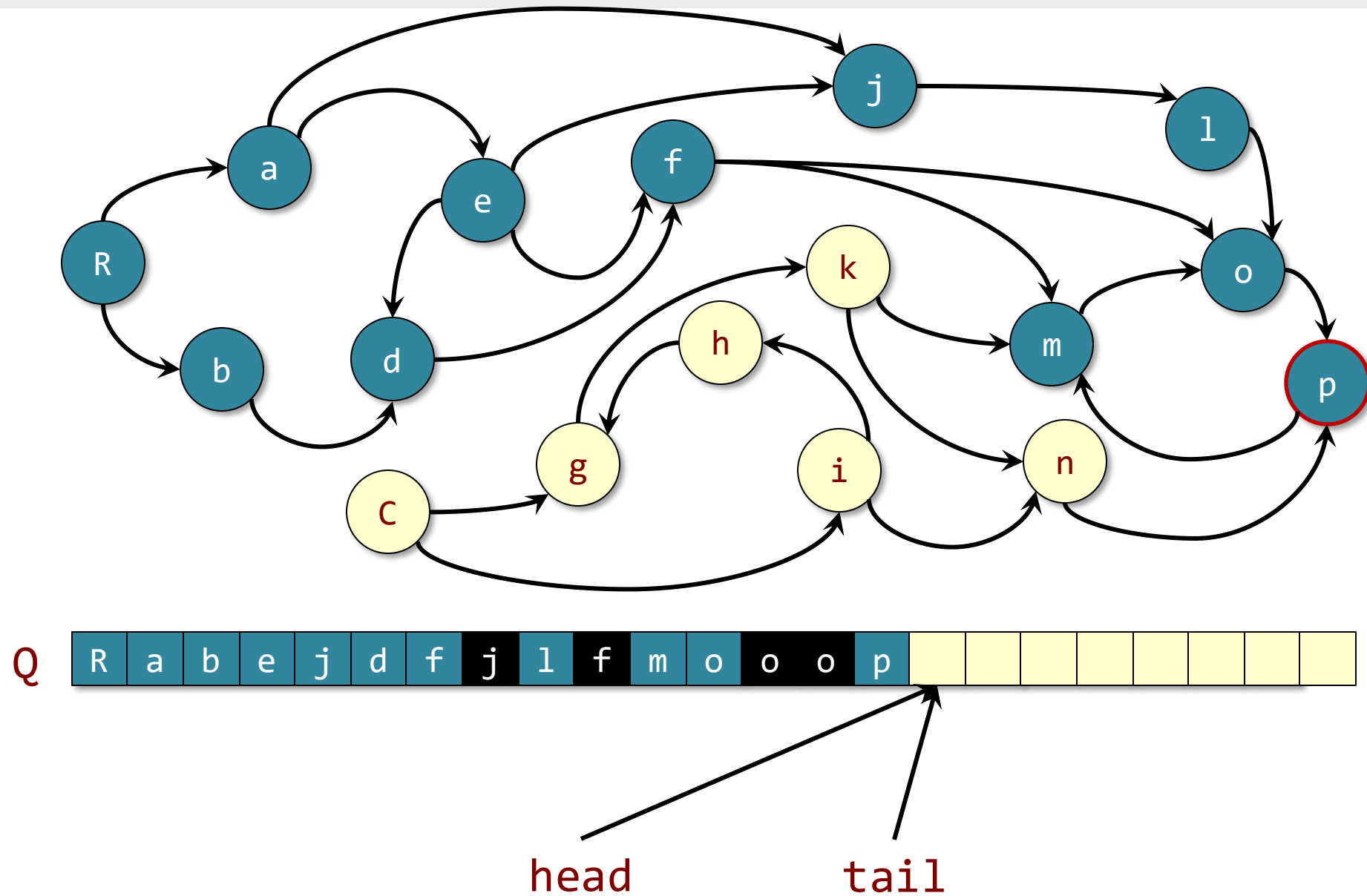
Breadth-First Search



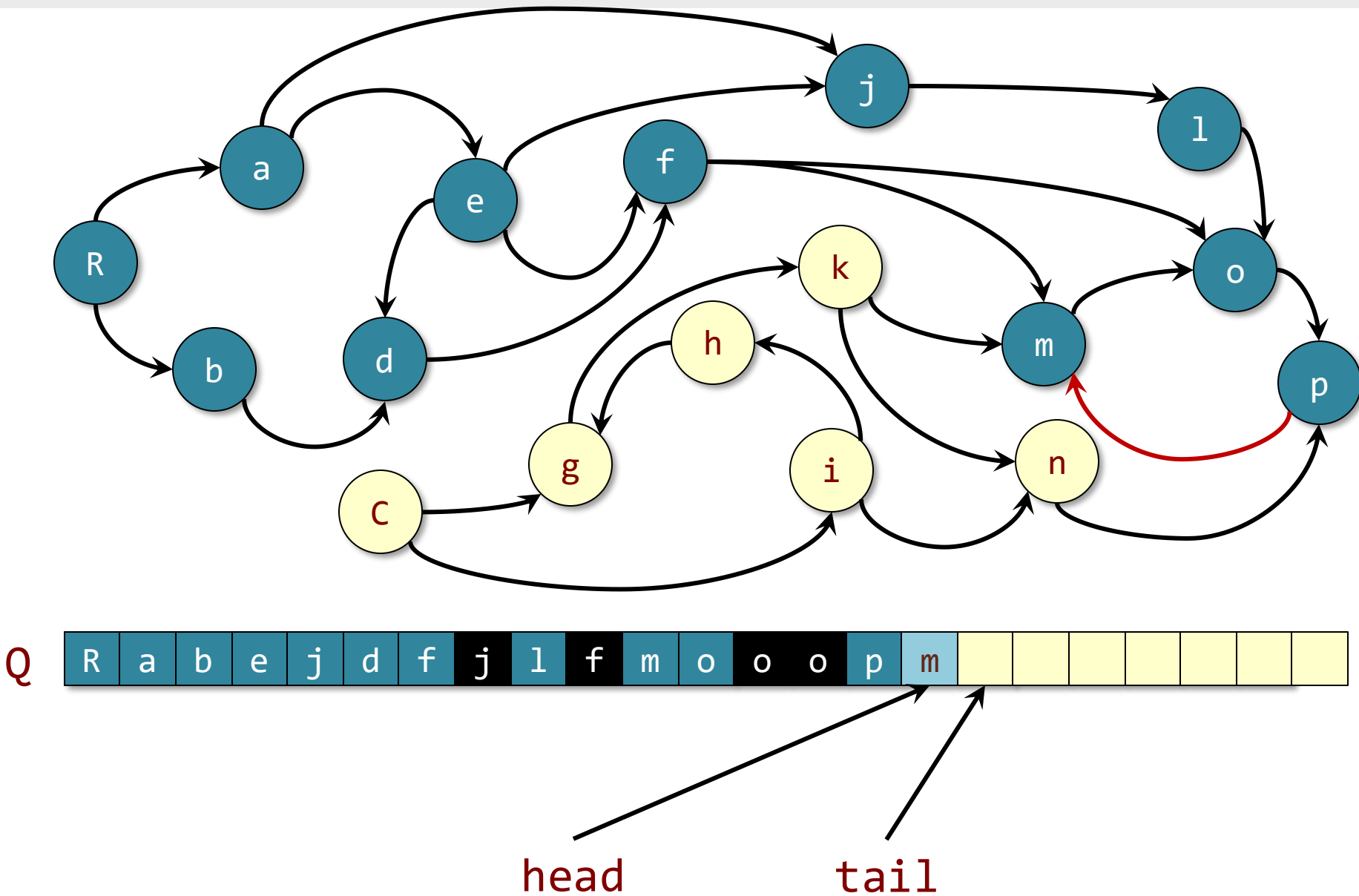
Breadth-First Search



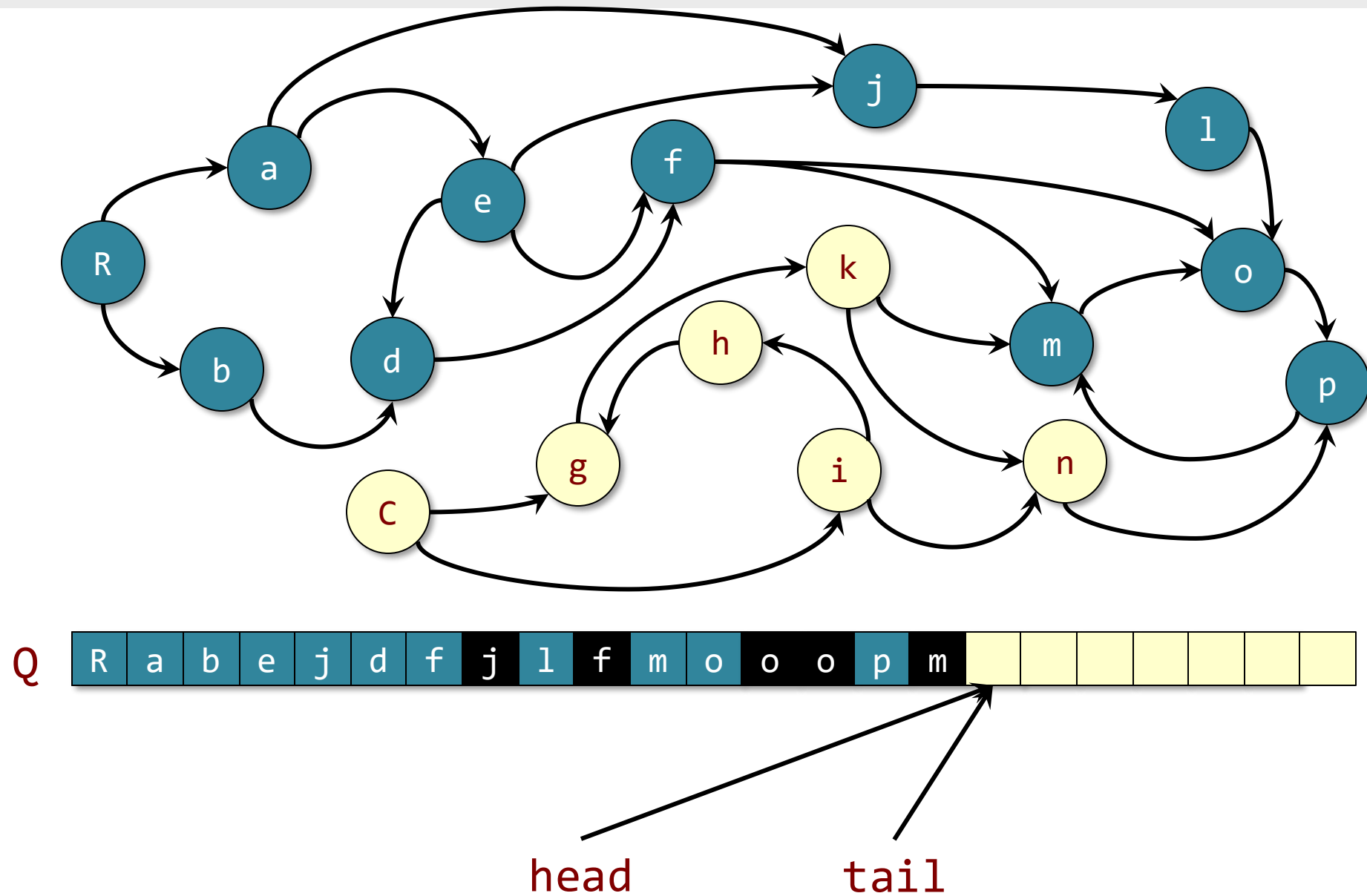
Breadth-First Search



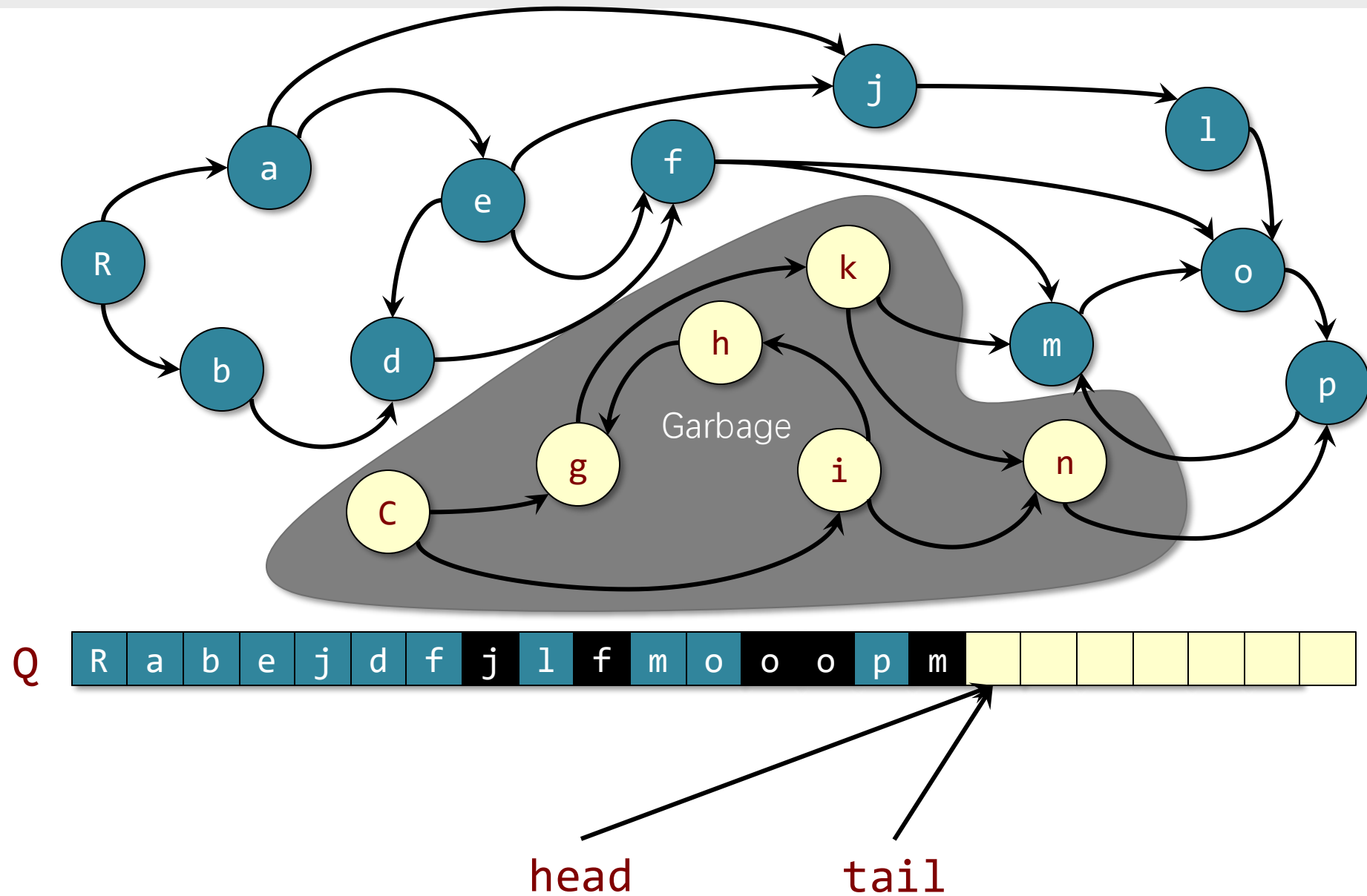
Breadth-First Search



Breadth-First Search



Breadth-First Search



Mark-and-Sweep

- **Mark stage:** Breadth-first search marked all of the live objects.
- **Sweep stage:** Scan over memory to free unmarked objects

Mark-and-sweep doesn't deal with fragmentation

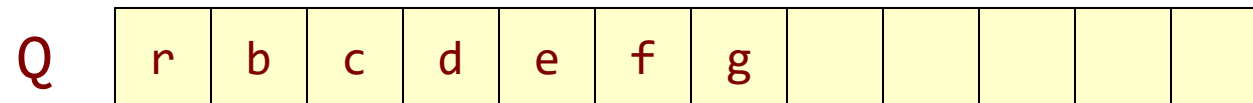
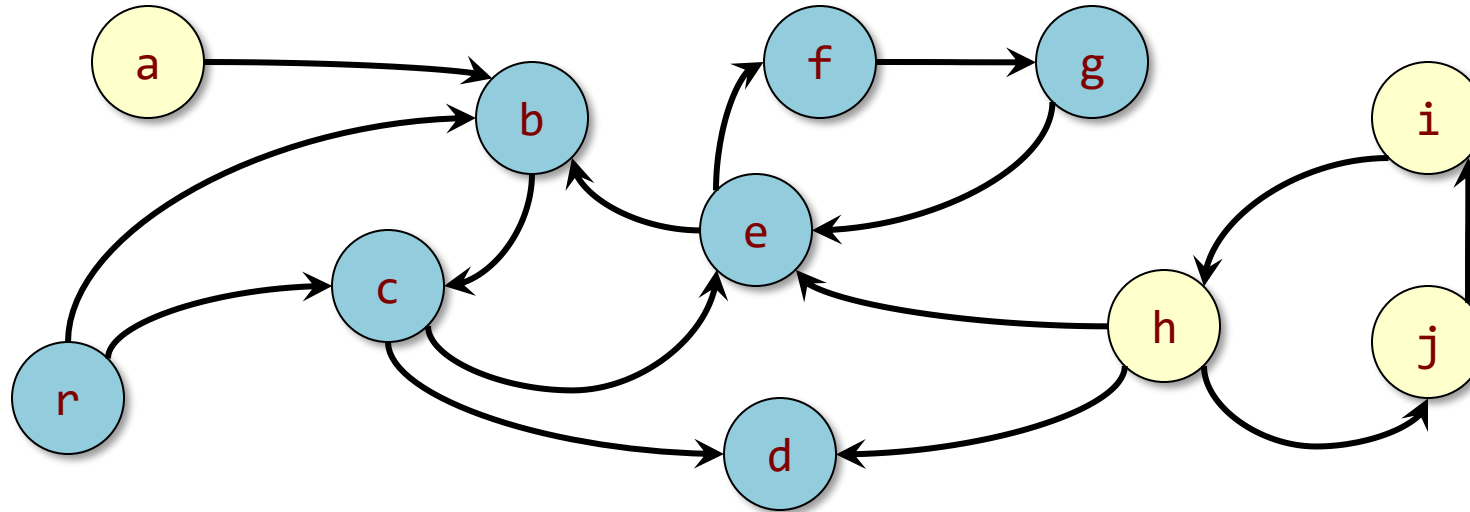
Summary

	Manual	Reference Counting	Mark and Sweep	
Ease of Use	Bad	Medium	Good	
Throughput	Good	Medium	Medium	
Latency	Good	Good	Bad	
External Fragmentation	Bad	Bad	Bad	
Example	C Malloc/Free	C++ std::shared_ptr	Java	

STOP-AND-COPY GARBAGE COLLECTION



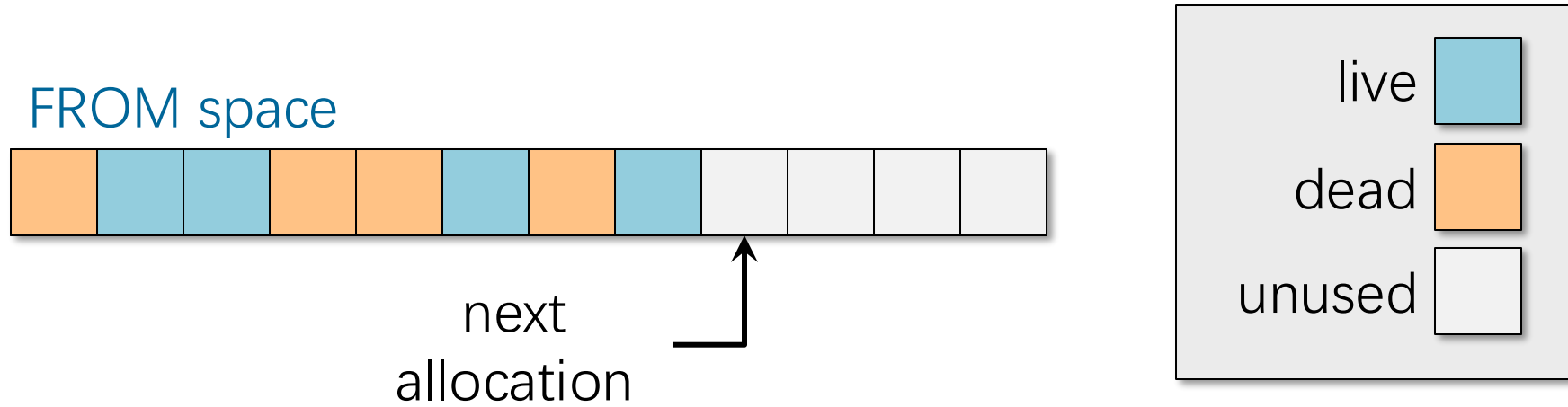
Breadth-First Search



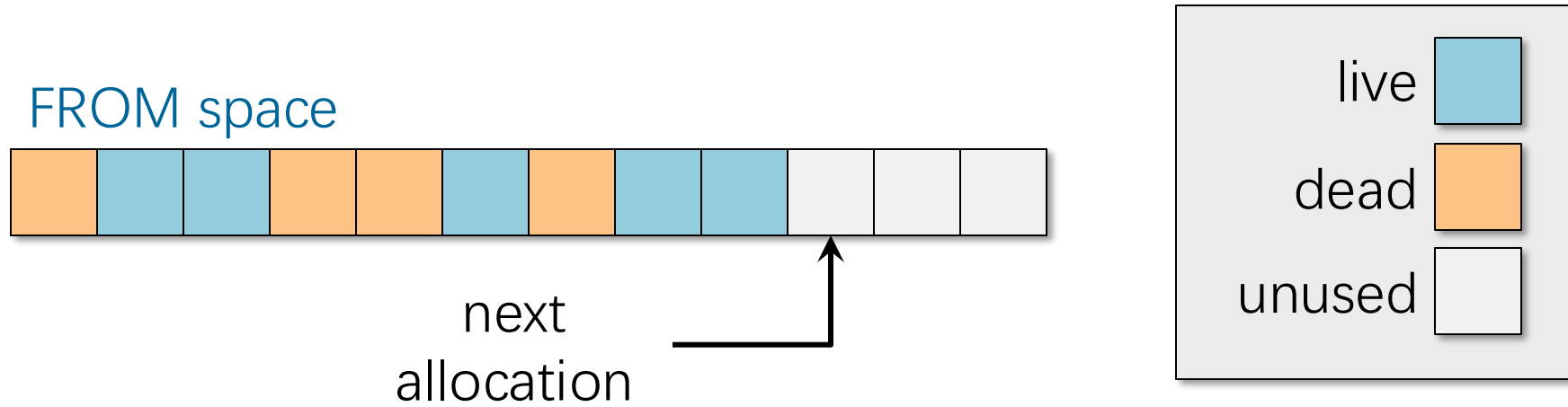
Observation

All live vertices are placed in contiguous storage in Q .

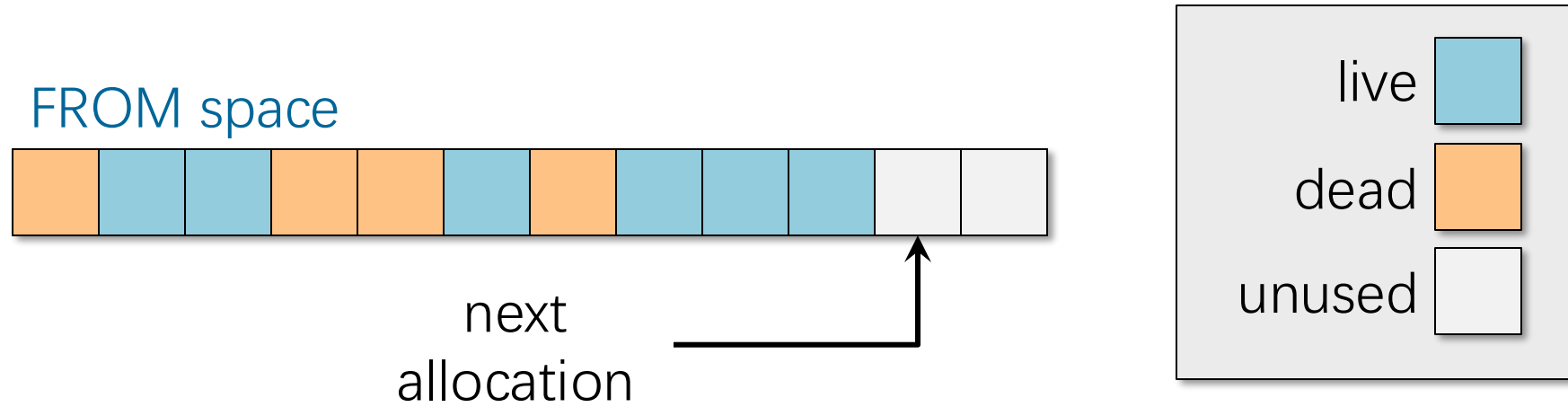
Copying Garbage Collector



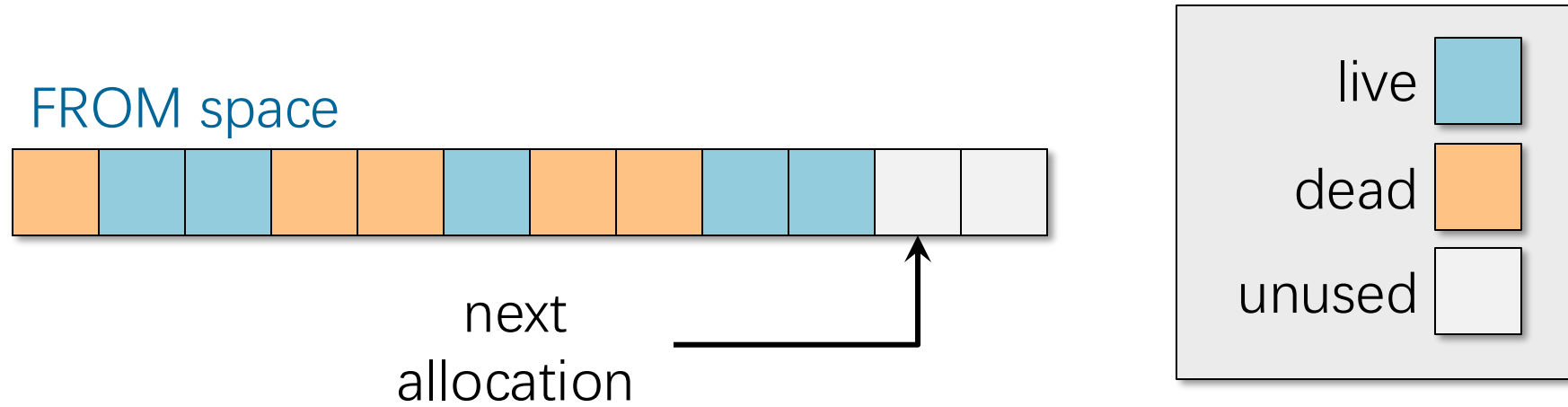
Copying Garbage Collector



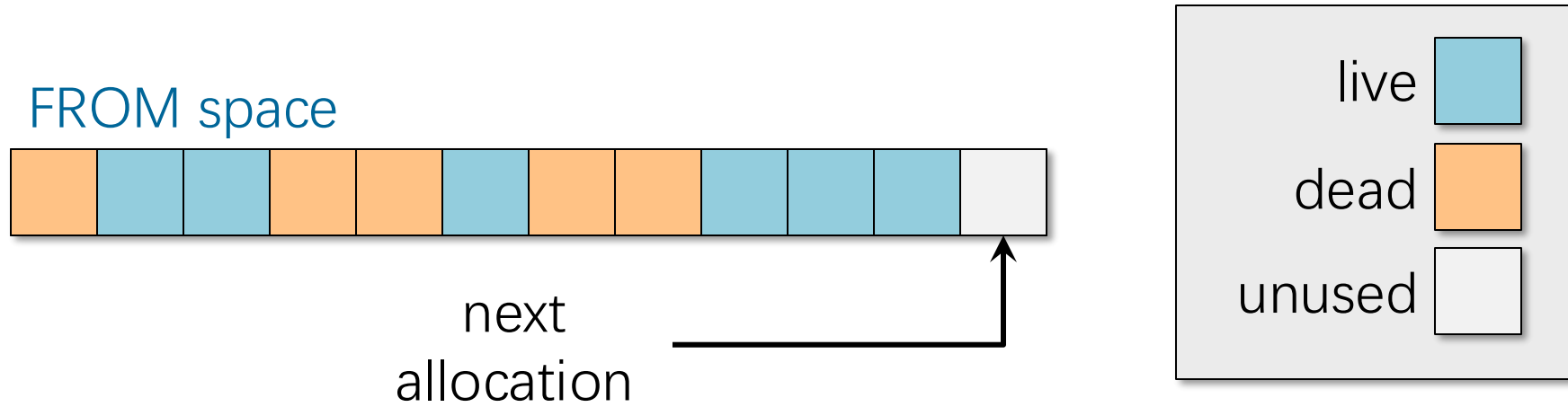
Copying Garbage Collector



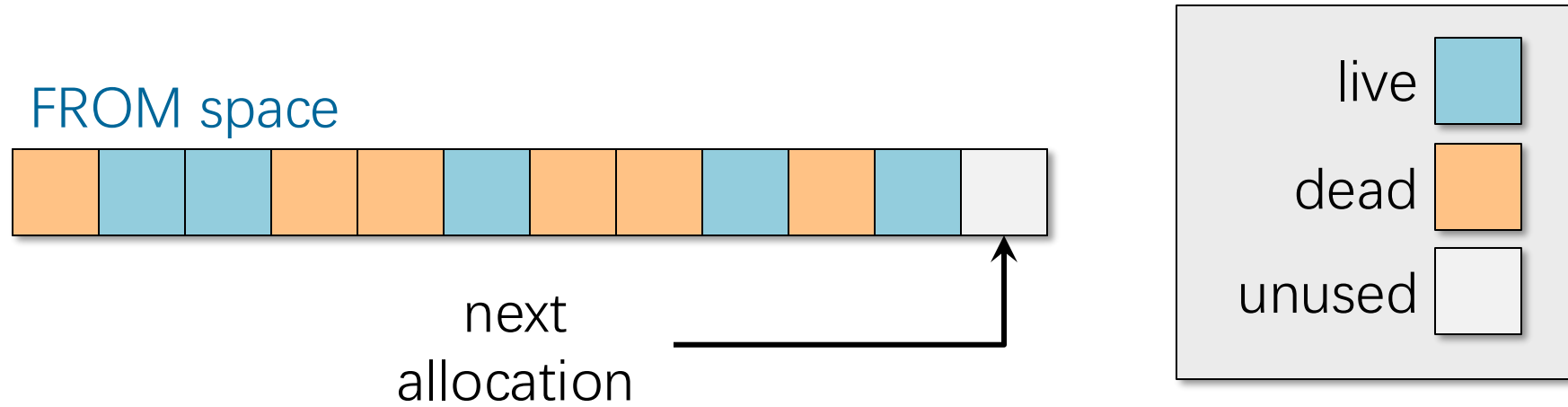
Copying Garbage Collector



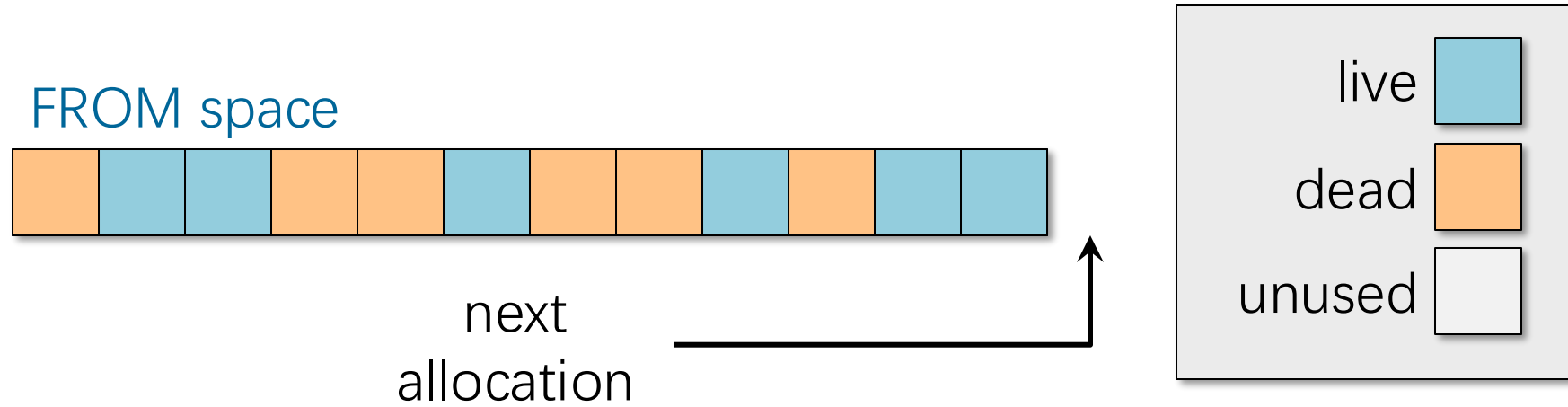
Copying Garbage Collector



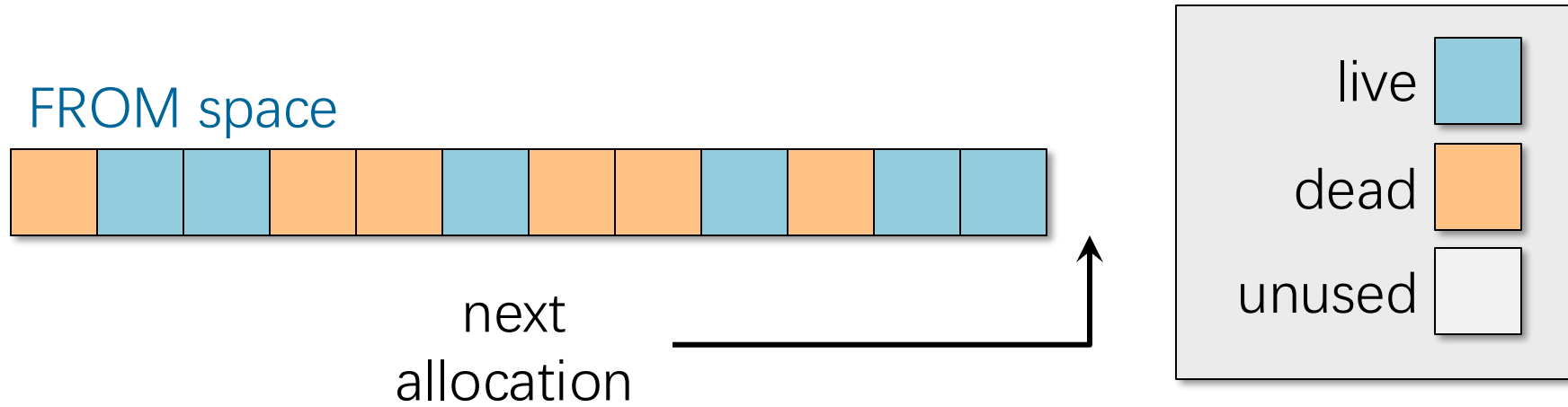
Copying Garbage Collector



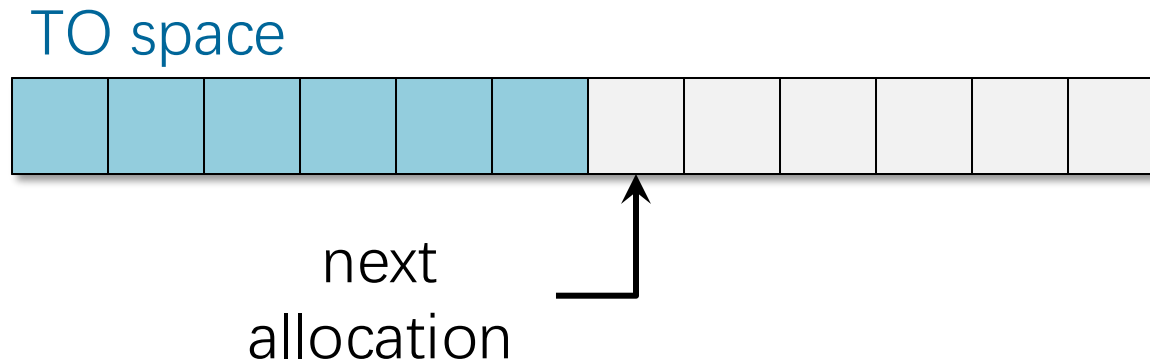
Copying Garbage Collector



Copying Garbage Collector



When FROM space is “full,” copy live storage using BFS with TO space as the queue.



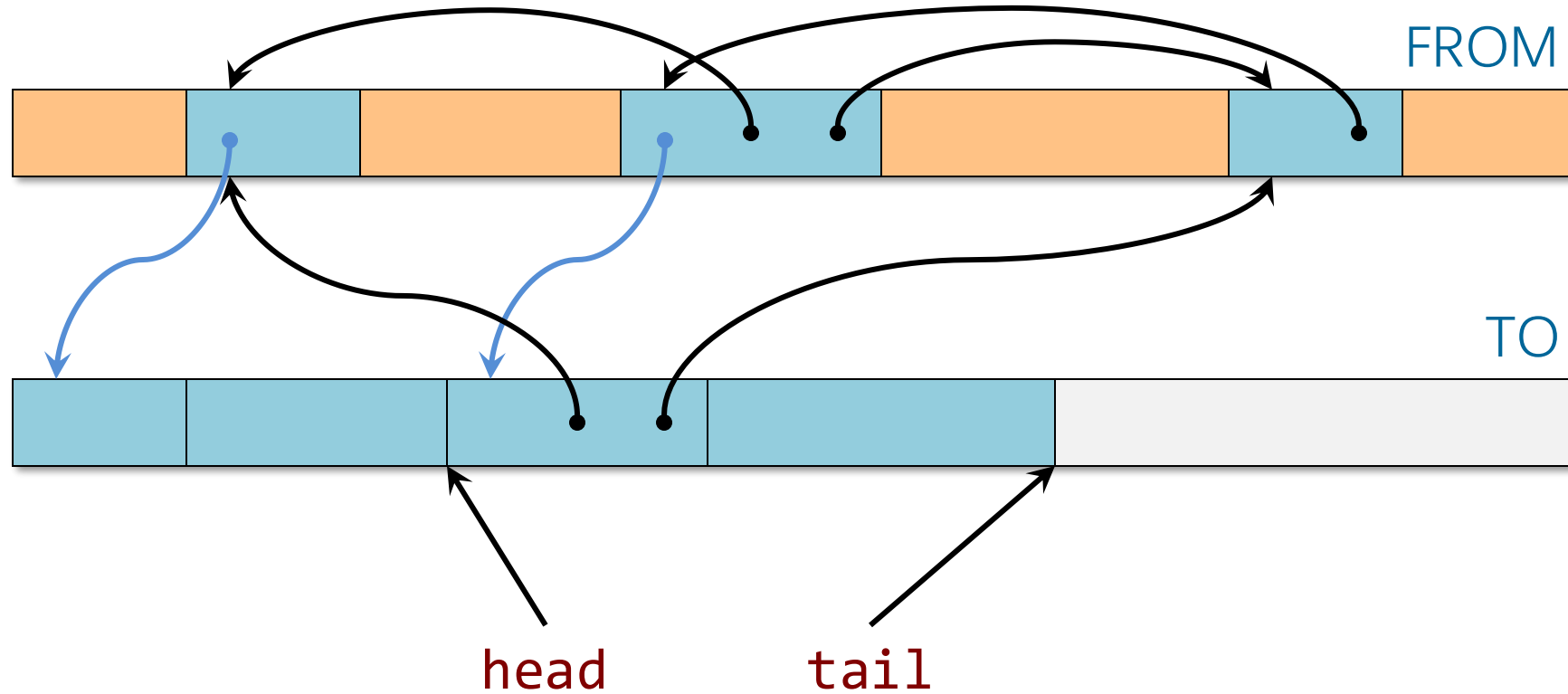
Any Problem with this?

Updating Pointers

- Since the **FROM** address of an object is not generally equal to the **TO** address of the object, **pointers must be updated**.
- How to update?
 - ▣ When an object is copied to the **TO** space, store a forwarding pointer in the **FROM** object, which implicitly marks it as moved.
 - ▣ When an object is removed from the FIFO queue in the **TO** space, update all its pointers.

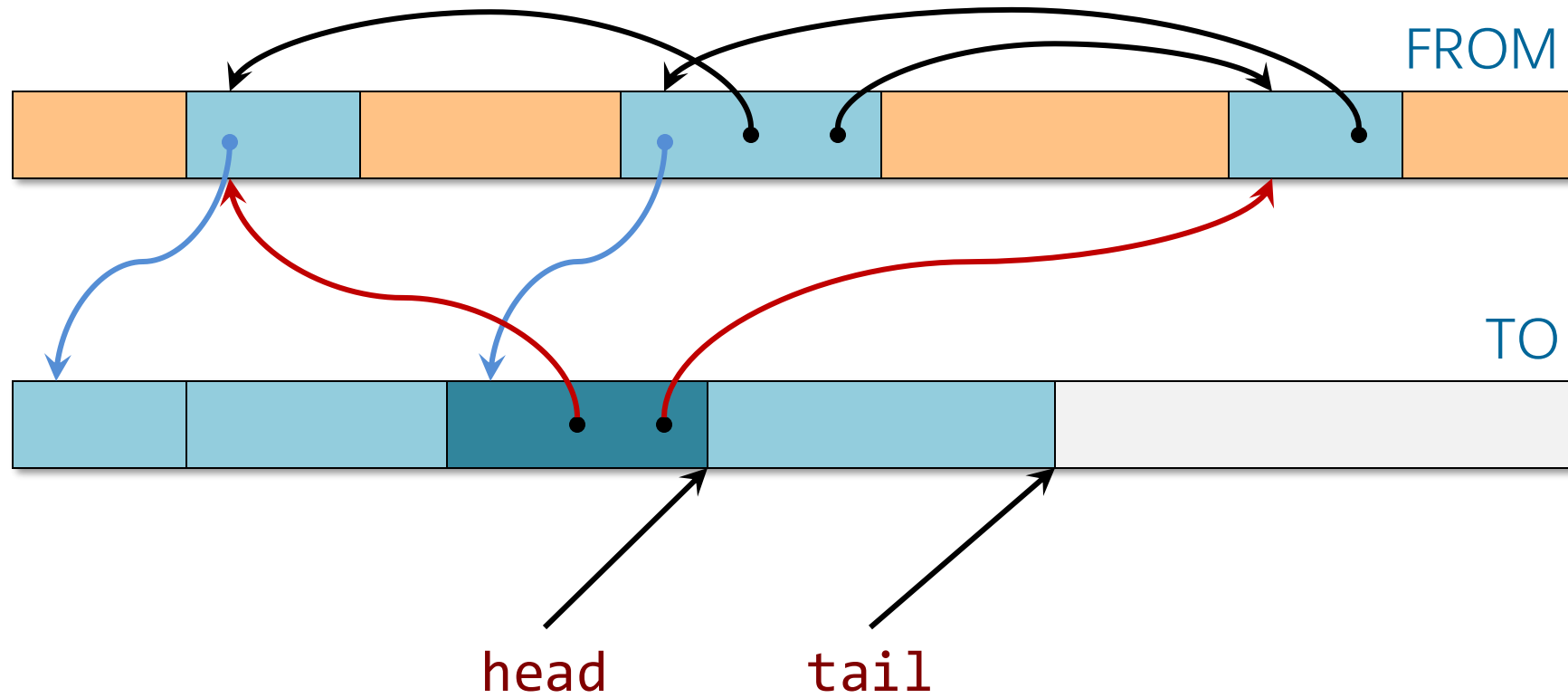
Example

BLUE arrows are FORWARDING pointers



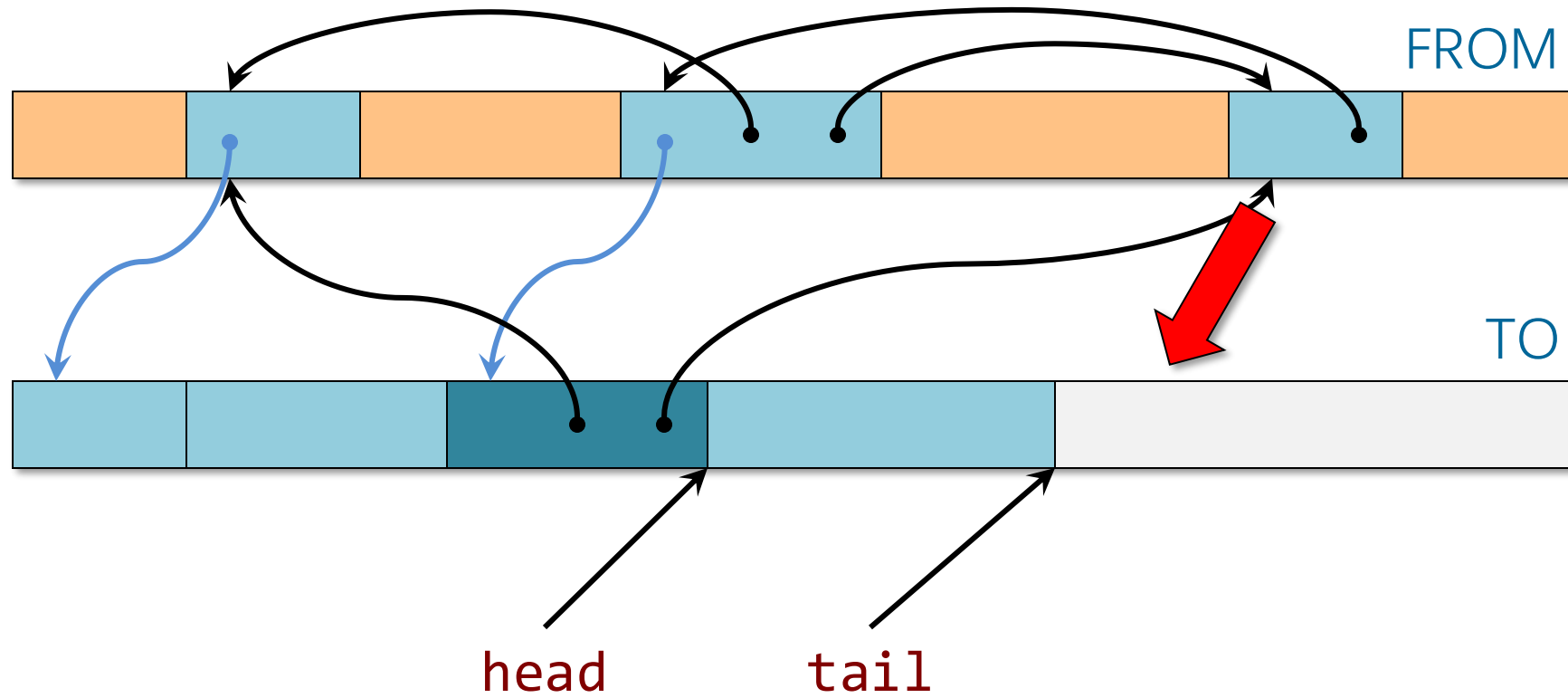
Remove an item from the queue.

Example



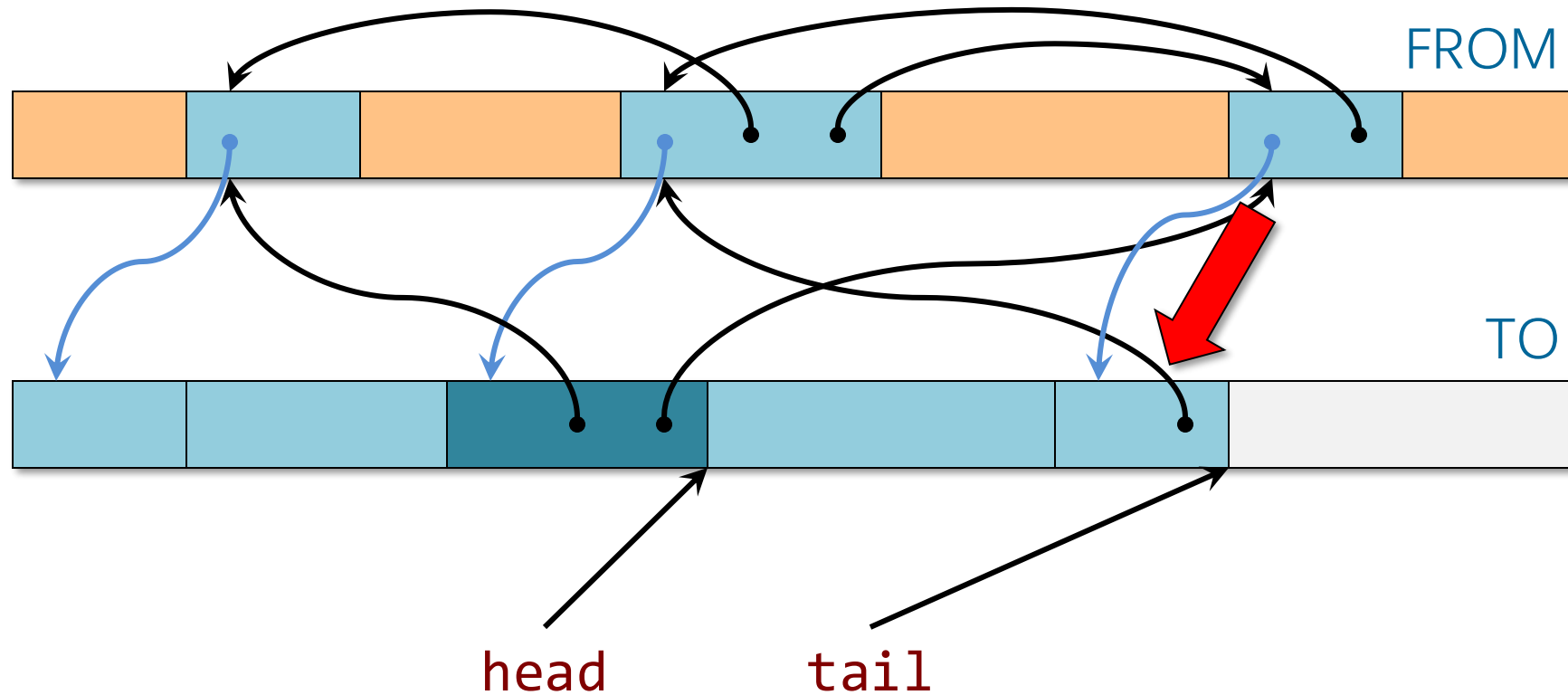
Remove an item from the queue.

Example



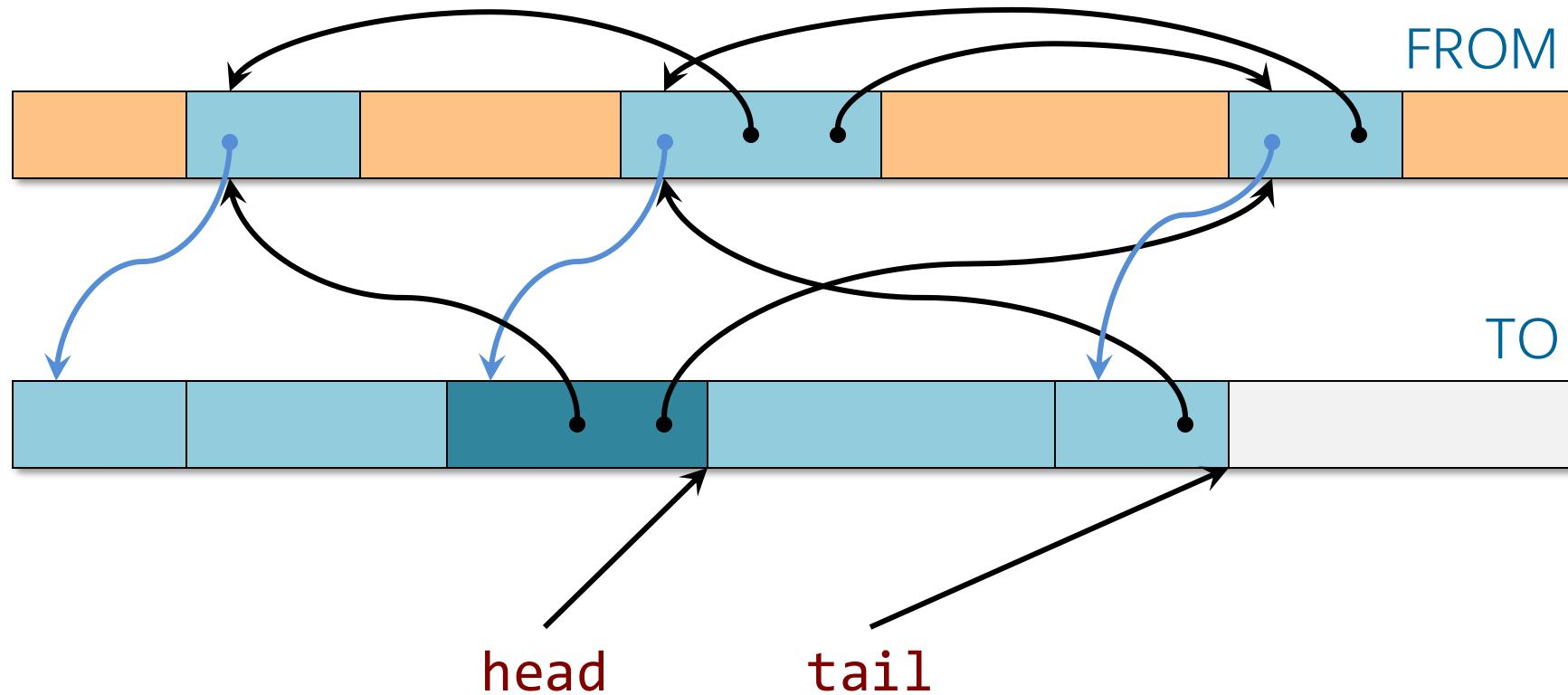
Enqueue adjacent vertices.

Example



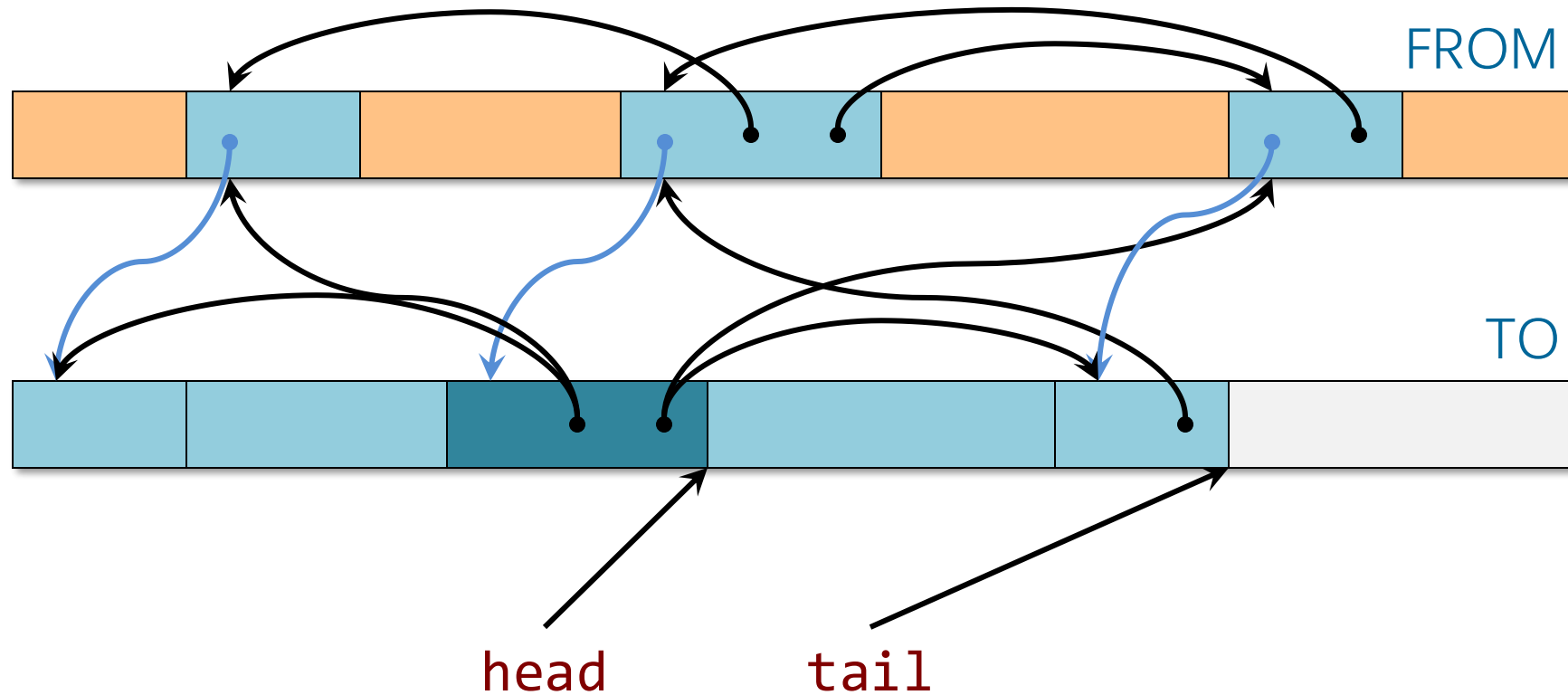
Enqueue adjacent vertices.
Place forwarding pointers in **FROM** vertices.

Example



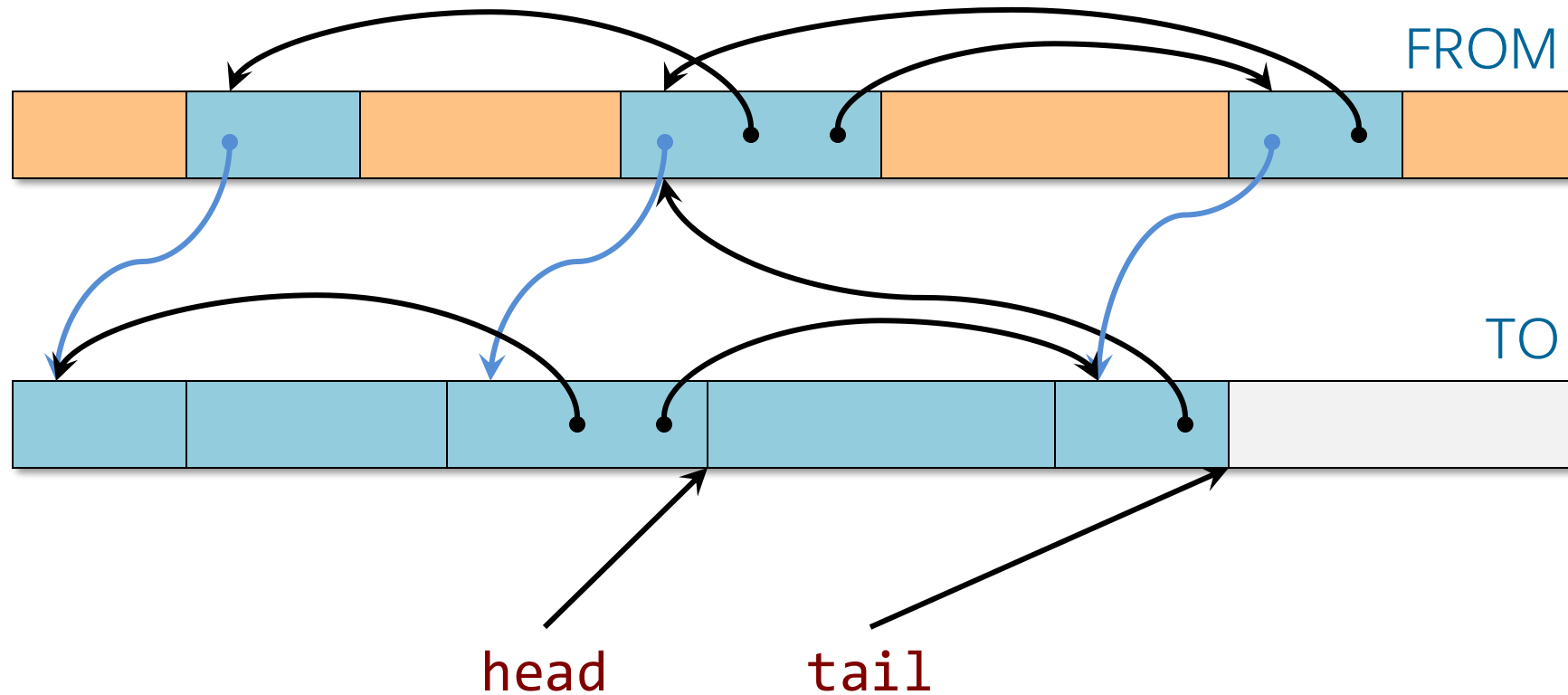
Update the pointers in the removed item to refer to its adjacent items in the **TO** space.

Example



Update the pointers in the removed item to refer to its adjacent items in the **TO** space.

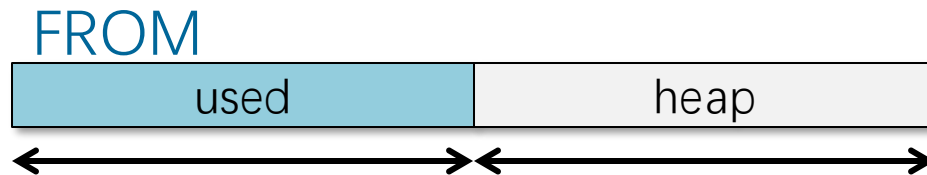
Example



Linear time to copy and update all vertices.
No need to go over unreachable objects.

When Is the FROM Space “Full”?

- Request new heap space **equal to** used space, i.e. **double** the space



- The cost of GC is proportional to the size of the new heap space
⇒ amortized $O(1)$ overhead, assuming user program touches all memory allocated
- VM space required is $O(1)$ times optimal
 - ▣ Factor of 2 for two separate (FROM and TO) spaces (do not interfere)
 - ▣ Factor of 2 comes for doubling the size of the array

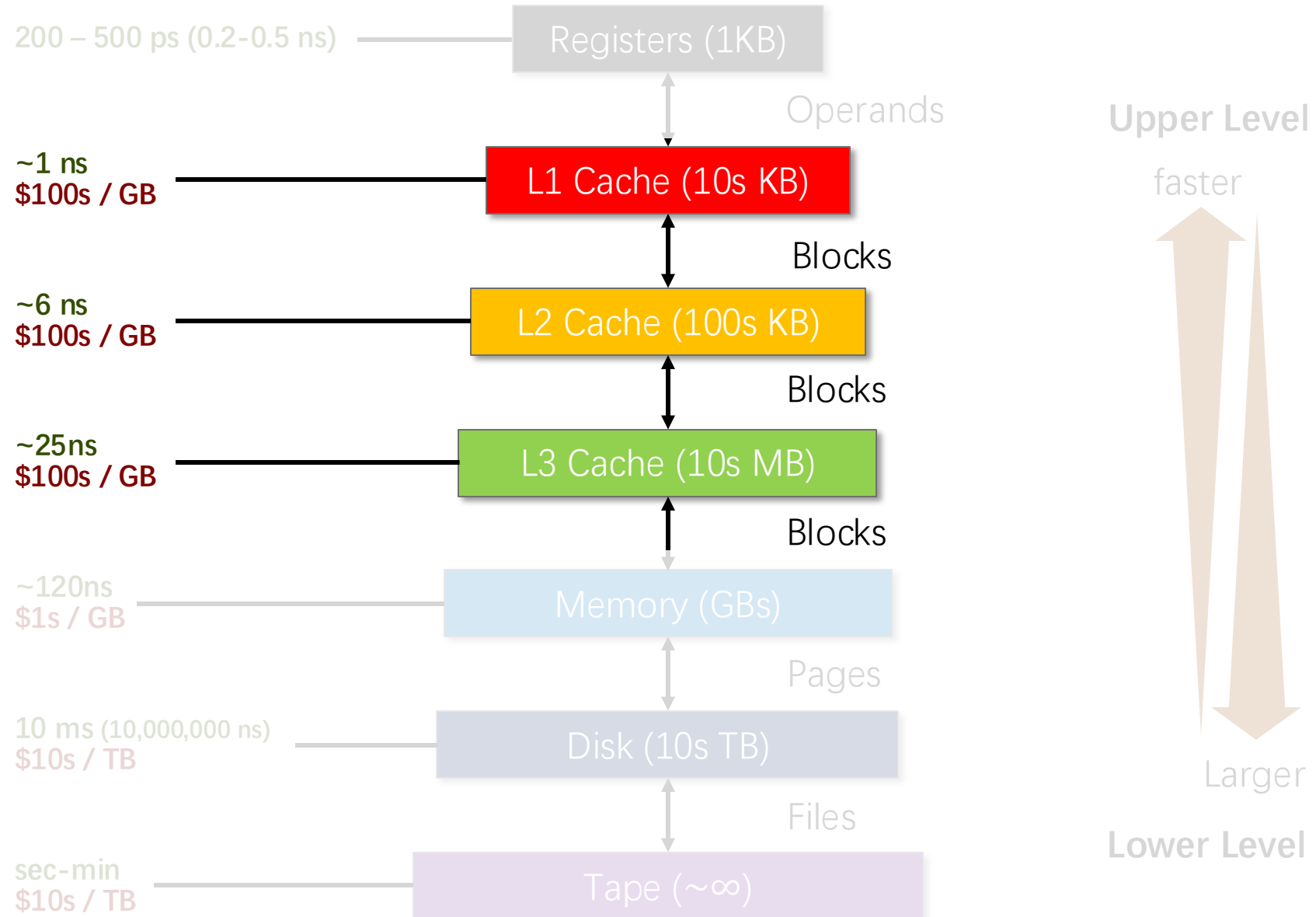
Summary

	Manual	Reference Counting	Mark and Sweep	Stop and Copy
Ease of Use	Bad	Medium	Good	Good
Throughput	Good	Medium	Medium	Bad
Latency	Good	Good	Bad	Bad
External Fragmentation	Bad	Bad	Bad	Good
Example	C Malloc/Free	C++ std::shared_ptr	Java	C#

Outline

- Virtual Memory System
- Memory Allocation Strategies
 - ▣ Stacks
 - ▣ Heaps
- Garbage Collection
 - ▣ Reference Counting
 - ▣ Mark and Sweep
 - ▣ Stop-and-Copy
- Memory Hierarchy and Organization
 - ▣ Cache Hierarchy
 - ▣ Memory Level Parallelism

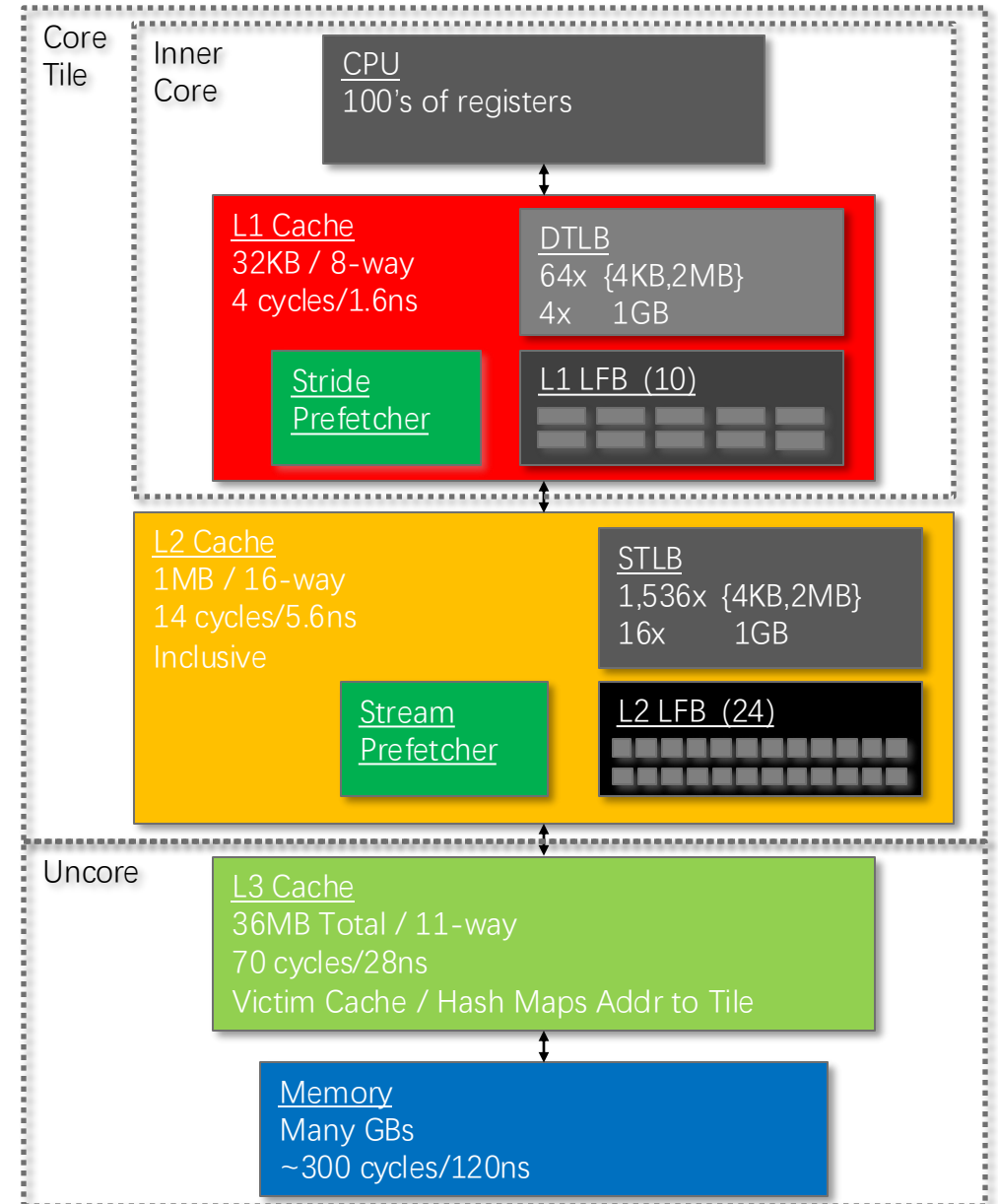
Levels of the Memory Hierarchy



Cache Hierarchy

Intel's Cascade Lake is the test system for MicroBenchmarks in this lecture.

- DTLB (Data Translation Lookaside Buffer)
- LFB (Line Fill Buffer)
 - ▣ used to temporarily hold data being fetched from main memory or a higher level cache
- STLB (Supervisor TLB)
 - ▣ similar to the DTLB, but specifically used for **supervisor-level** (or kernel-level) memory accesses, as opposed to the DTLB, which handles user-level memory accesses.



Cascade Lake – 24 Core

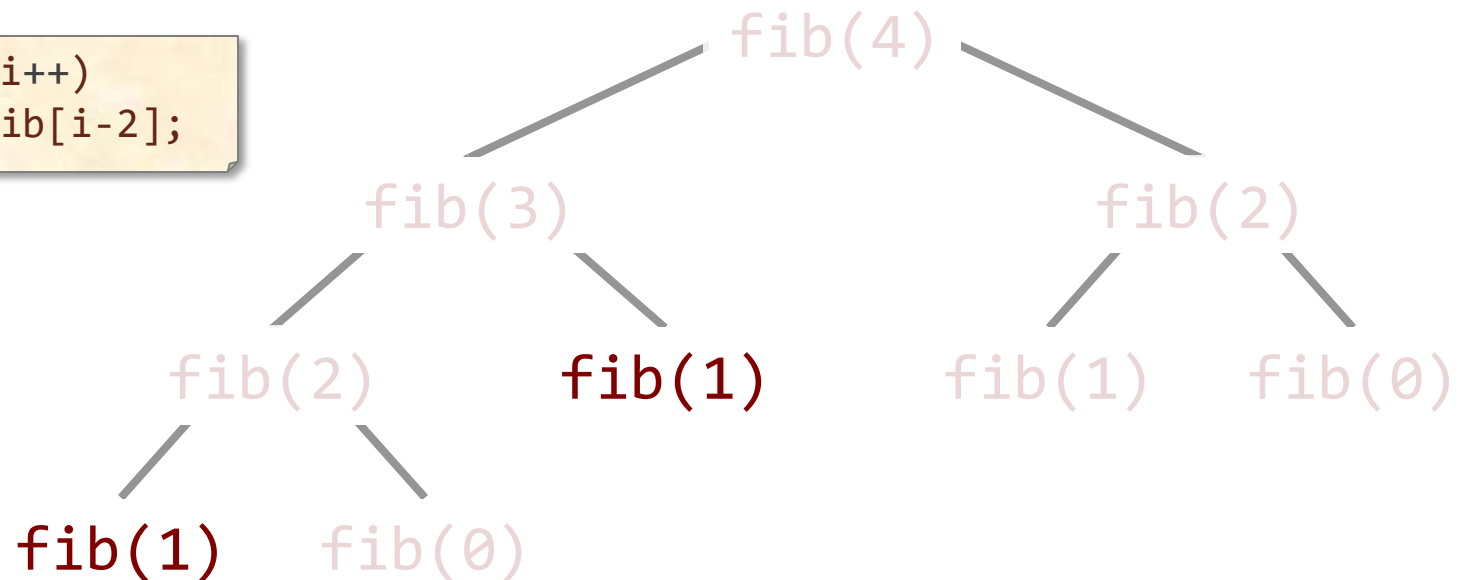


Principle of Locality

A **program** often accesses a relatively small portion of the address space at any given time.

- **Temporal Locality** (Locality in Time): If an item is referenced, it will tend to be referenced again soon (e.g., loops, reuse)

```
for (int i = 2; i < N; i++)  
    fib[i] = fib[i-1] + fib[i-2];
```



Principle of Locality

A **program** often accesses a relatively small portion of the address space at any given time.

- **Temporal Locality** (Locality in Time): If an item is referenced, it will tend to be referenced again soon (e.g., loops, reuse)
- **Spatial Locality** (Locality in Space): If an item is referenced, items whose addresses are close by tend to be referenced soon (e.g., straight-line code, array access)

```
int sum = 0;
for (int i = 0; i < N; i++)
    sum += A[i];
```

Caches, the single greatest computer architectural innovation of the last 60 years, rely on **locality** to be effective.

Cache Issues

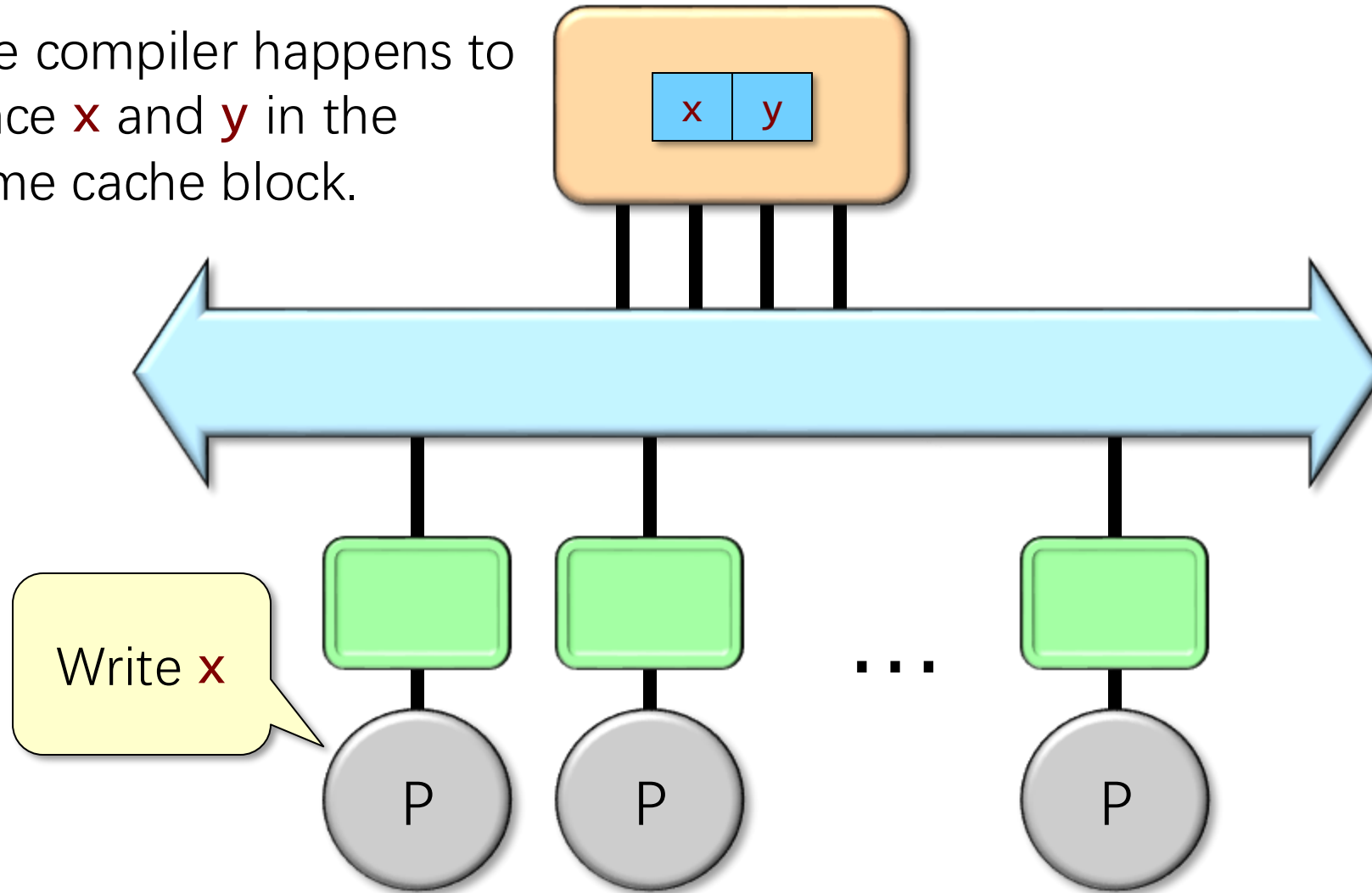
- Compulsory (Cold) Miss
 - The **first** time the data is accessed
 - **Prefetching** may be able to reduce the cost
- Capacity Miss
 - The previous access has been **evicted** because too much data touched in between
 - “Working Set” too large
 - Shorten **reuse distance**: reorganize data access so reuse occurs before getting evicted
 - Prefetch otherwise
- Conflict Miss
 - Multiple data items mapped to the same location. Evicted even before cache is full
 - Rearrange data layout and/or pad arrays
 - Associativity helps

Cache Sharing Issues

- True Sharing Miss
 - Thread in another processor wanted the data, it got moved to the other cache
- False Sharing Miss
 - Other processor used different data in the same cache line. So the line got moved

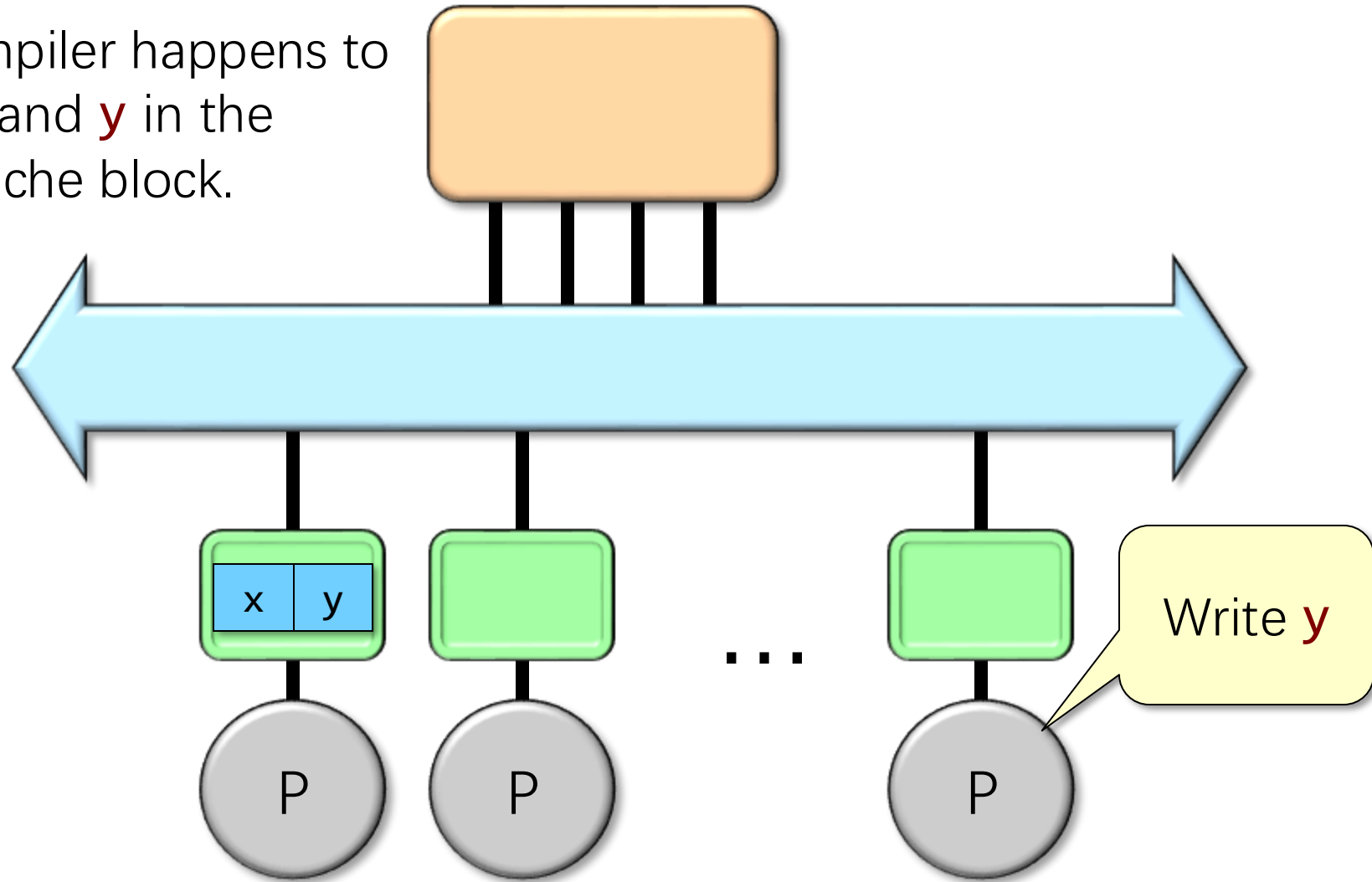
False Sharing Example

The compiler happens to place **x** and **y** in the same cache block.



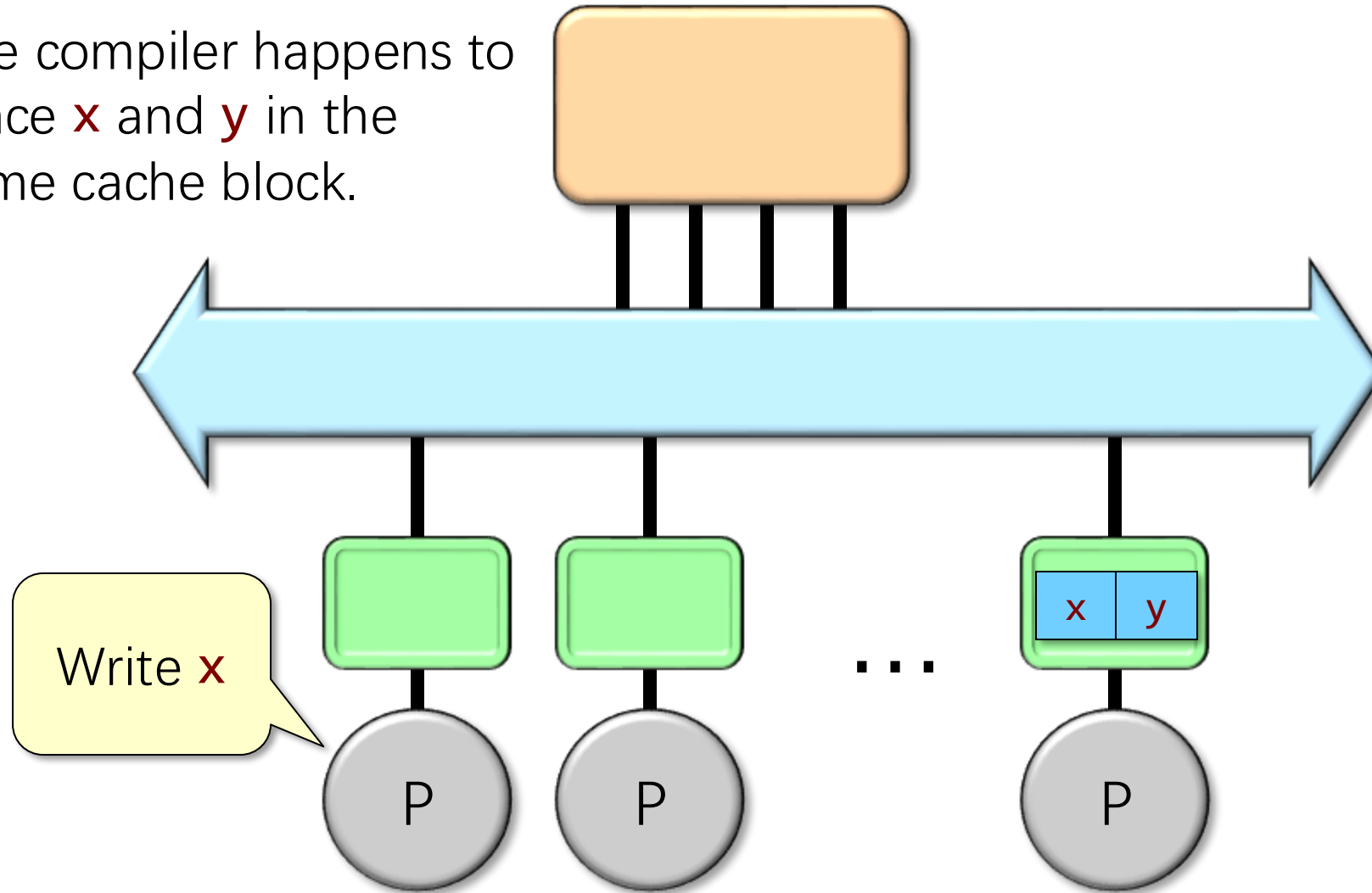
False Sharing Example

The compiler happens to place **x** and **y** in the same cache block.



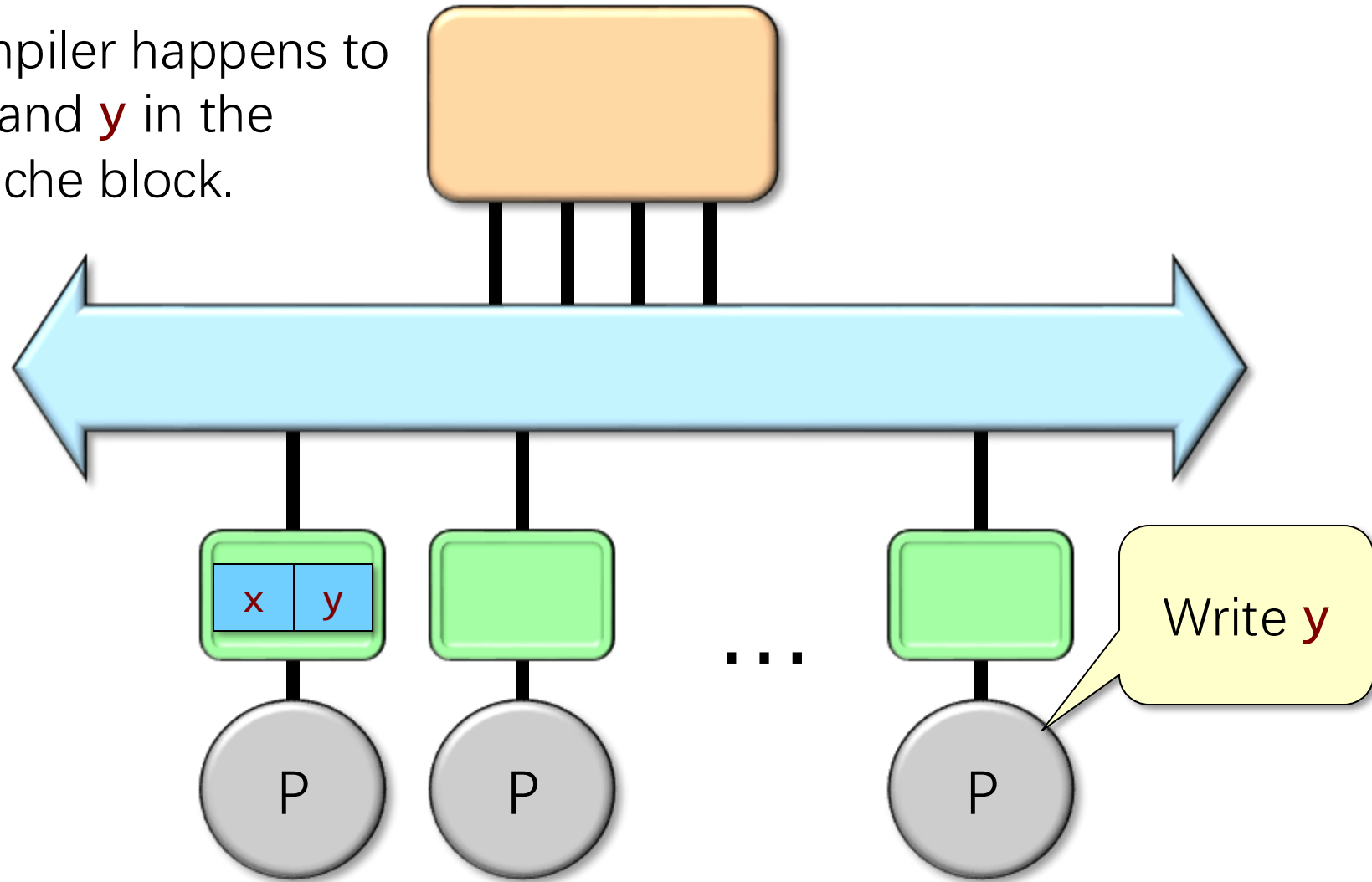
False Sharing Example

The compiler happens to place **x** and **y** in the same cache block.



False Sharing Example

The compiler happens to place **x** and **y** in the same cache block.



Cache Sharing Issues

- True Sharing Miss
 - ▣ Thread in another processor wanted the data, it got moved to the other cache
 - ▣ Minimize sharing/locks
- False Sharing Miss
 - ▣ Other processor used different data in the same cache line. So the line got moved
 - ▣ Pad data and make sure structures such as locks don't get into the same cache line

Outline

- Virtual Memory System
- Memory Allocation Strategies
 - Stacks
 - Heaps
- Garbage Collection
 - Reference Counting
 - Mark and Sweep
 - Stop-and-Copy
- Memory Hierarchy and Organization
 - Cache Hierarchy
 - **Memory Level Parallelism**

MICROBENCHMARK



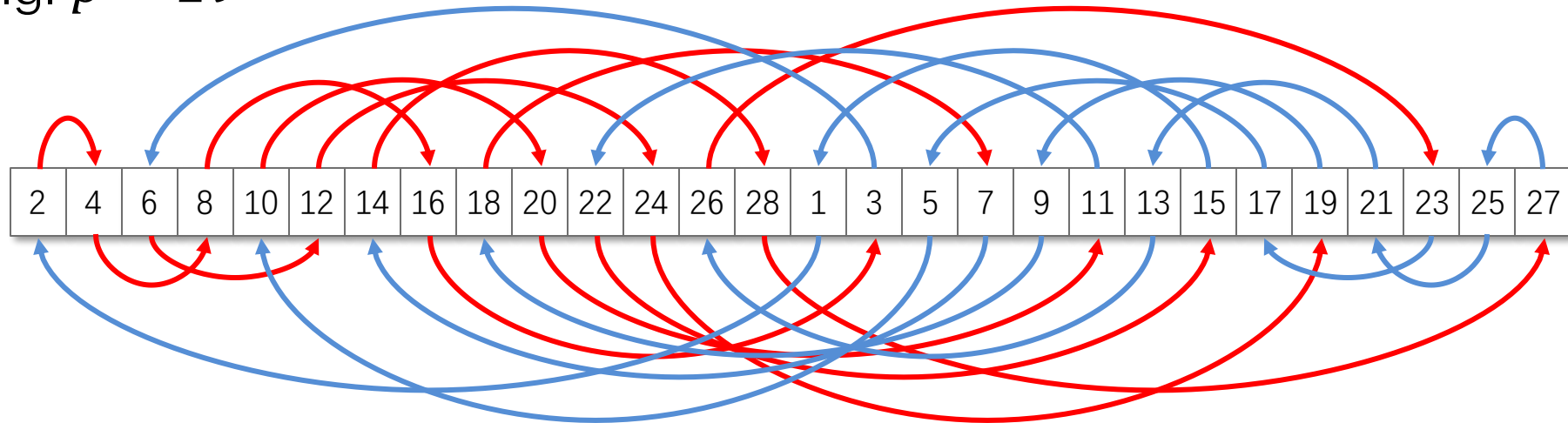
Cycle Microbenchmark

Let p be a prime such that 2 is a primitive root of the finite field $\text{GF}(p)$.

We initialize an array A with the cycle:

$$A[n] = 2n \bmod p \quad \forall n \in \{1 \dots p-1\}$$

e.g. $p = 29$



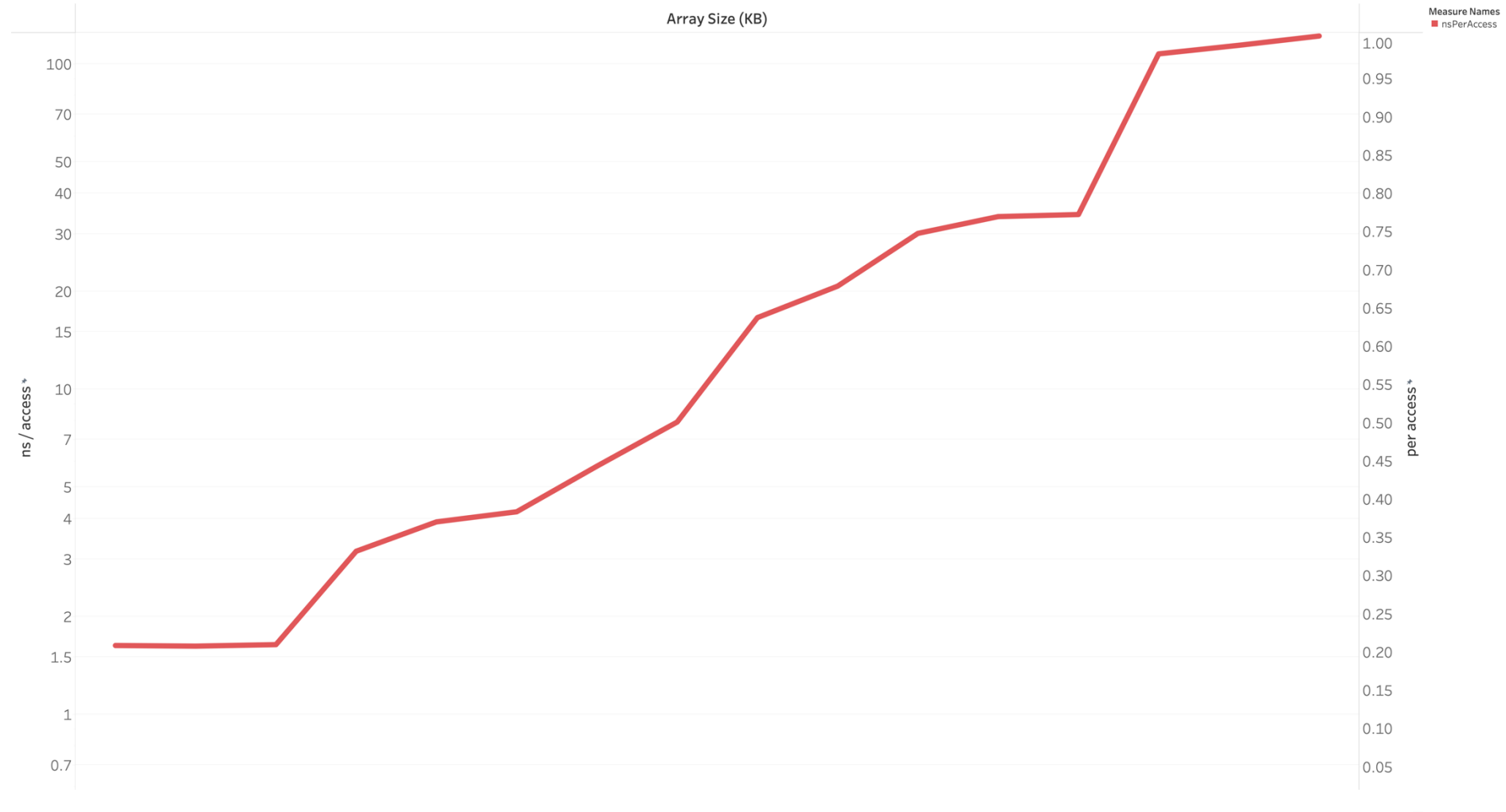
Cycle, ReadNext

```
int cycleWithReadNext(int *A, int p, int iterations) {  
    int sum = 0;  
    int n = 1;  
    for (int i = 0; i < iterations; i++) {  
        sum += A[n];  
        n = A[n];  
    }  
    return sum;  
}
```

```
cycleWithReadNext(int*, int, int):  
    test edx, edx  
    jle .L4  
    xor ecx, ecx  
    mov eax, 1  
    xor esi, esi  
.L3:  
    movsx rax, DWORD PTR [rdi+rax*4]  
    add ecx, 1  
    add esi, eax  
    cmp edx, ecx  
    jne .L3  
    mov eax, esi  
    ret  
.L4:  
    xor esi, esi  
    mov eax, esi  
    ret
```

Cycle, ReadNext (Red)

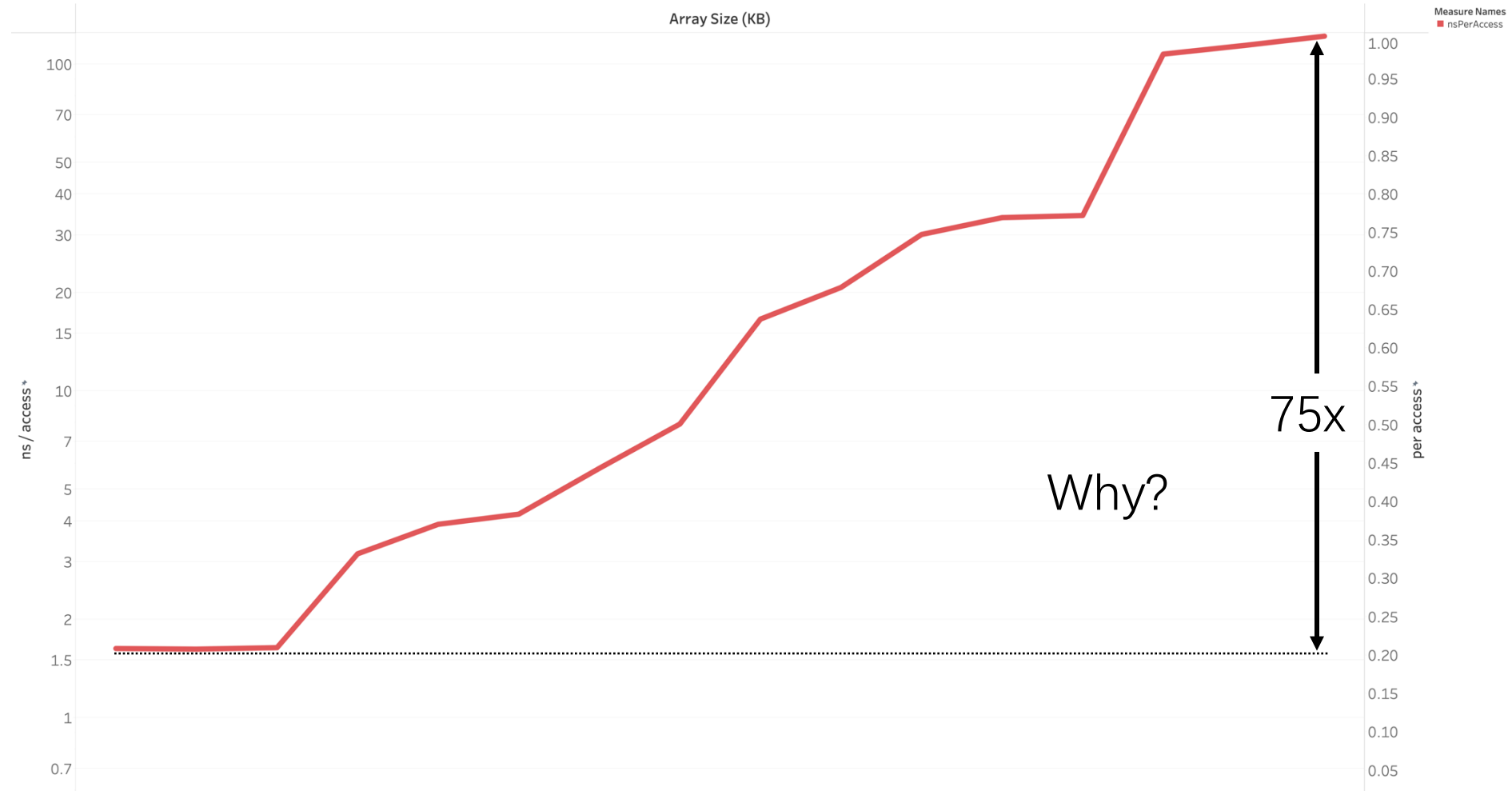
Performance Counters for Cycle / ReadNext



The trends of nsPerAccess and nsPerAccess for Array Size (KB). Color shows details about nsPerAccess. The data is filtered on Pattern and Next Index. The Pattern filter keeps Cycle. The Next Index filter keeps ReadNext.

Cycle, ReadNext (Red)

Performance Counters for Cycle / ReadNext



The trends of nsPerAccess and nsPerAccess for Array Size (KB). Color shows details about nsPerAccess. The data is filtered on Pattern and Next Index. The Pattern filter keeps Cycle. The Next Index filter keeps ReadNext.

Data Dependency

```
int cycleWithReadNext(int *A, int p, int iterations) {  
    int sum = 0;  
    int n = 1;  
    for (int i = 0; i < iterations; i++) {  
        sum += A[n];  
        n = A[n];  
    }  
    return sum;  
}
```



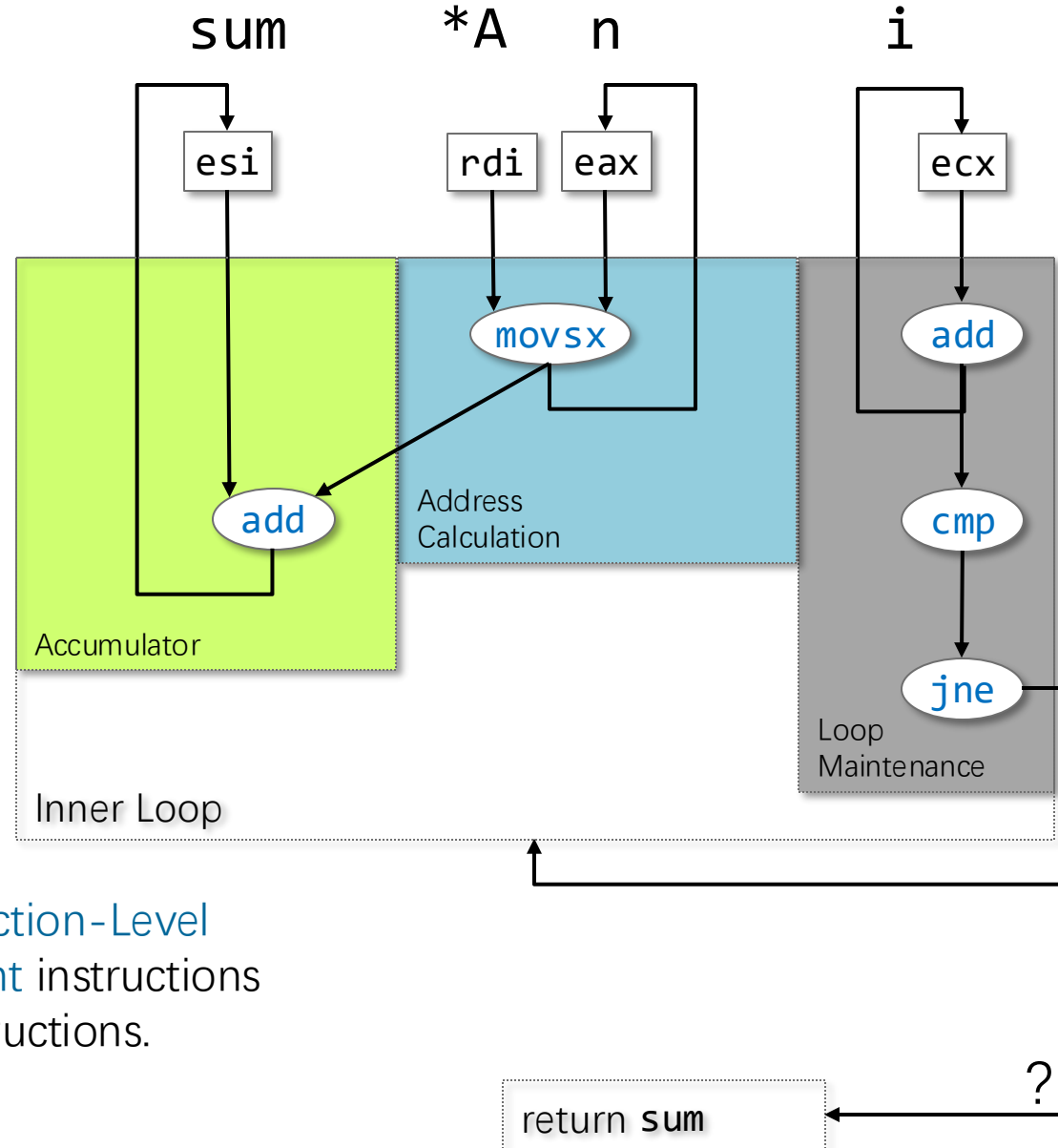
Loop-carried
dependence goes
through **memory**

```
cycleWithReadNext(int*, int, int):  
    test edx, edx  
    jle .L4  
    xor ecx, ecx  
    mov eax, 1  
    xor esi, esi  
    .L3:  
    movsx rax, DWORD PTR [rdi+rax*4]  
    add ecx, 1  
    add esi, eax  
    cmp edx, ecx  
    jne .L3  
    mov eax, esi  
    ret  
    .L4:  
    xor esi, esi  
    mov eax, esi  
    ret
```

OoO Execution

```
for (int i = 0; i < iterations; i++) {  
    sum += A[n];  
    n = A[n];  
}
```

```
.L3:  
    movsx rax, DWORD PTR [rdi+rax*4]  
    add ecx, 1  
    add esi, eax  
    cmp edx, ecx  
    jne .L3  
    mov eax, esi  
    ret
```

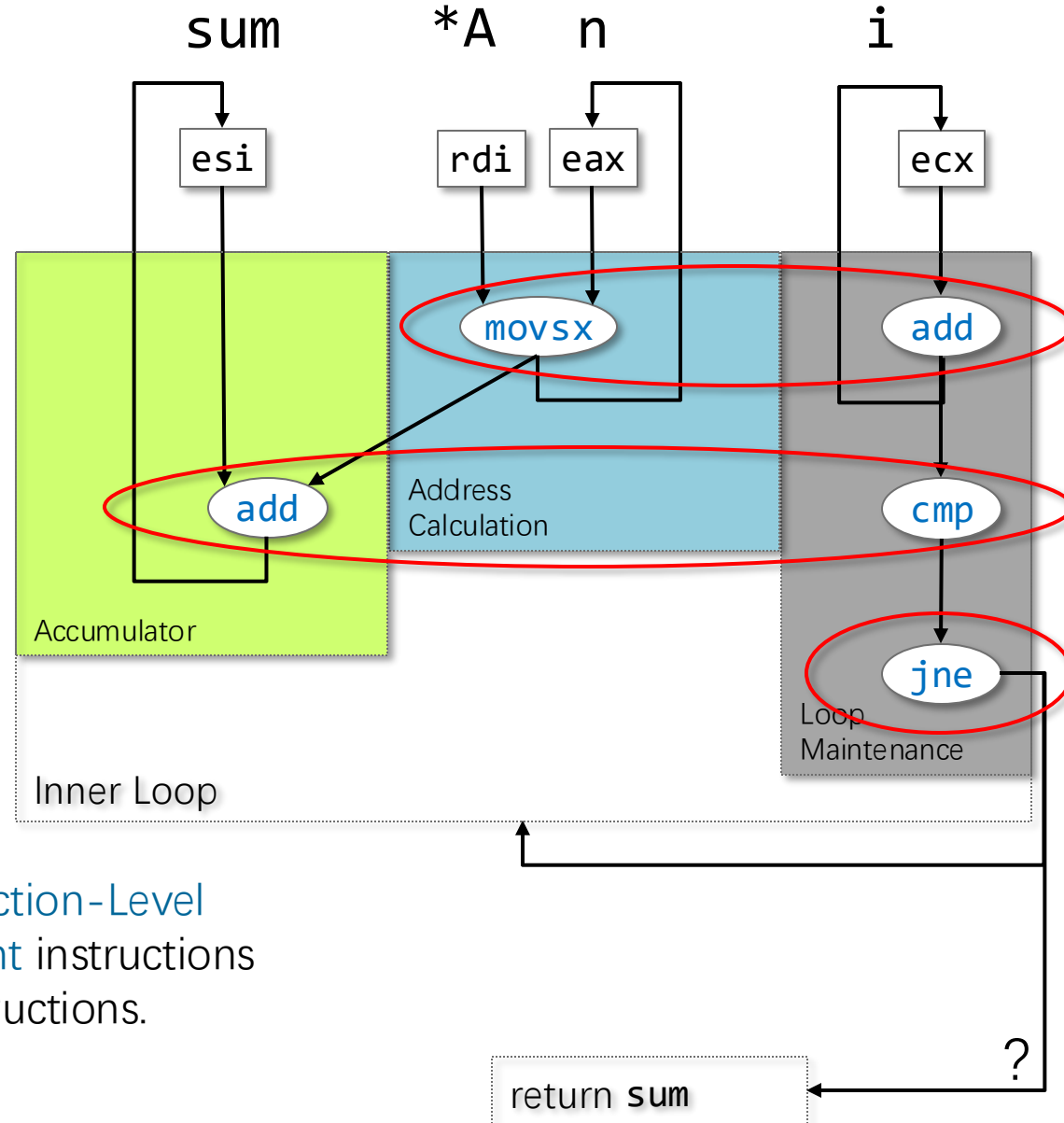


Out-of-Order (OoO) cores exploit **Instruction-Level Parallelism** (ILP) by executing **independent** instructions in the dynamically unfolding DAG of instructions.

OoO Execution

```
for (int i = 0; i < iterations; i++) {  
    sum += A[n];  
    n = A[n];  
}
```

```
.L3:  
movsx rax, DWORD PTR [rdi+rax*4]  
add ecx, 1  
add esi, eax  
cmp edx, ecx  
jne .L3  
mov eax, esi  
ret
```

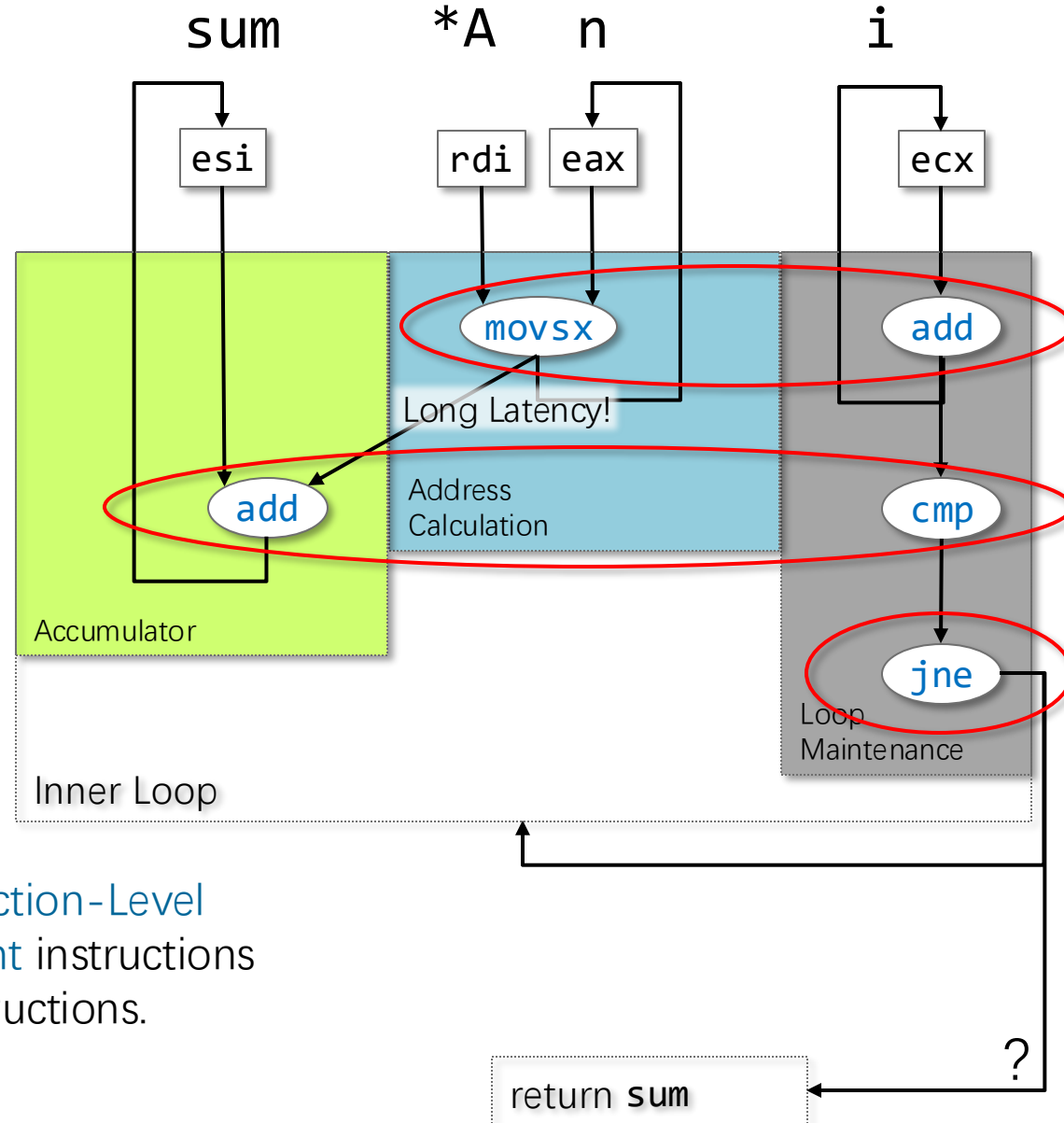


Out-of-Order (OoO) cores exploit **Instruction-Level Parallelism** (ILP) by executing **independent** instructions in the dynamically unfolding DAG of instructions.

OoO Execution

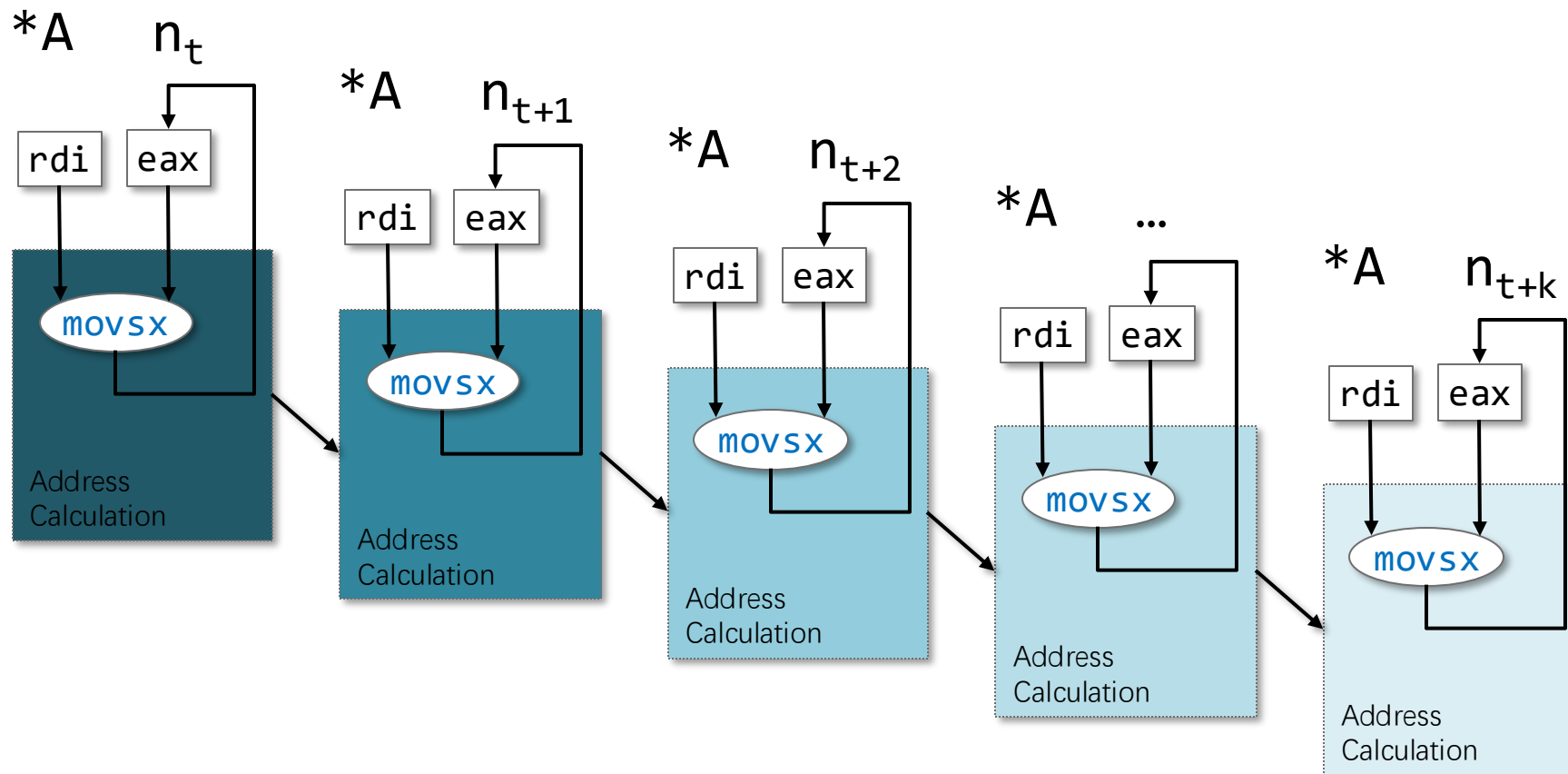
```
for (int i = 0; i < iterations; i++) {  
    sum += A[n];  
    n = A[n];  
}
```

```
.L3:  
movsx rax, DWORD PTR [rdi+rax*4]  
add ecx, 1  
add esi, eax  
cmp edx, ecx  
jne .L3  
mov eax, esi  
ret
```



Out-of-Order (OoO) cores exploit **Instruction-Level Parallelism** (ILP) by executing **independent** instructions in the dynamically unfolding DAG of instructions.

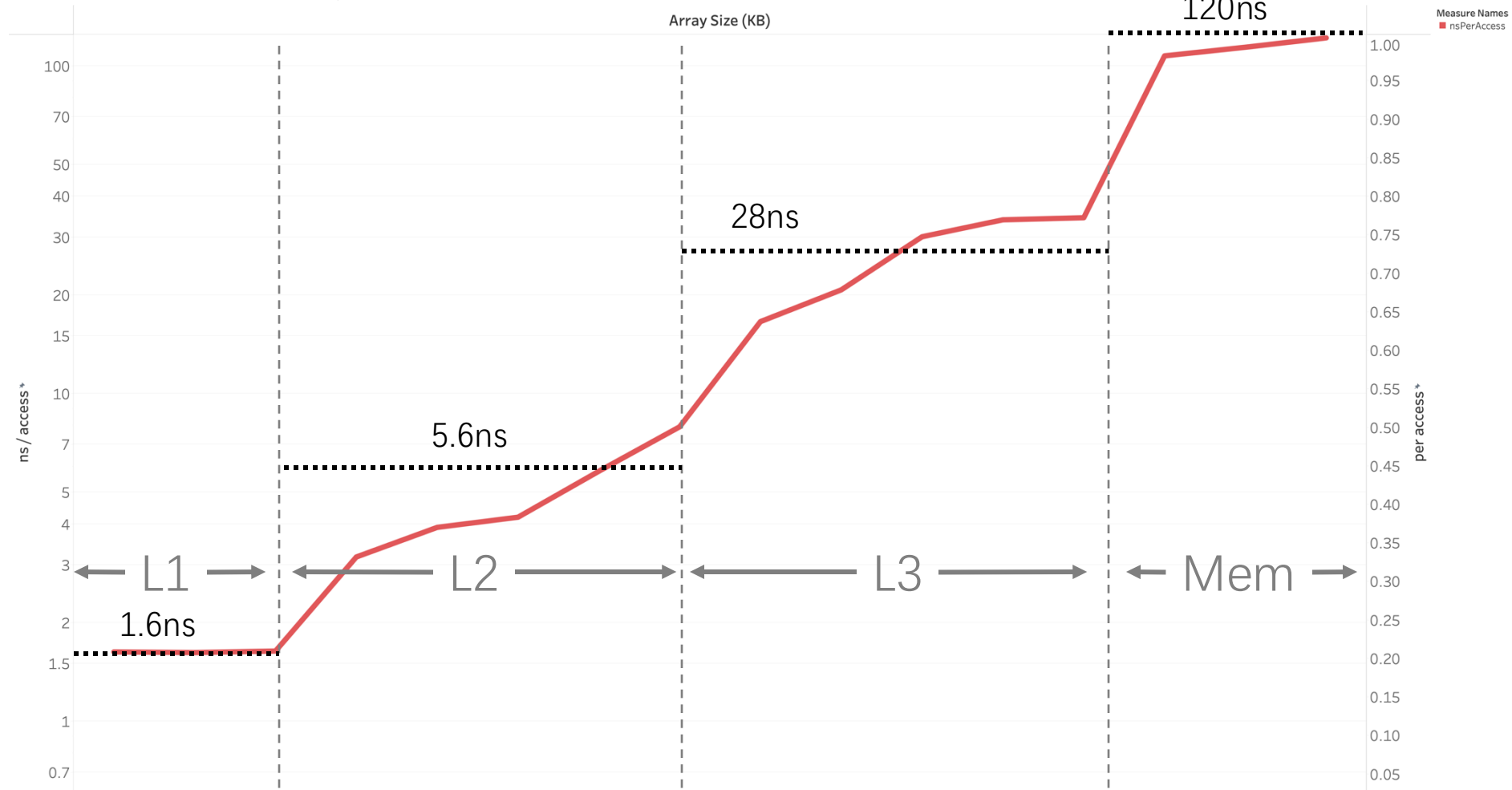
Data-Dependency



Strict data-dependency through `*A` between each iteration

Performance vs Array Size

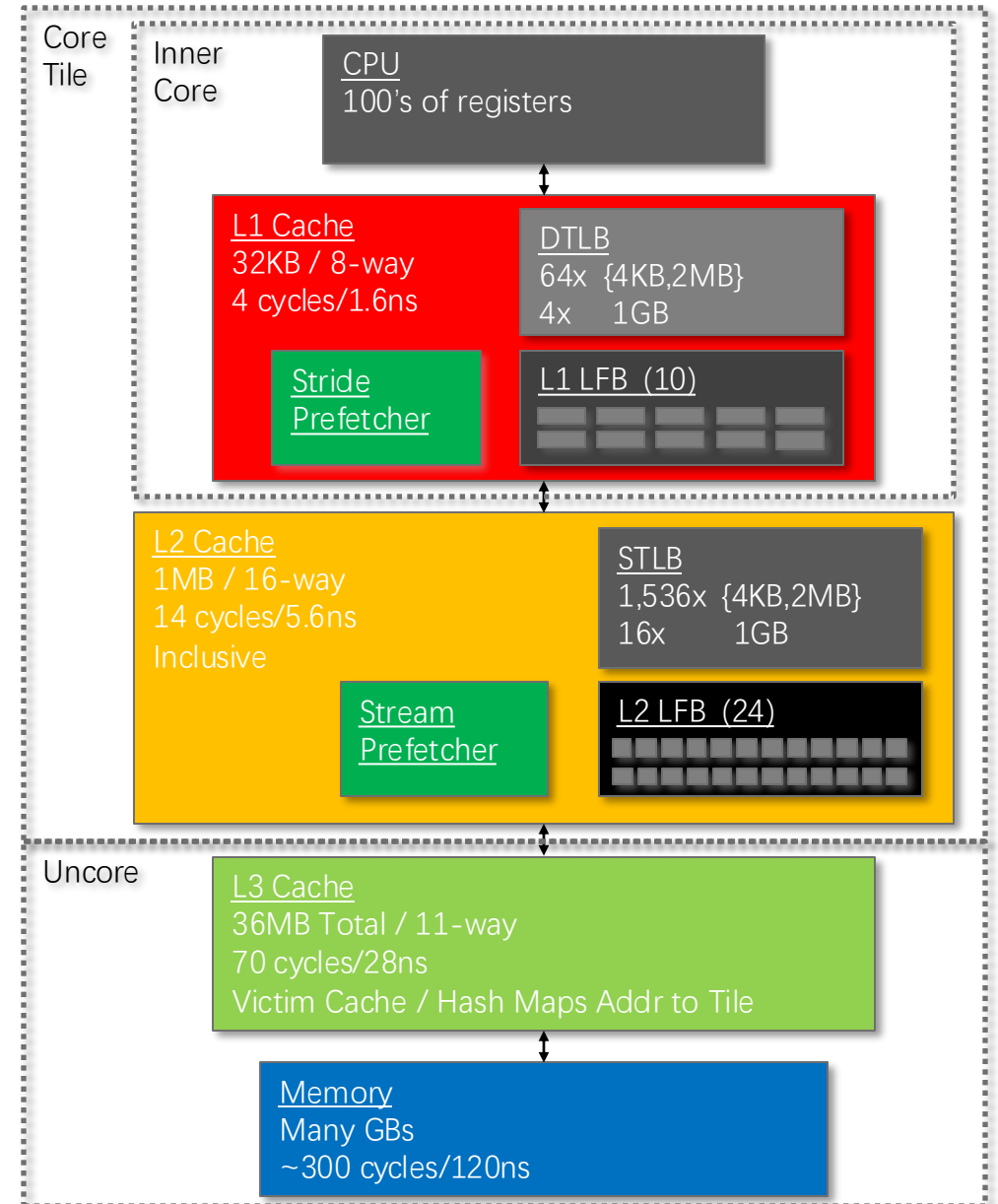
Performance Counters for Cycle / ReadNext



The trends of nsPerAccess and nsPerAccess for Array Size (KB). Color shows details about nsPerAccess. The data is filtered on Pattern and Next Index. The Pattern filter keeps Cycle. The Next Index filter keeps ReadNext.

Cache Hierarchy

- Intel's Cascade Lake is the test system for MicroBenchmarks in this lecture.



PERFORMANCE COUNTERS



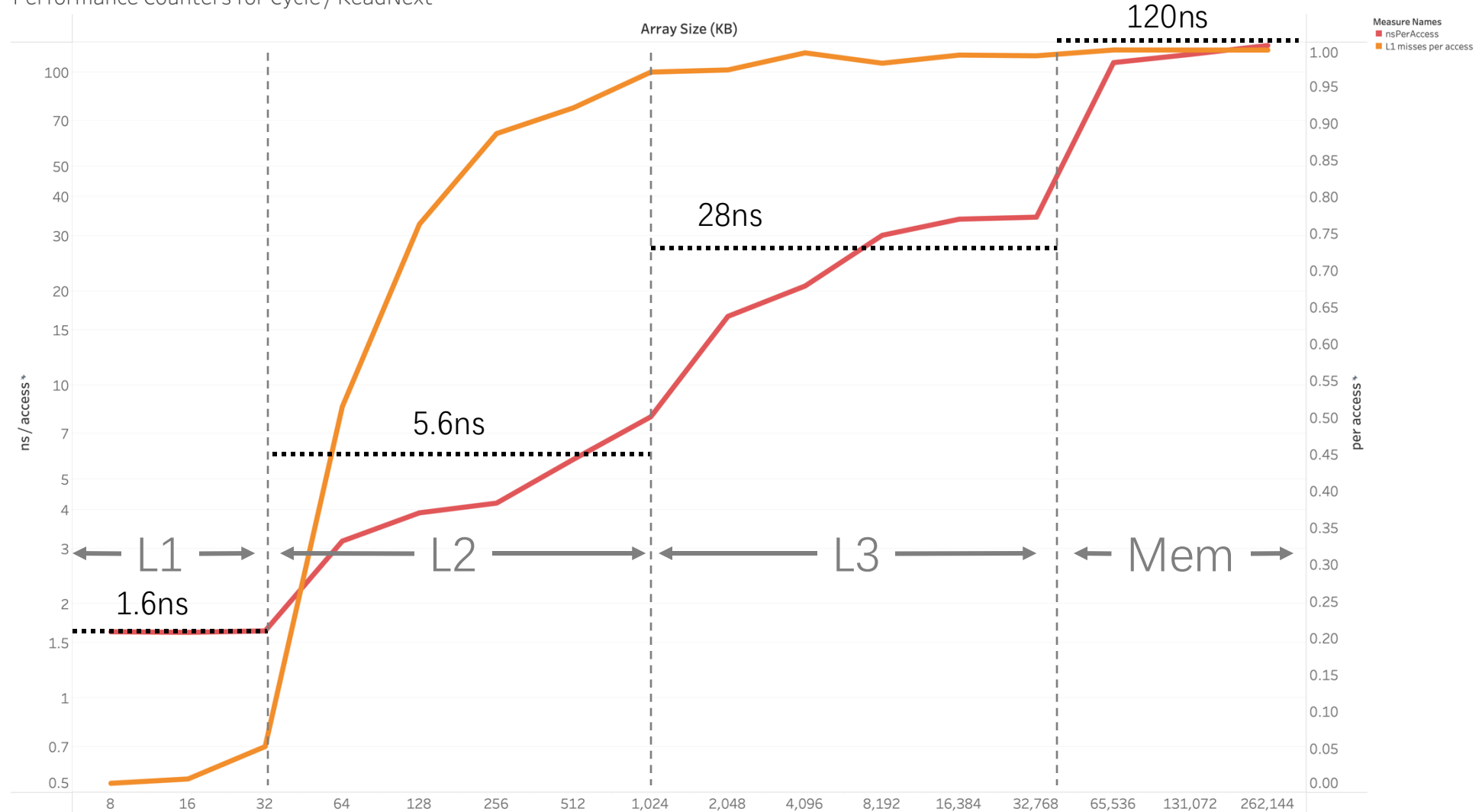
Performance Counters

Modern processors have a lot of performance counters that track all manner of **events** that are relevant to improving performance, e.g.,

- Cycles
- Cycles spent stalled waiting for memory
- Cycles stalled waiting for instruction cache
- Number of loads outstanding
- Cache misses at each cache level
- TLB misses
- Branch mispredictions
- etc.

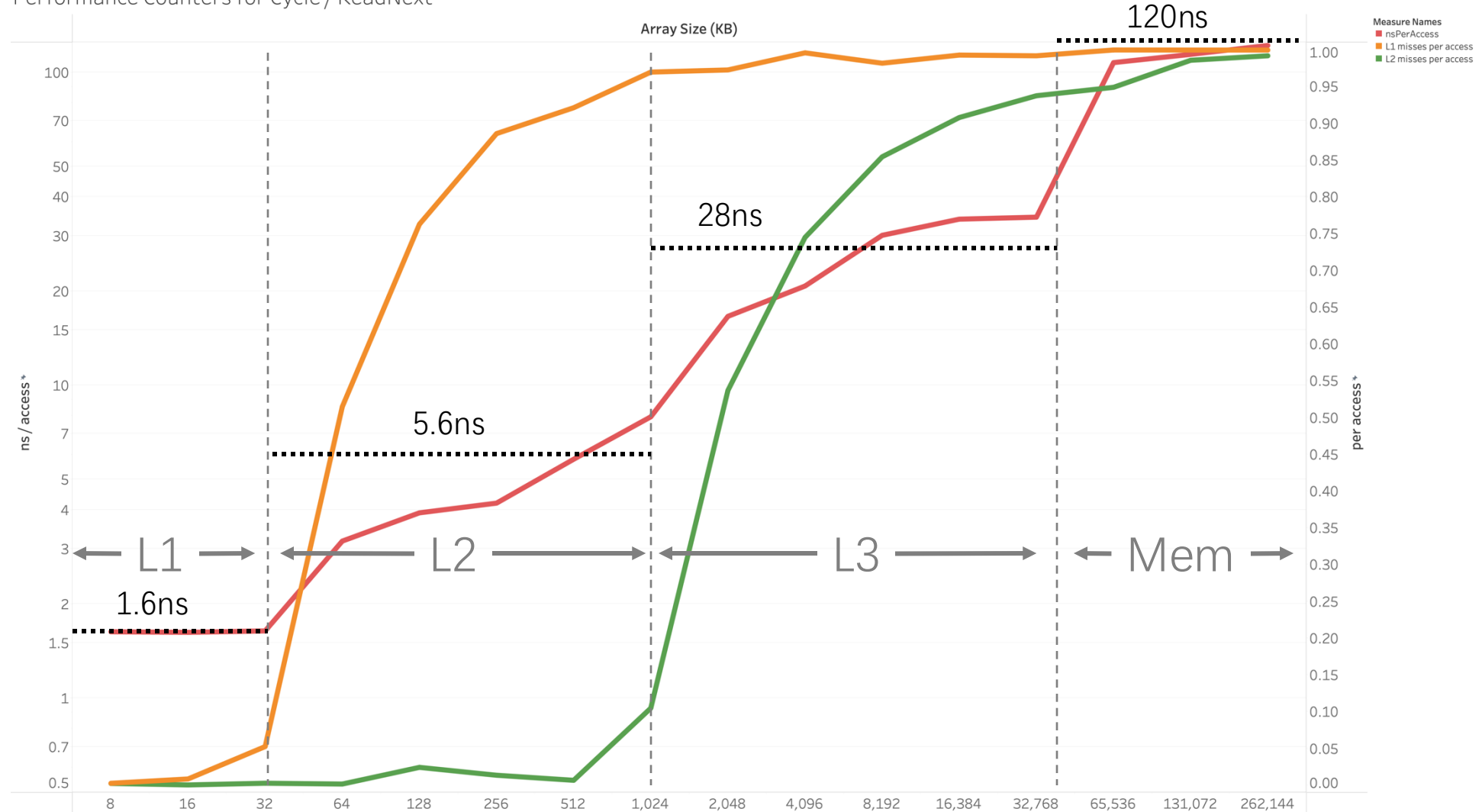
L1 Misses (Orange)

Performance Counters for Cycle / ReadNext



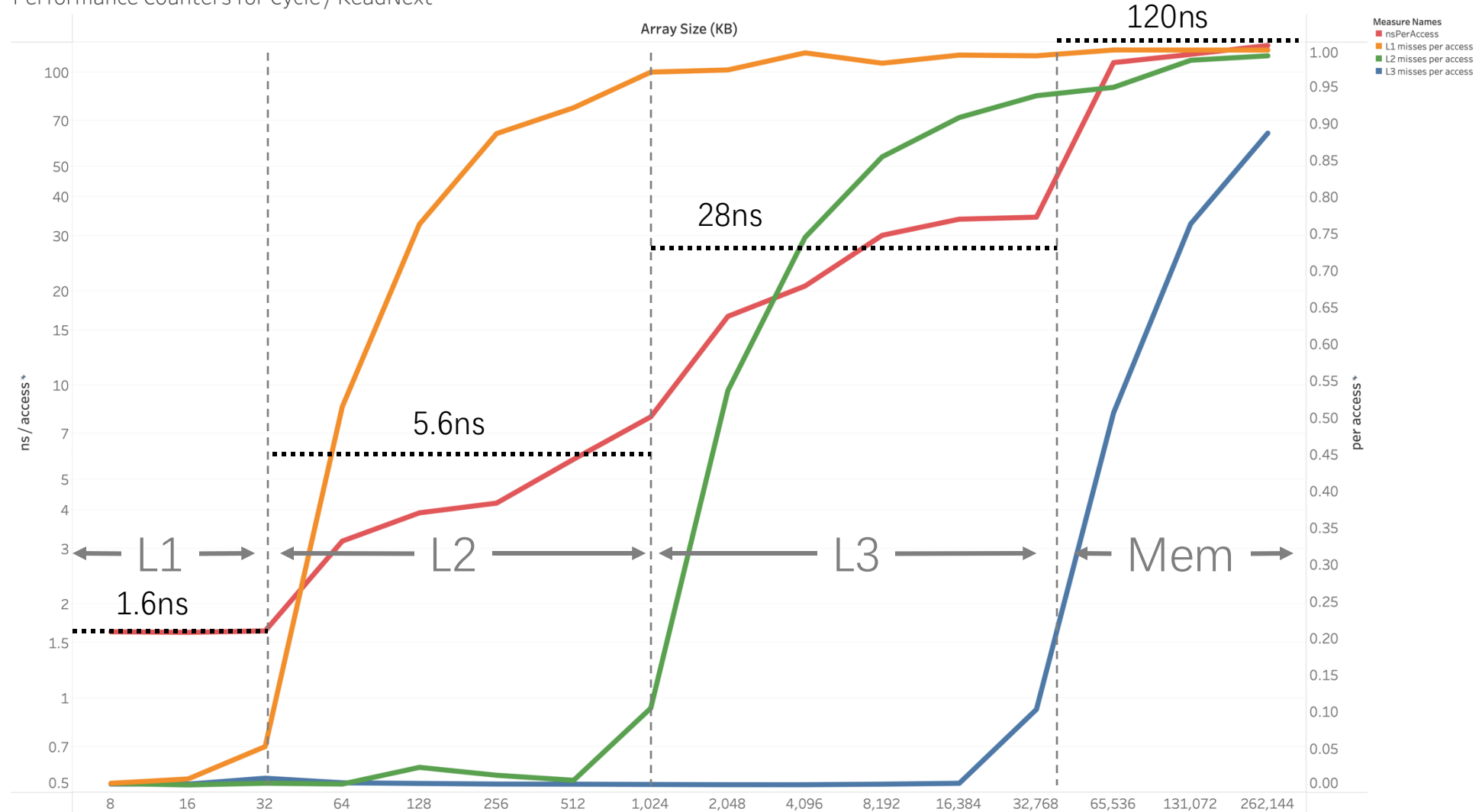
L2 Misses (Green)

Performance Counters for Cycle / ReadNext



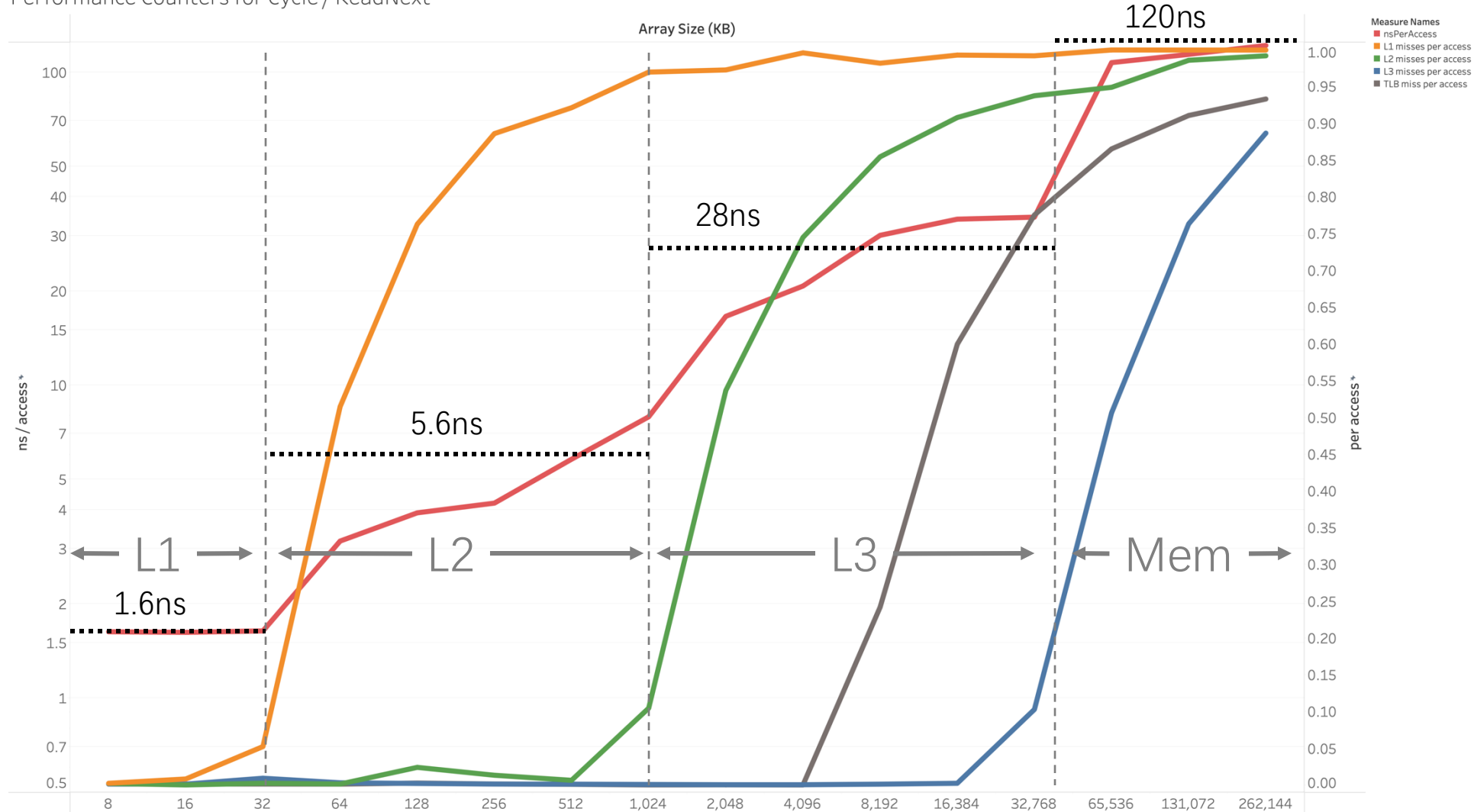
L3 Misses (Blue)

Performance Counters for Cycle / ReadNext



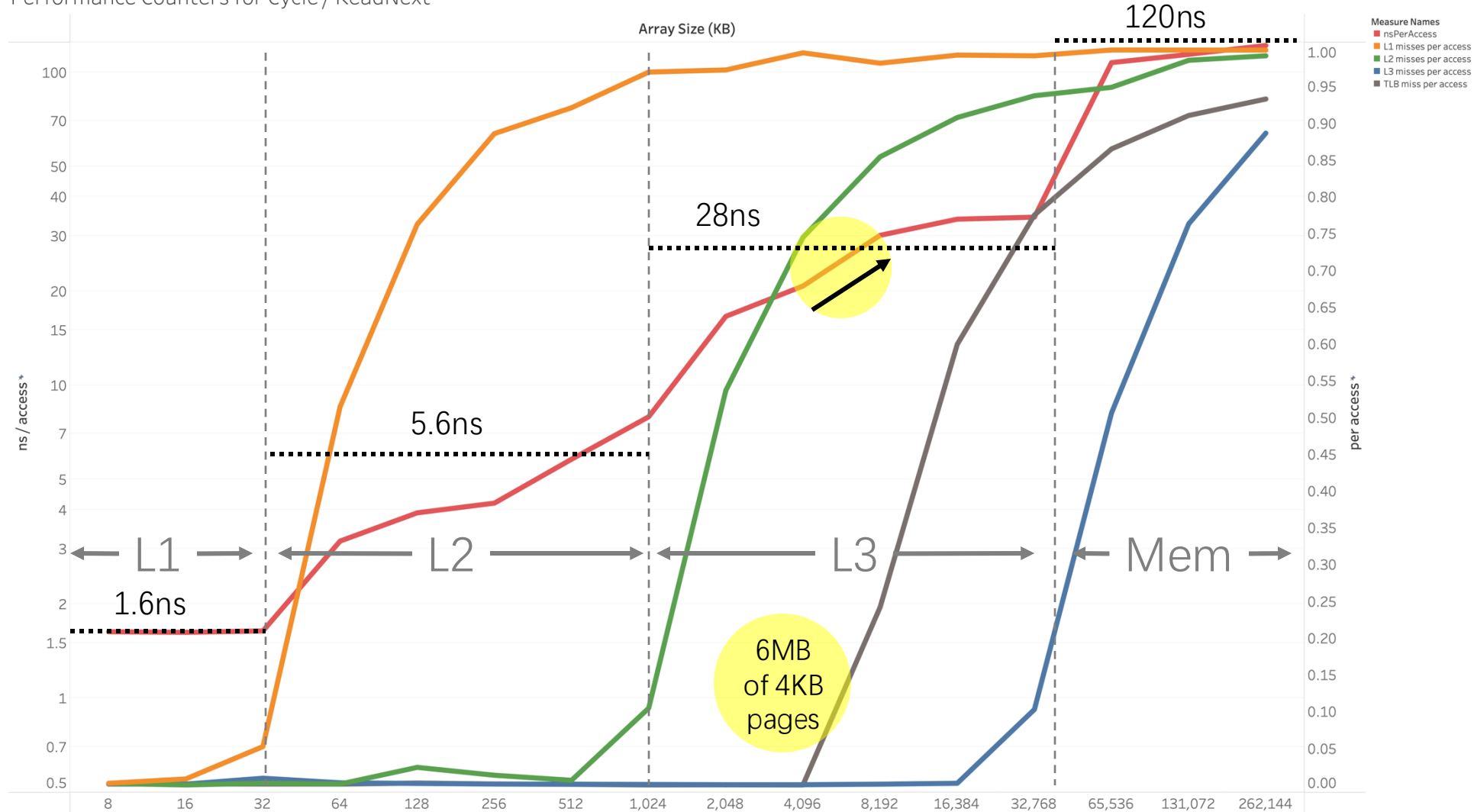
TLB Misses (Grey)

Performance Counters for Cycle / ReadNext



TLB Misses (Grey)

Performance Counters for Cycle / ReadNext



CALCULATE NEXT ADDRESS



Cycle, CalculateNext

```
int cycleWithCalcNext(int *A, int p, int iterations) {  
    int sum = 0;  
    int n = 1;  
    for (int i = 0; i < iterations; i++) {  
        sum += A[n];  
        n = 2 * n;  
        n = n > p ? n - p : n;  
    }  
    return sum;  
}
```

5 more instructions
than ReadNext

```
cycleWithCalcNext(int*, int, int):  
    test edx, edx  
    jle .L5  
    xor ecx, ecx  
    mov eax, 1  
    xor r8d, r8d  
.L4:  
    movsx r9, eax  
    add eax, eax  
    add r8d, DWORD PTR [rdi+r9*4]  
    mov r9d, eax  
    sub r9d, esi  
    cmp esi, eax  
    cmovl eax, r9d  
    add ecx, 1  
    cmp edx, ecx  
    jne .L4  
    mov eax, r8d  
    ret  
.L5:  
    xor r8d, r8d  
    mov eax, r8d  
    ret
```

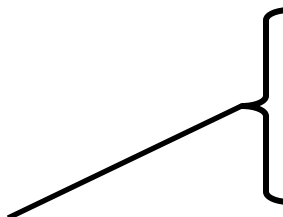
Cycle, CalculateNext

```
int cycleWithCalcNext(int *A, int p, int iterations) {  
    int sum = 0;  
    int n = 1;  
    for (int i = 0; i < iterations; i++) {  
        sum += A[n];  
        n = 2 * n;  
        n = n > p ? n - p : n;  
    }  
    return sum;  
}
```



Loop-carried dependence
goes through a register

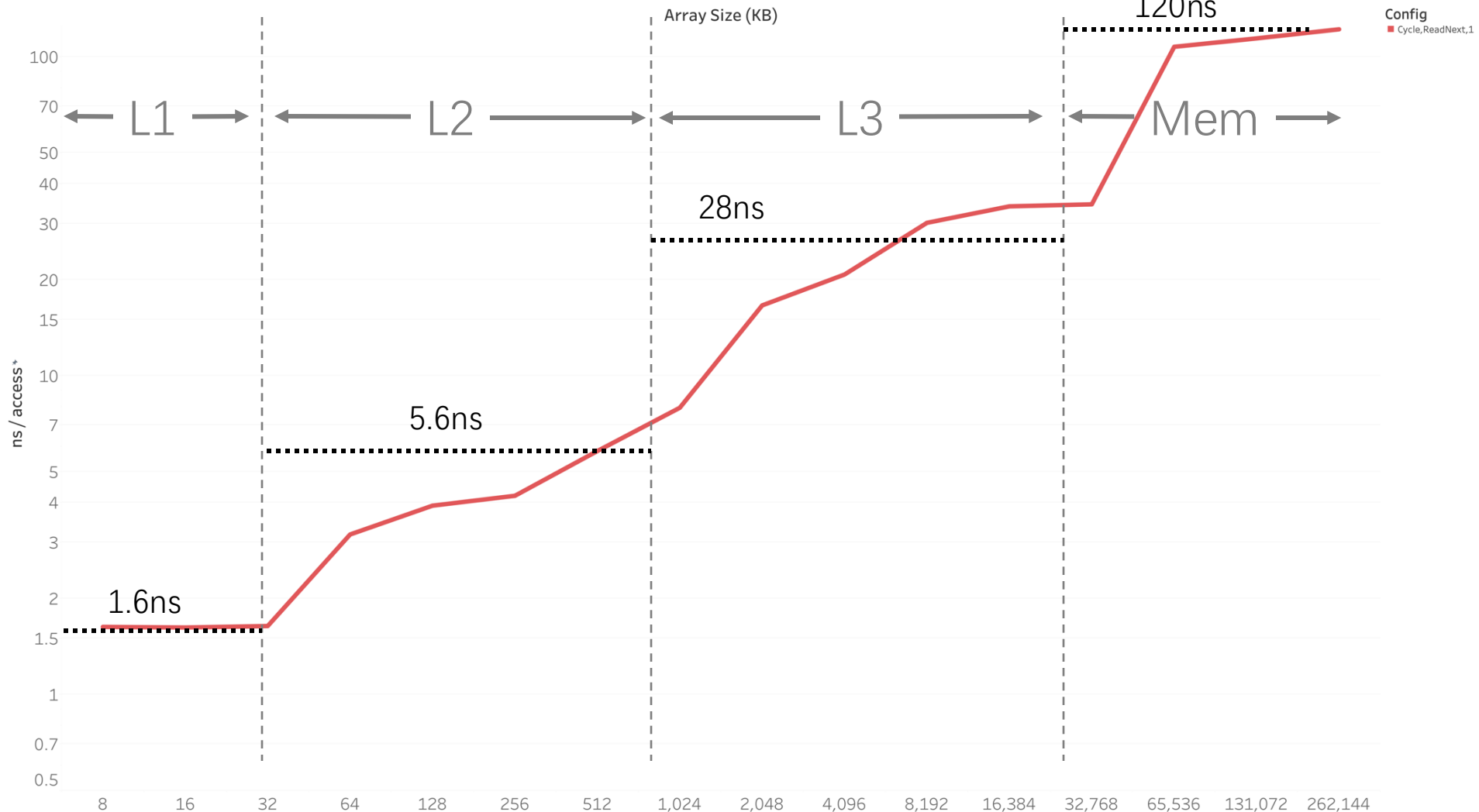
5 more instructions
than ReadNext



```
cycleWithCalcNext(int*, int, int):  
    test edx, edx  
    jle .L5  
    xor ecx, ecx  
    mov eax, 1  
    xor r8d, r8d  
.L4:  
    movsx r9, eax  
    add eax, eax  
    add r8d, DWORD PTR [rdi+r9*4]  
    mov r9d, eax  
    sub r9d, esi  
    cmp esi, eax  
    cmovl eax, r9d  
    add ecx, 1  
    cmp edx, ecx  
    jne .L4  
    mov eax, r8d  
    ret  
.L5:  
    xor r8d, r8d  
    mov eax, r8d  
    ret
```

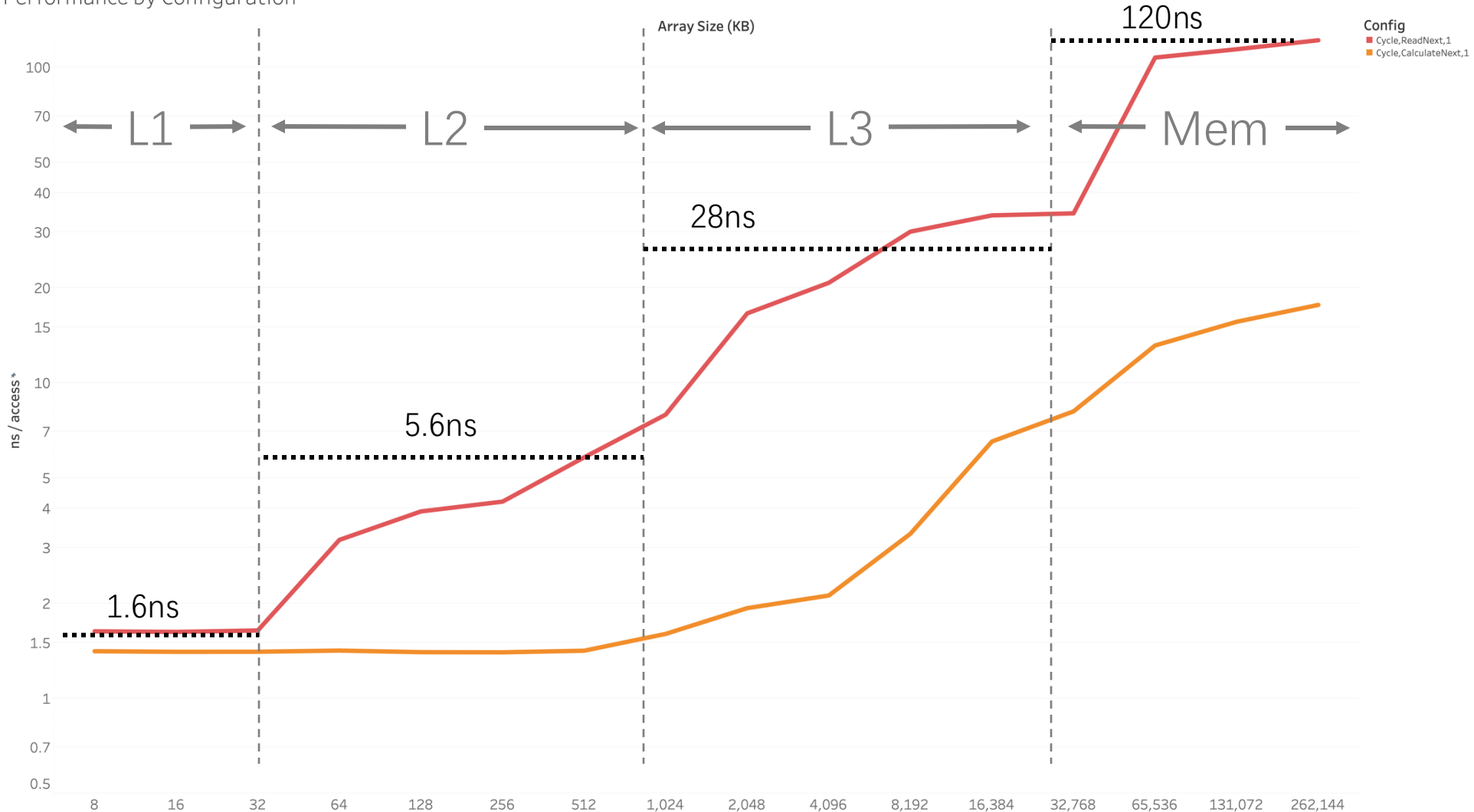
Cycle / ReadNext (Red)

Performance by Configuration

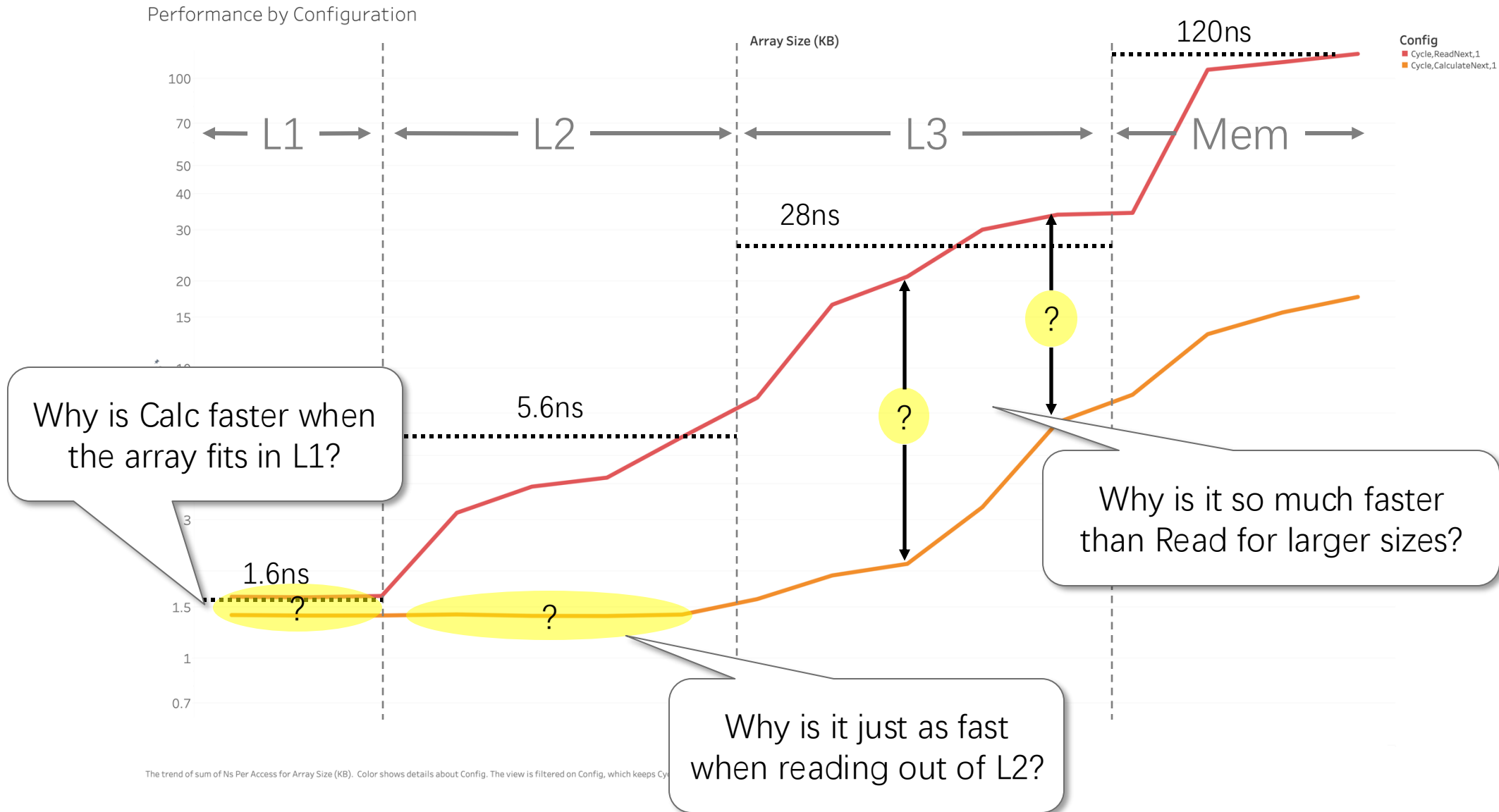


Cycle, CalculateNext (Orange)

Performance by Configuration



Cycle, CalculateNext (Orange)

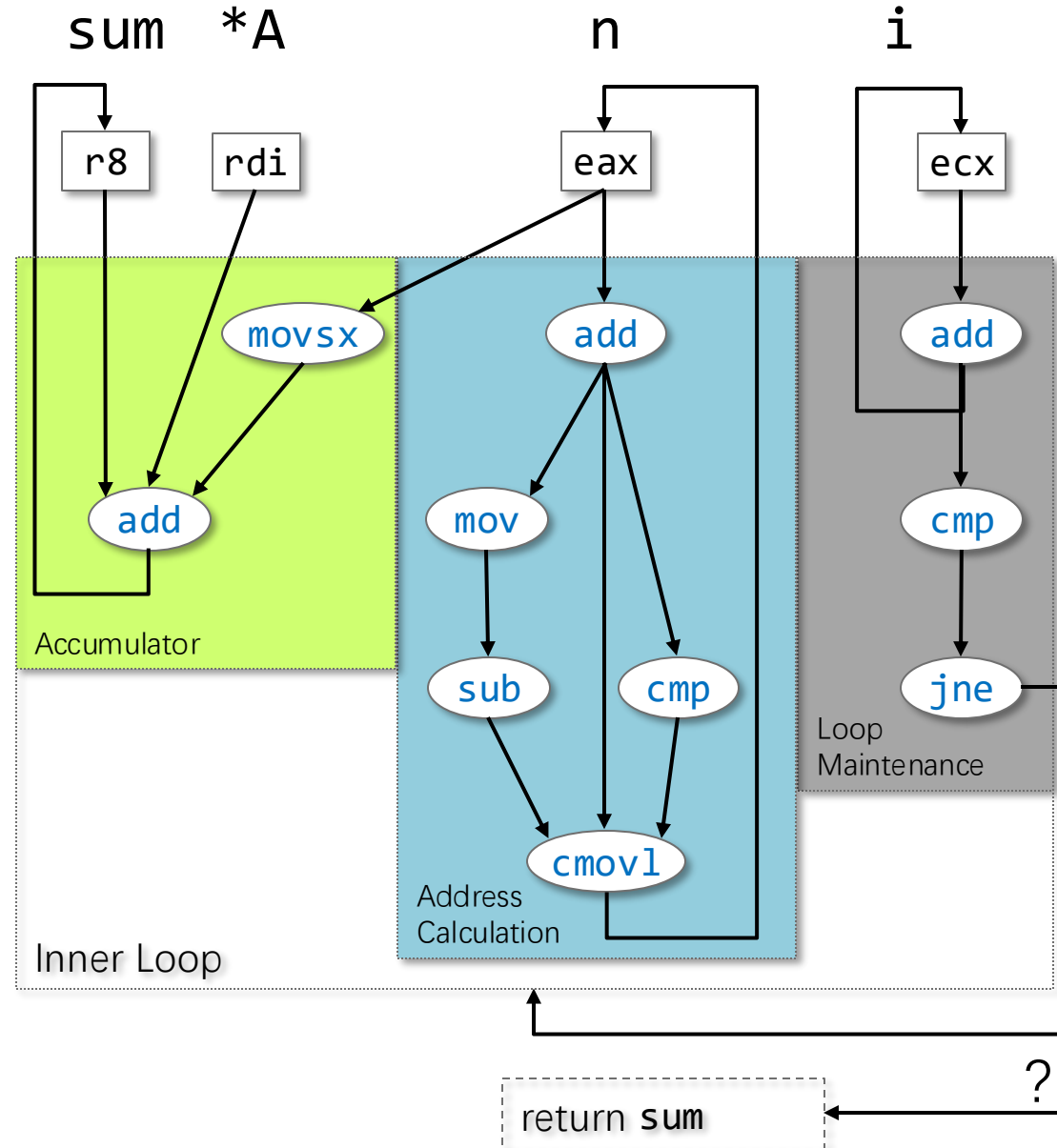


OoO Execution

```
for (int i = 0; i < iterations; i++)  
{  
    sum += A[n];  
    n = 2 * n;  
    n = n > p ? n - p : n;  
}
```

.L4:

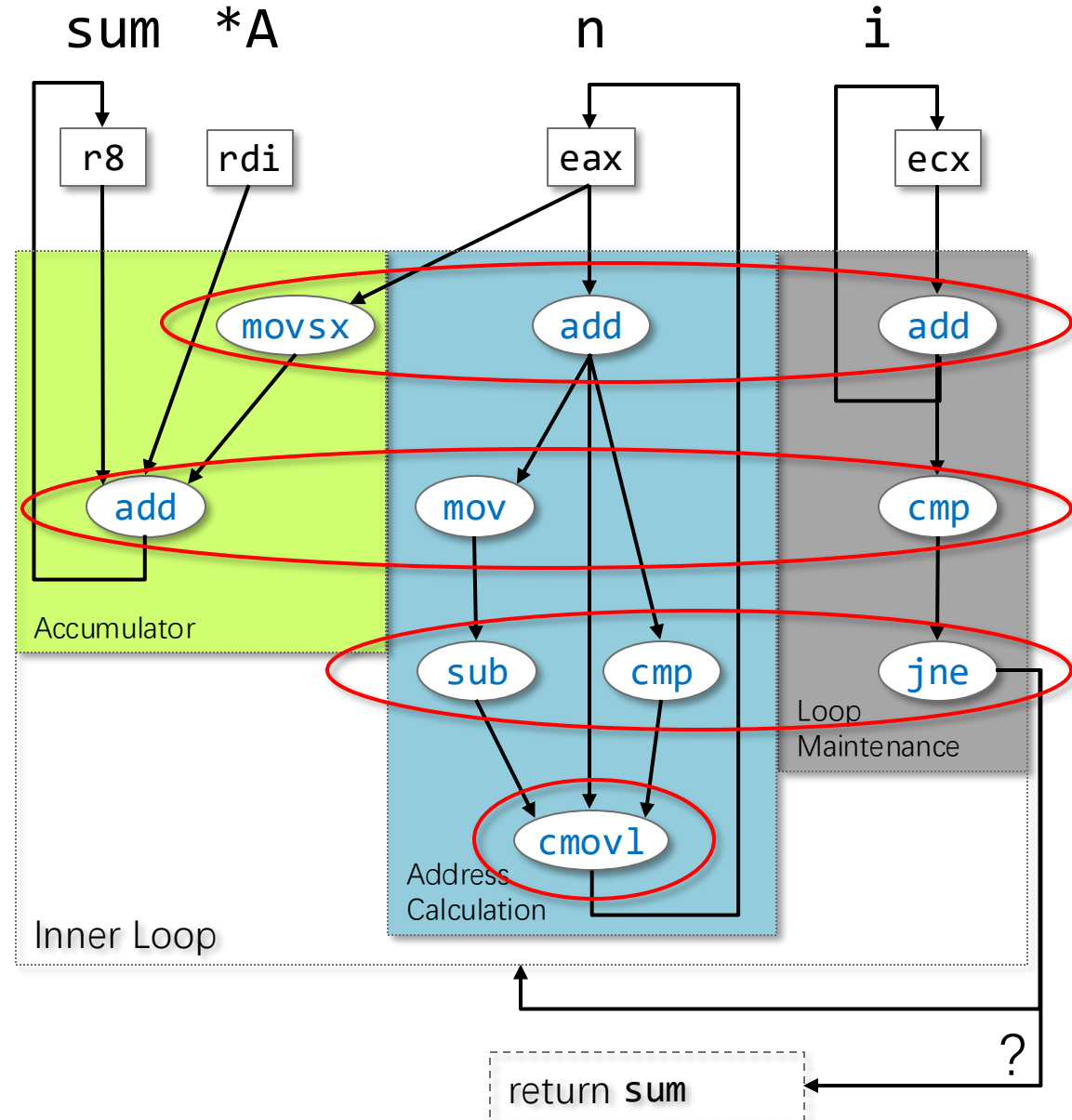
```
movsx r9, eax  
add eax, eax  
add r8d, DWORD PTR [rdi+r9*4]  
mov r9d, eax  
sub r9d, esi  
cmp esi, eax  
cmovl eax, r9d  
add ecx, 1  
cmp edx, ecx  
jne .L4  
mov eax, r8d  
ret
```



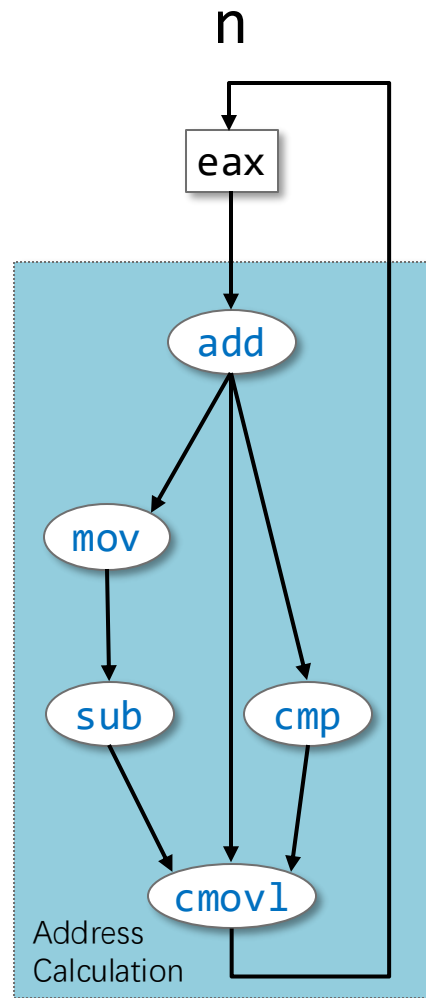
OoO Execution

```
for (int i = 0; i < iterations; i++)  
{  
    sum += A[n];  
    n = 2 * n;  
    n = n > p ? n - p : n;  
}
```

```
.L4:  
    movsx r9, eax  
    add eax, eax  
    add r8d, DWORD PTR [rdi+r9*4]  
    mov r9d, eax  
    sub r9d, esi  
    cmp esi, eax  
    cmovl eax, r9d  
    add ecx, 1  
    cmp edx, ecx  
    jne .L4  
    mov eax, r8d  
    ret
```

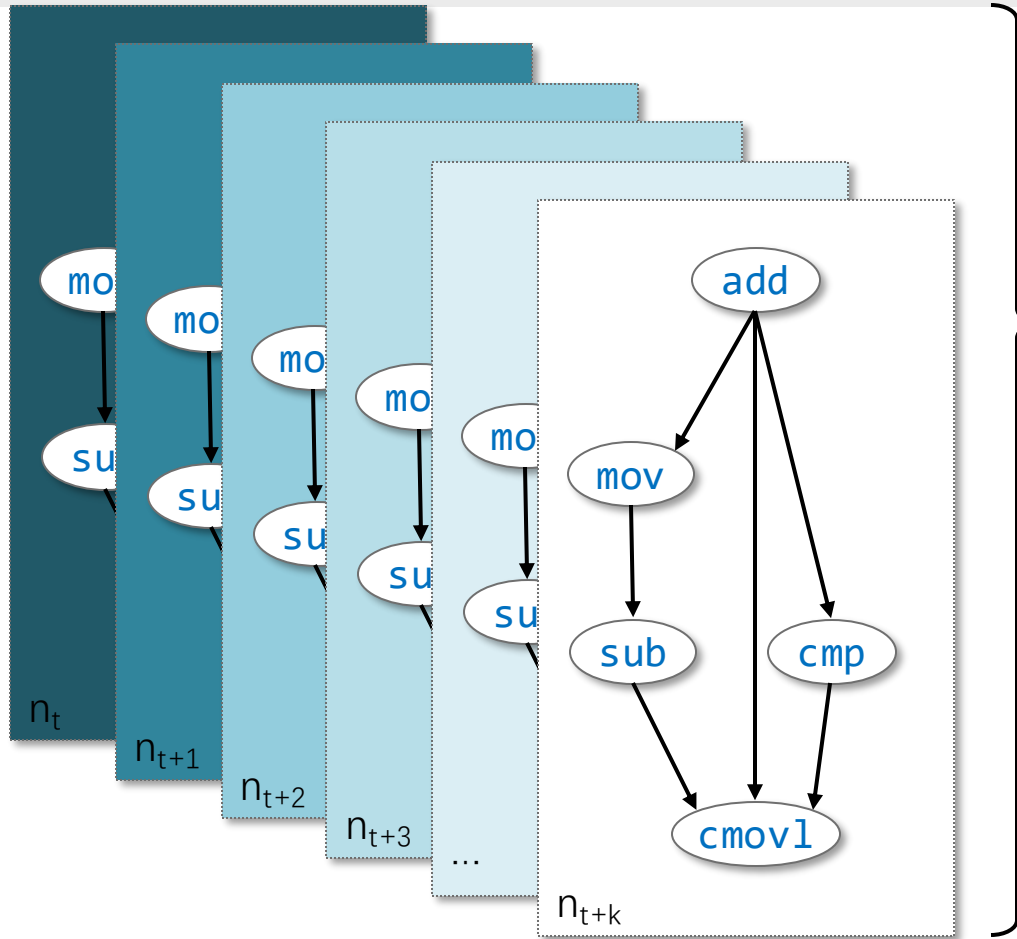


OoO Execution

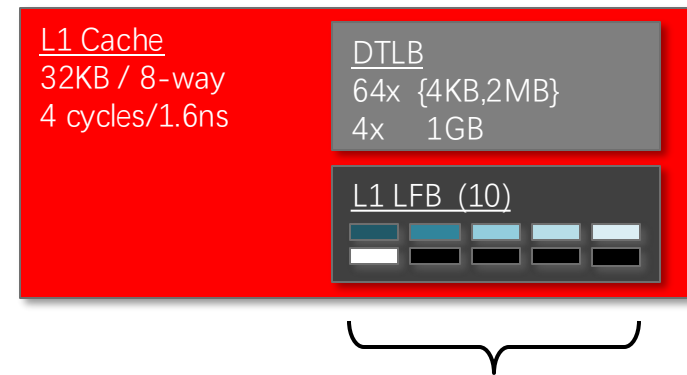


Address calculation is now independent of the contents of *A.

OoO Execution



Indices for several iterations can be calculated in quick succession



Loads for $A[n_t] \dots A[n_{t+k}]$ can go out in parallel for $k \leq 10$, each of which consumes one Line Fill Buffer (LFB).

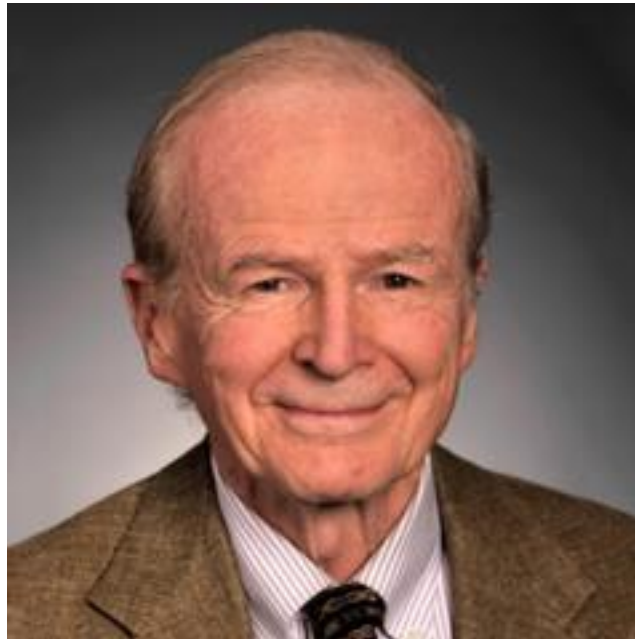
MEMORY-LEVEL PARALLELISM



Little's Law

$$L = \lambda W$$

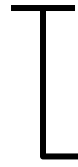
Concept from queueing theory often applied to shopping.



John D.C. Little

Little's Law

$$L = \lambda W$$

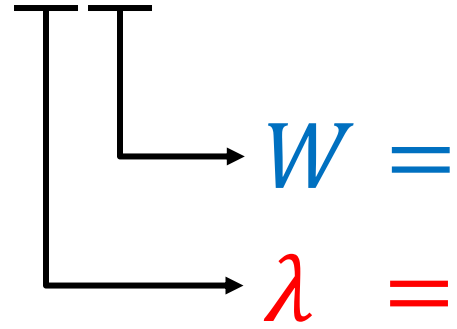


$$W =$$

Average service time

Little's Law

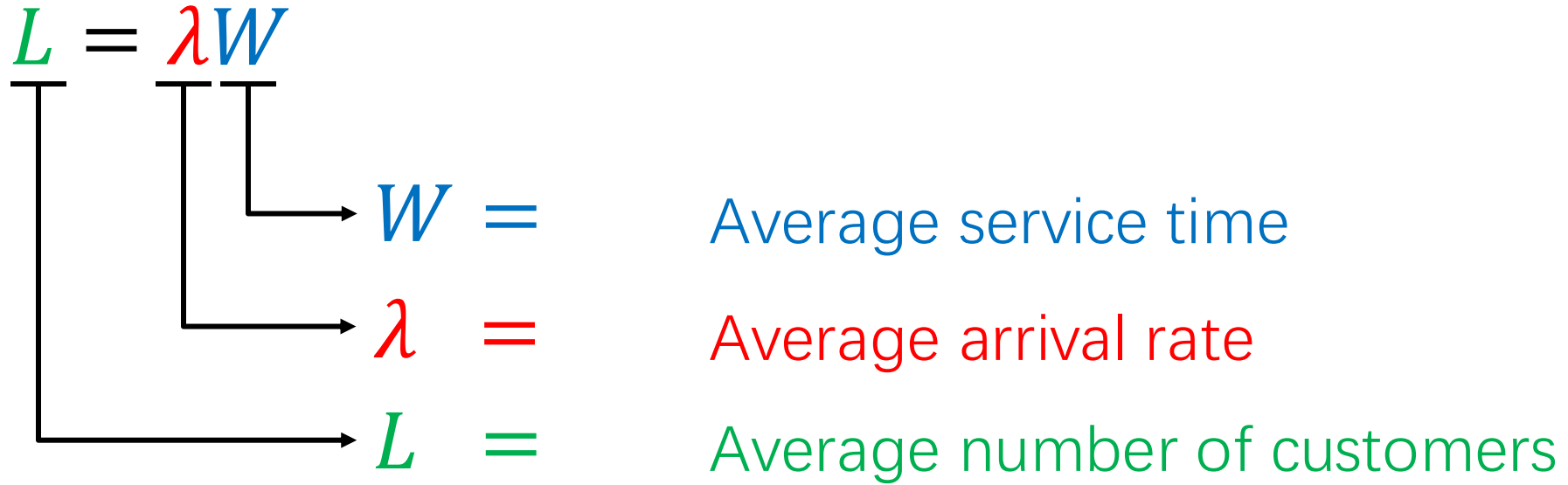
$$L = \lambda W$$



Average service time

Average arrival rate

Little's Law



Little's Law

$$\frac{W}{L} = \lambda^{-1}$$

W = Average latency (ns)

λ^{-1} = nsPerAccess

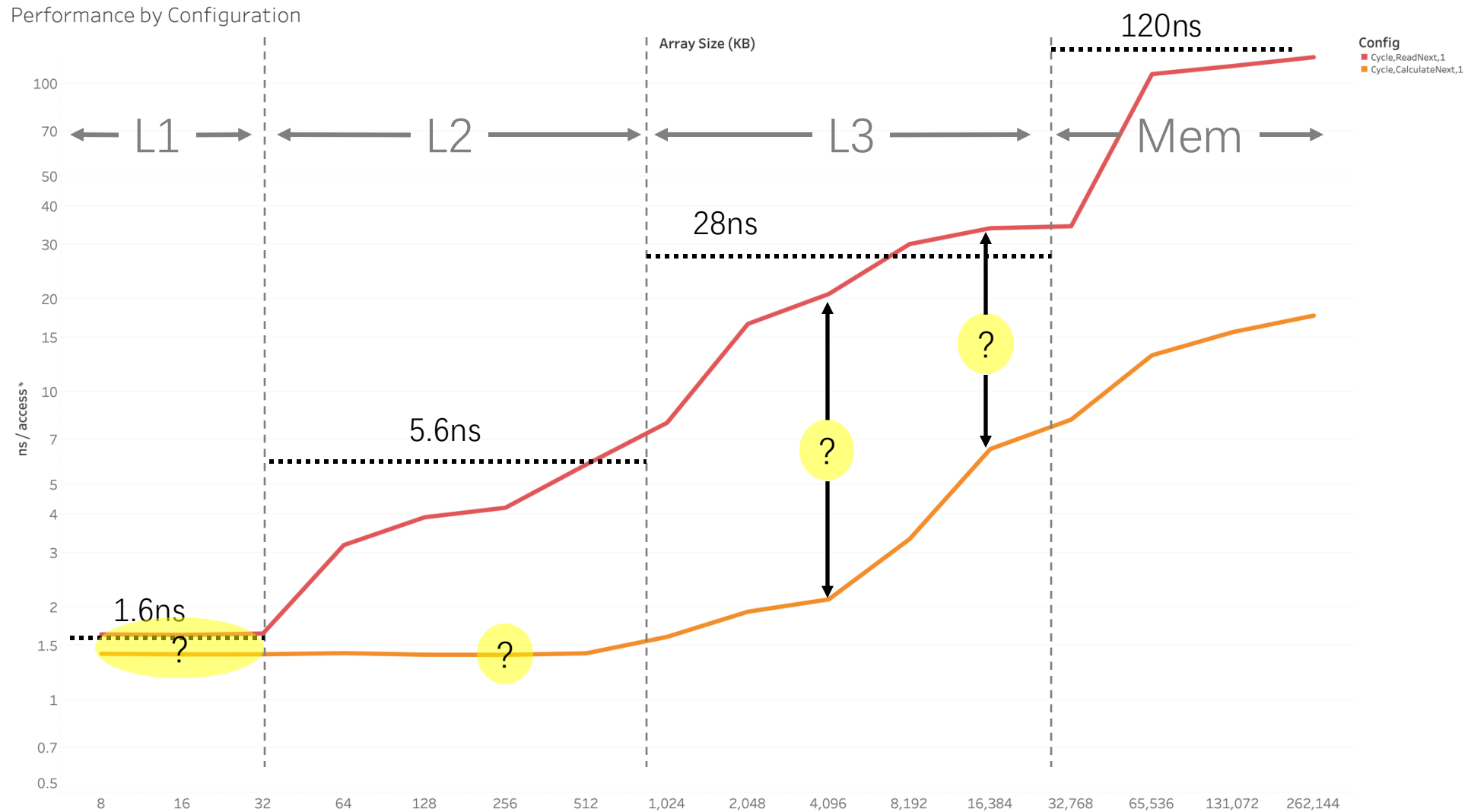
L = Memory-Level Parallelism

We can also use it to reason about **Memory-Level Parallelism** in a processor

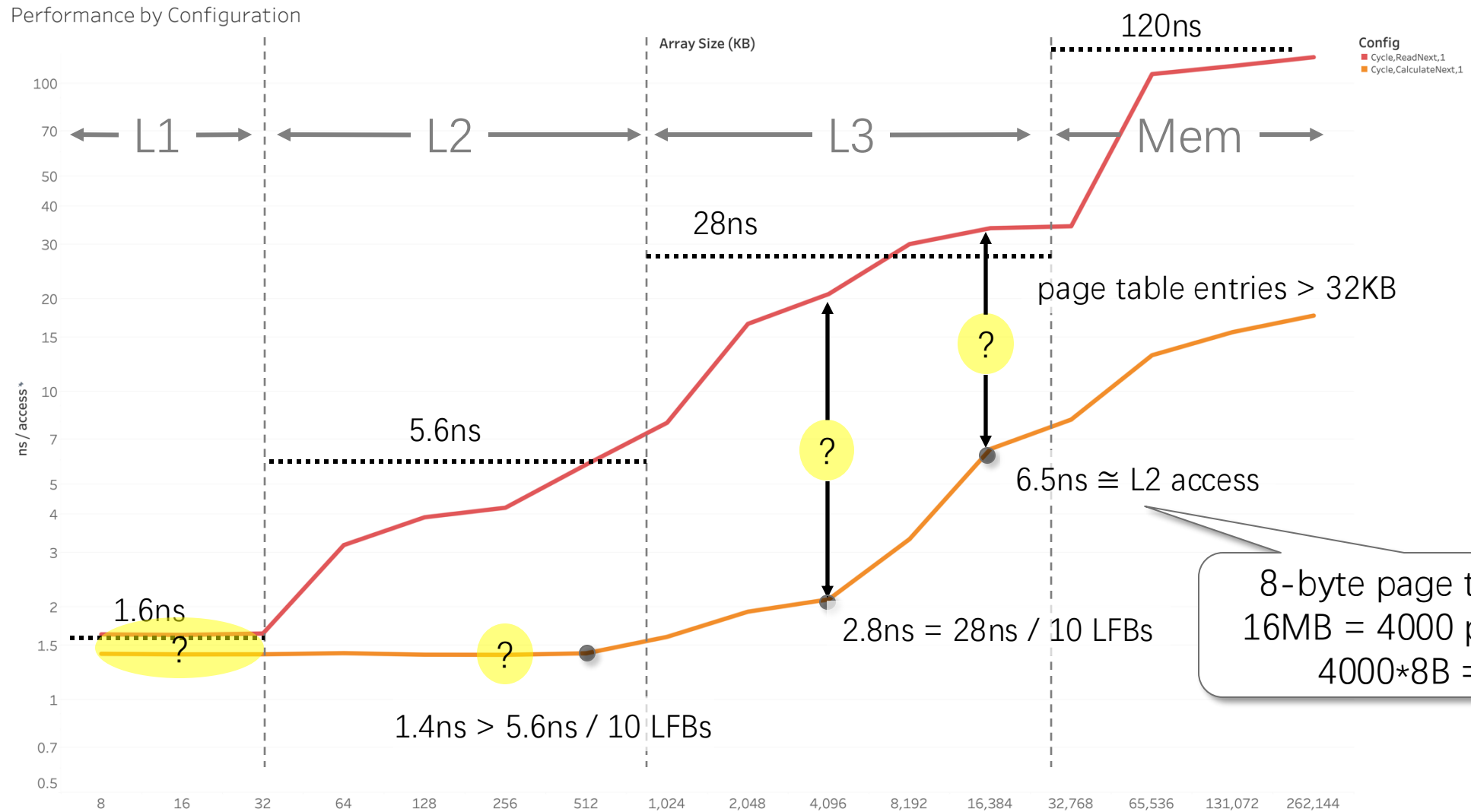


Loads for $A[n_t] \dots A[n_{t+k}]$ can go out in parallel for $k \leq 10$, each of which consumes one Line Fill Buffer (LFB).

Cycle, CalculateNext (Orange)



Cycle, CalculateNext (Orange)



What if we cheat?

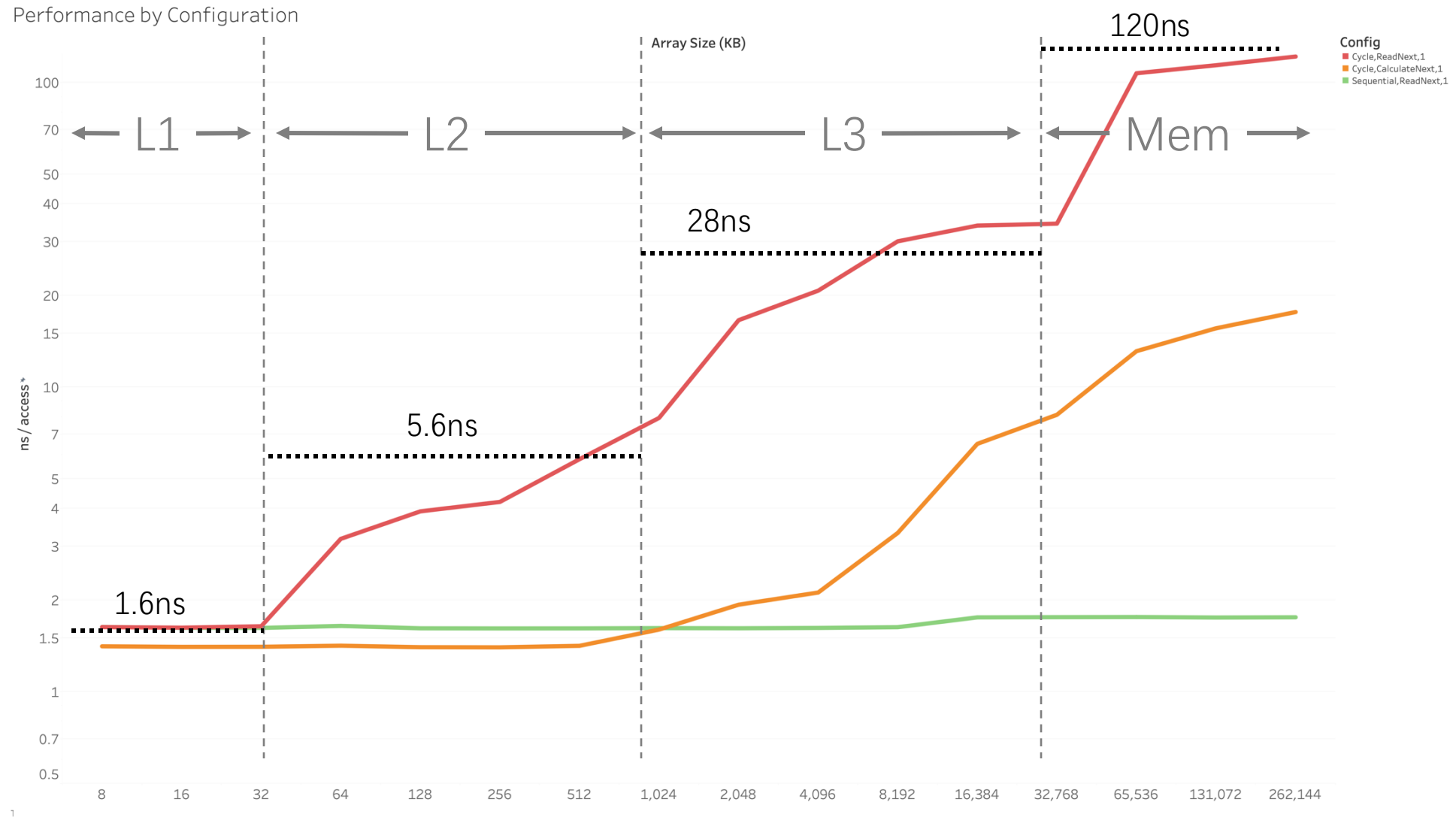
```
int sequentialWithReadNext(int *A, int p, int iterations) {  
    int sum = 0;  
    int n = 1;  
    for (int i = 0; i < iterations; i++) {  
        sum += A[n];  
        n = A[n];  
    }  
    return sum;  
}
```



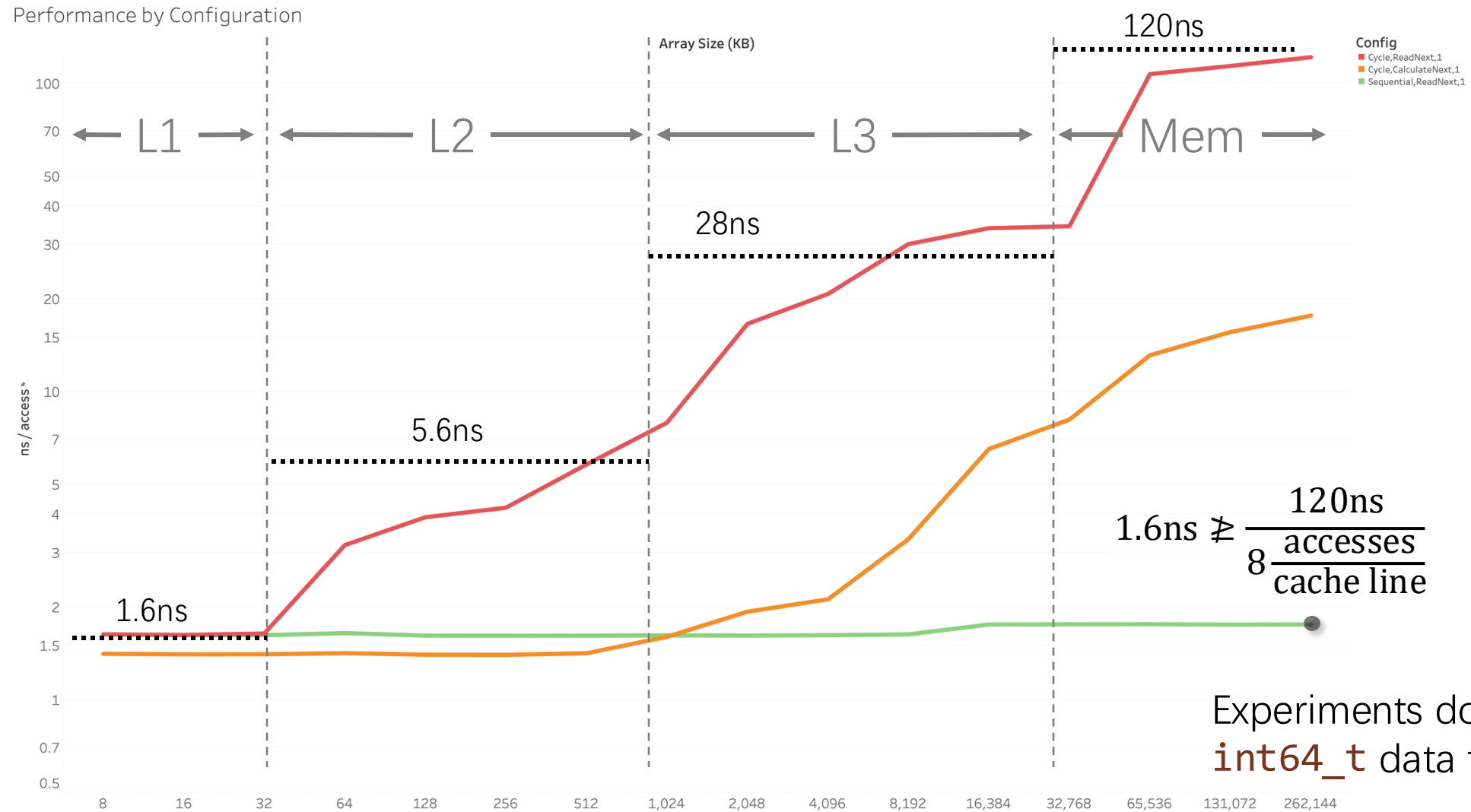
A is initialized to a sequential cycle,
otherwise identical to `cycleWithReadNext()`

```
sequentialWithReadNext(int*, int, int):  
    test edx, edx  
    jle .L4  
    xor ecx, ecx  
    mov eax, 1  
    xor esi, esi  
.L3:  
    movsx rax, DWORD PTR [rdi+rax*4]  
    add ecx, 1  
    add esi, eax  
    cmp edx, ecx  
    jne .L3  
    mov eax, esi  
    ret  
.L4:  
    xor esi, esi  
    mov eax, esi  
    ret
```

Sequential, ReadNext (Green)



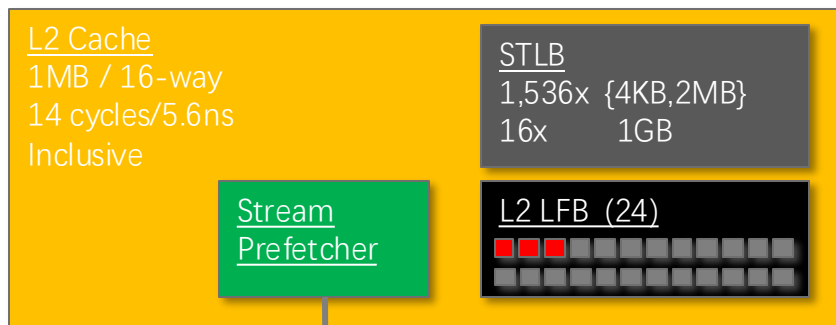
Cheating really works



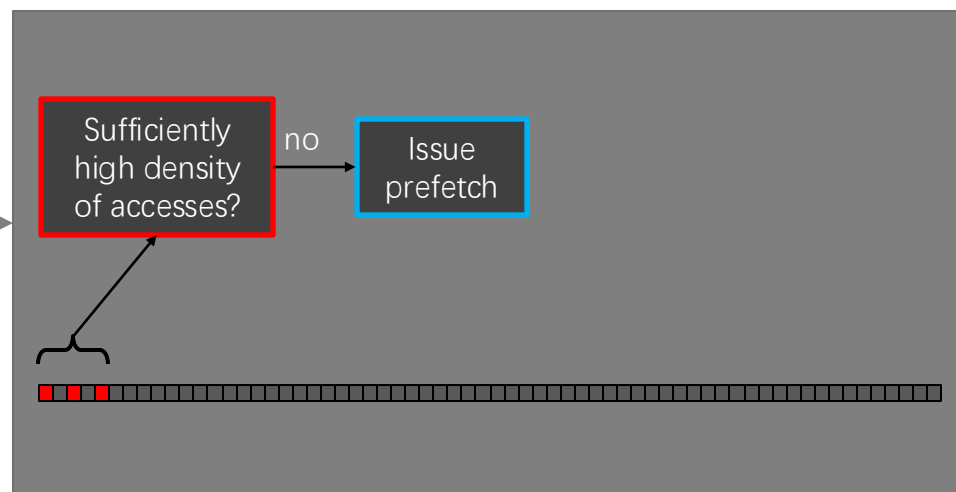
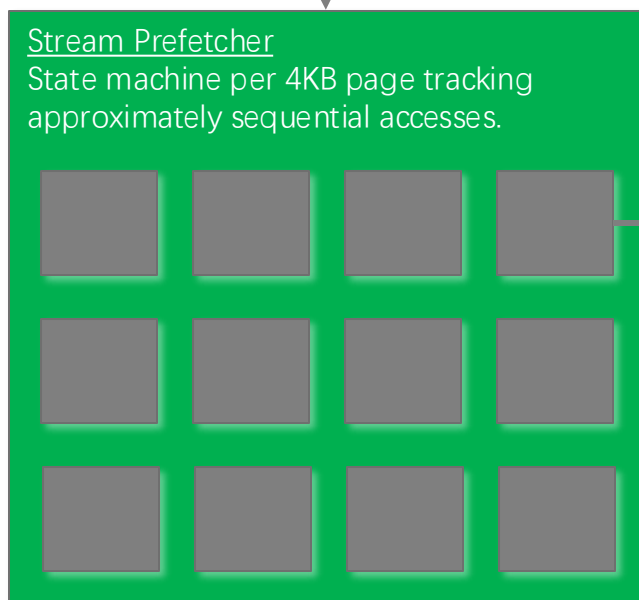
L2 STREAM PREFETCHER



Stream Prefetcher



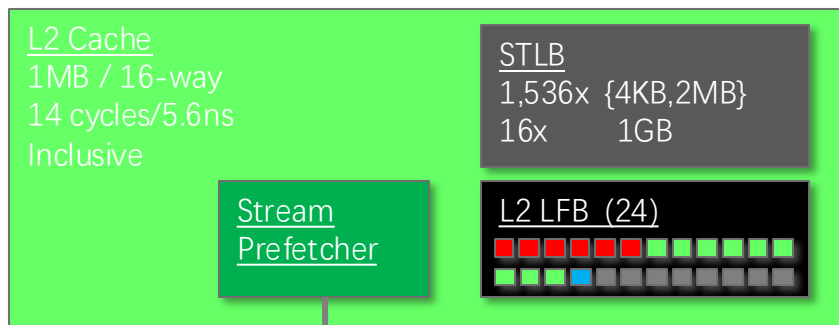
The Stream Prefetcher finds L1 miss streams that are approximately sequential and prefetches ahead.



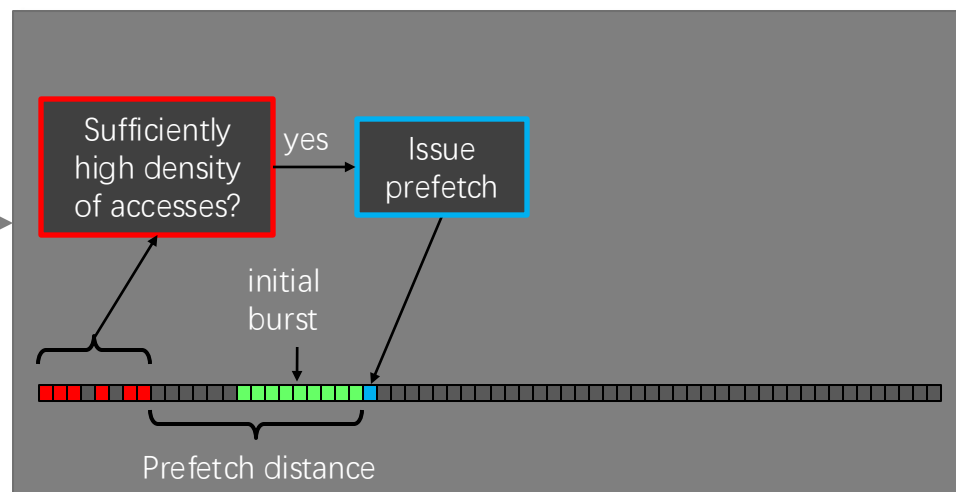
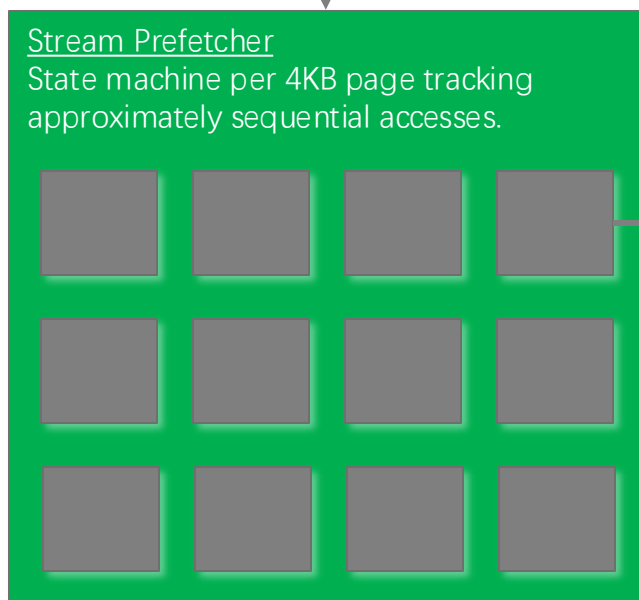
64 Cache Lines / 4KB Page



Stream Prefetcher



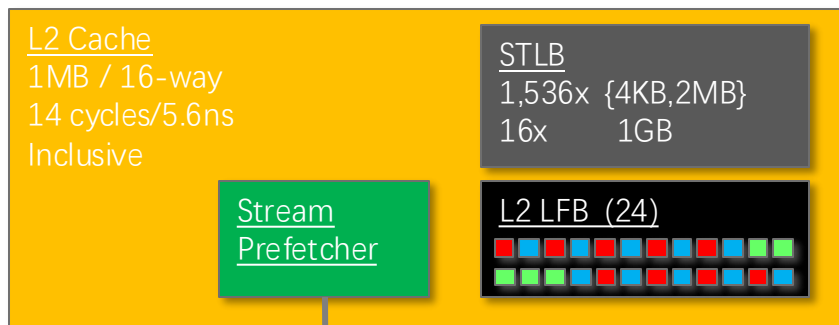
The Stream Prefetcher finds L1 miss streams that are approximately sequential and prefetches ahead.



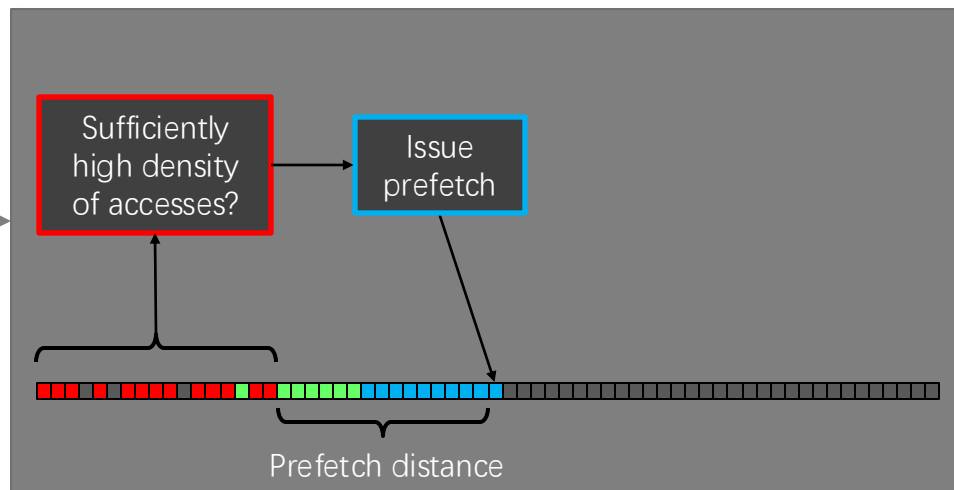
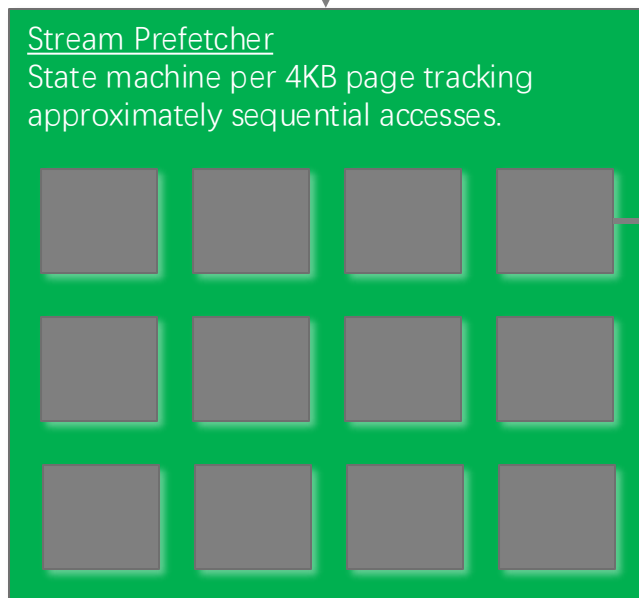
64 Cache Lines / 4KB Page



Stream Prefetcher



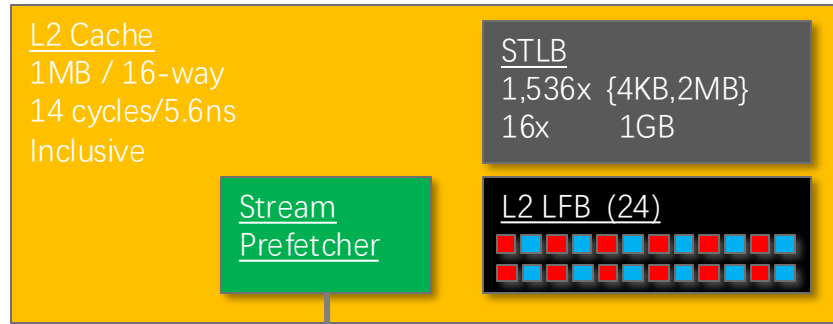
Using a larger set of Line Fill Buffers at the L2 level, the core's maximum Memory-Level Parallelism for streaming patterns increases.



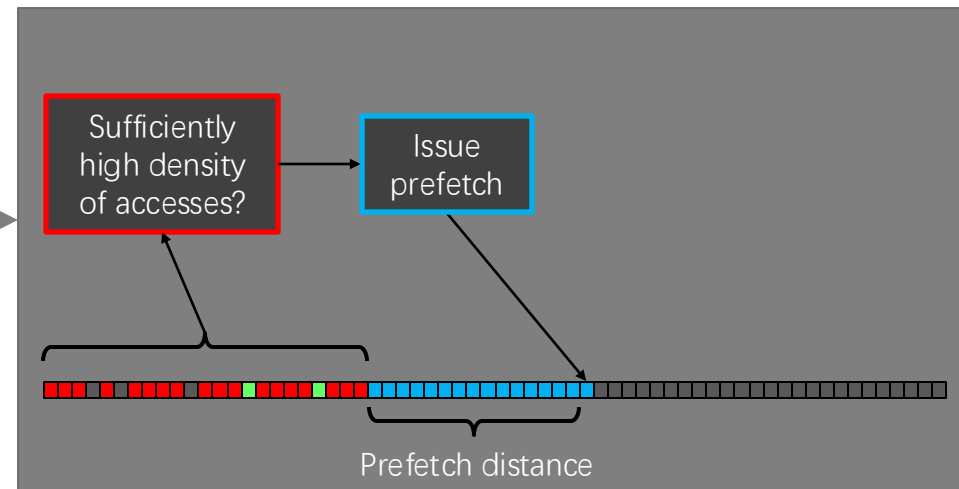
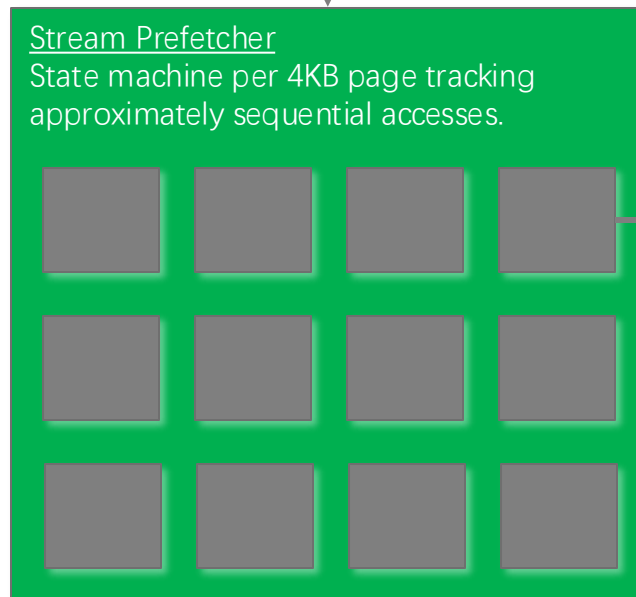
64 Cache Lines / 4KB Page



Stream Prefetcher



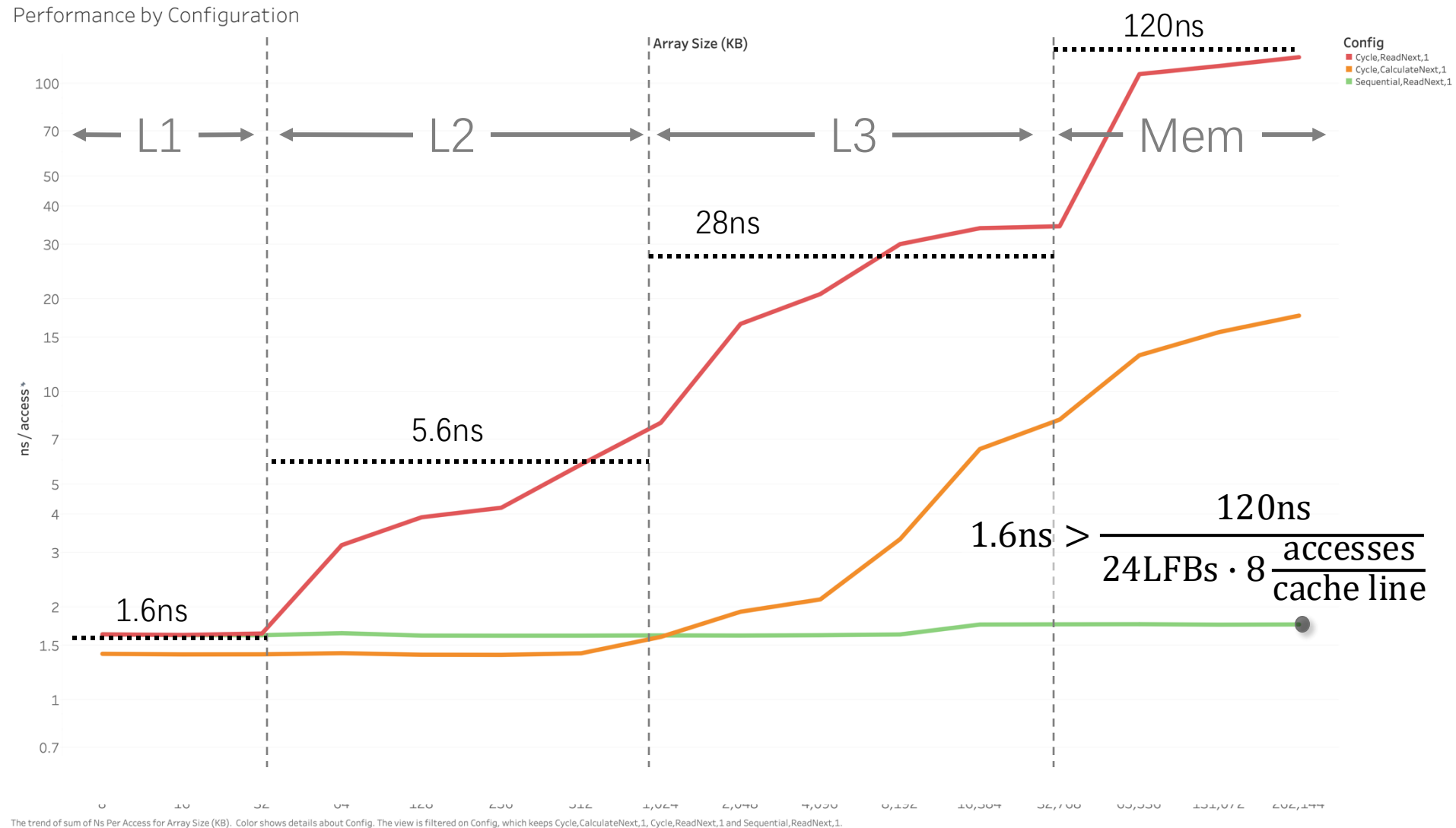
These prefetches are only installed in the L2, however.



64 Cache Lines / 4KB Page



Cheating really works



Sequential, CalculateNext

```
int sequentialWithCalcNext(int *A, int p, int iterations) {  
    int sum = 0;  
    int n = 1;  
    for (int i = 0; i < iterations; i++) {  
        sum += A[n];  
        n = n + 1;  
        n = n > p ? n - p : n;  
    }  
    return sum;  
}
```

Let's combine the benefits of both Memory-Level Parallelism and sequential access.

```
sequentialWithCalcNext(int*, int, int):  
    test edx, edx  
    jle .L5  
    xor ecx, ecx  
    mov eax, 1  
    xor r8d, r8d  
.L4:  
    movsx r9, eax  
    add eax, 1  
    add r8d, DWORD PTR [rdi+r9*4]  
    mov r9d, eax  
    sub r9d, esi  
    cmp esi, eax  
    cmovl eax, r9d  
    add ecx, 1  
    cmp edx, ecx  
    jne .L4  
    mov eax, r8d  
    ret  
.L5:  
    xor r8d, r8d  
    mov eax, r8d  
    ret
```

Sequential, CalculateNext

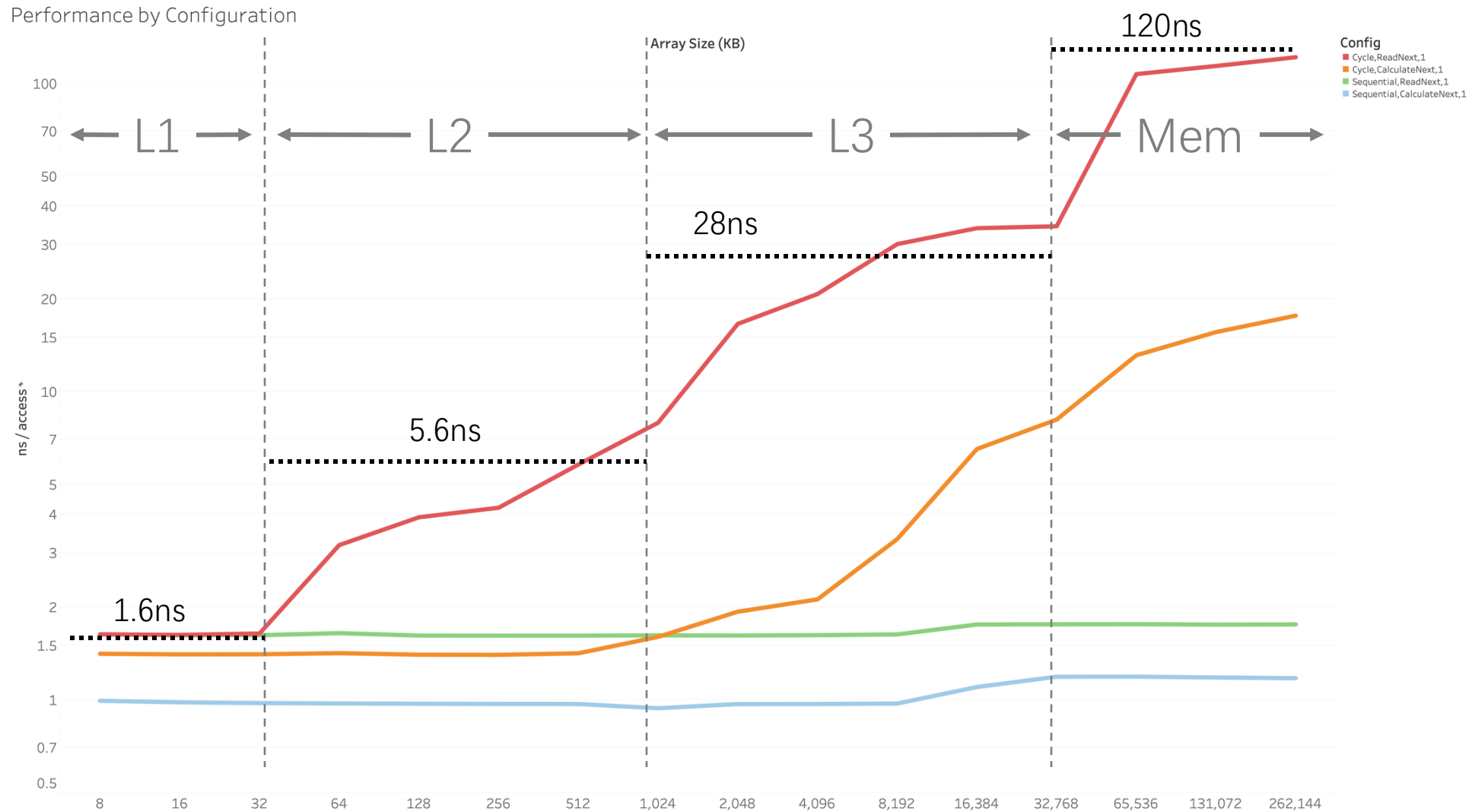
```
int sequentialWithCalcNext(int *A, int p, int iterations) {  
    int sum = 0;  
    int n = 1;  
    for (int i = 0; i < iterations; i++) {  
        sum += A[n];  
        n = n + 1;  
        n = n > p ? n - p : n;  
    }  
    return sum;  
}
```

Only difference with
Cycle (add 1 instead
of multiply by 2)

5 more instructions
than ReadNext

```
sequentialWithCalcNext(int*, int, int):  
    test edx, edx  
    jle .L5  
    xor ecx, ecx  
    mov eax, 1  
    xor r8d, r8d  
.L4:  
    movsx r9, eax  
    add eax, 1  
    add r8d, DWORD PTR [rdi+r9*4]  
    mov r9d, eax  
    sub r9d, esi  
    cmp esi, eax  
    cmovl eax, r9d  
    add ecx, 1  
    cmp edx, ecx  
    jne .L4  
    mov eax, r8d  
    ret  
.L5:  
    xor r8d, r8d  
    mov eax, r8d  
    ret
```

Sequential, CalculateNext (Blue)



Would blocking help?

```
int sequentialReadBlocked(int *A, int p, int iterations) {  
    int sum = 0;  
    int n = 1;  
    for (int i = 0; i < (iterations >> 4); i++) {  
        for (int j = 0; j < (1 << 4); j++) {  
            sum += A[n];  
            n = A[n];  
        }  
    }  
    return sum;  
}
```

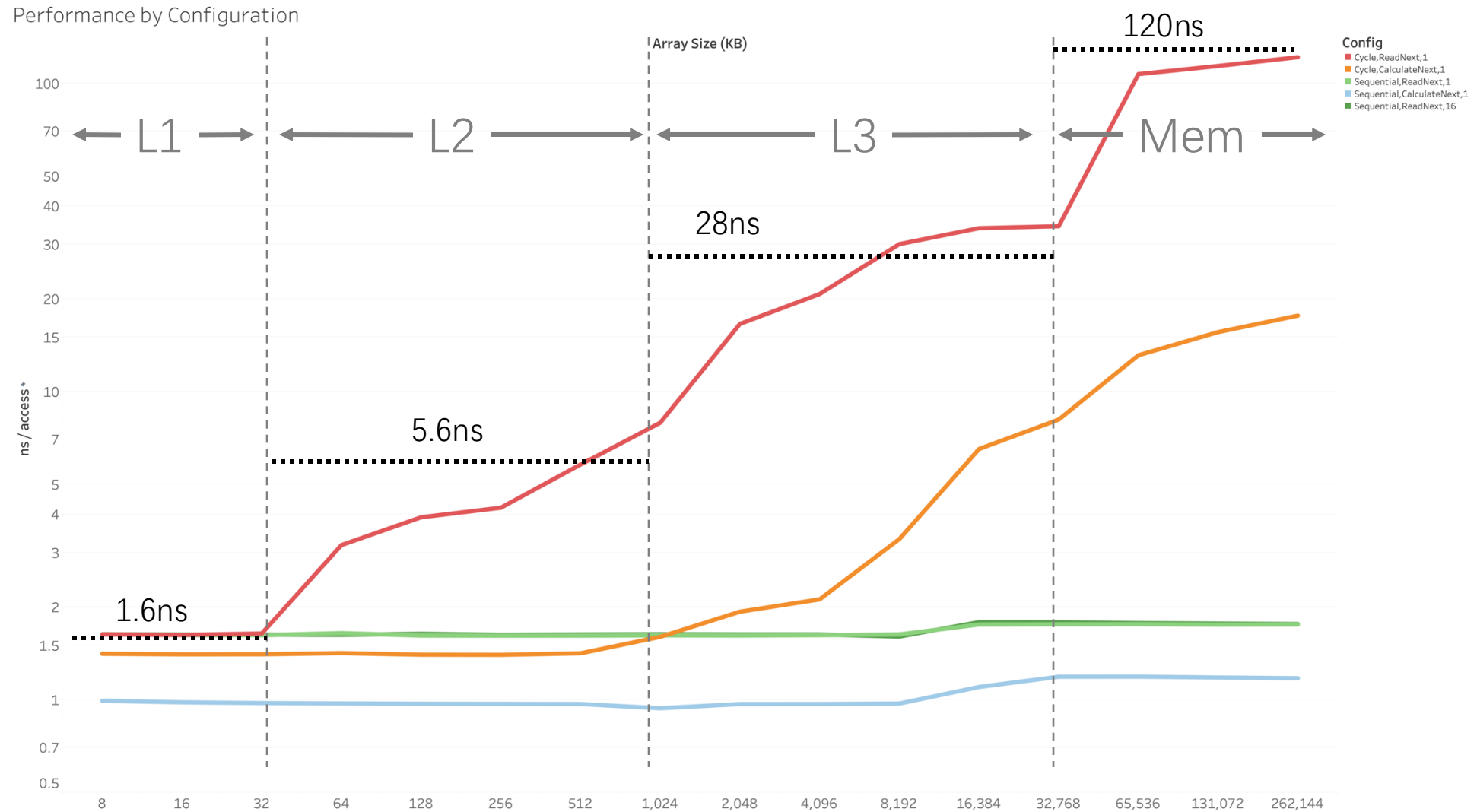
Loop-carried
dependence goes
through **memory**

x16 {

Loop Maintenance {

```
sequentialReadBlocked(int*, int, int):  
    mov rcx, rdi  
    mov edi, edx  
    sar edi, 3  
    test edi, edi  
    jle .L4  
    xor esi, esi  
    mov edx, 1  
    xor eax, eax  
.L3:  
    add esi, 1  
    movsx rdx, DWORD PTR [rcx+rdx*4]  
    add eax, edx  
    ...  
    add esi, 1  
    cmp edi, esi  
    jne .L3  
    ret  
.L4:  
    xor eax, eax  
    ret
```


Sequential, Read, 16x (Dark Green)



Why didn't that work?

We didn't alleviate the L1 data-dependency cycle, which is the bottleneck for the ReadNext pattern.

Blocking for CalculateNext?

```
int sequentialCalcBlocked(int *A, int p, int iterations) {  
    int sum = 0;  
    int n = 1;  
    for (int i = 0; i < (iterations >> 4); i++) {  
        for (int j = 0; j < (1 << 4); j++) {  
            sum += A[n];  
            n = n + 1;  
            n = n > p ? n - p : n;  
        }  
    }  
    return sum;  
}
```

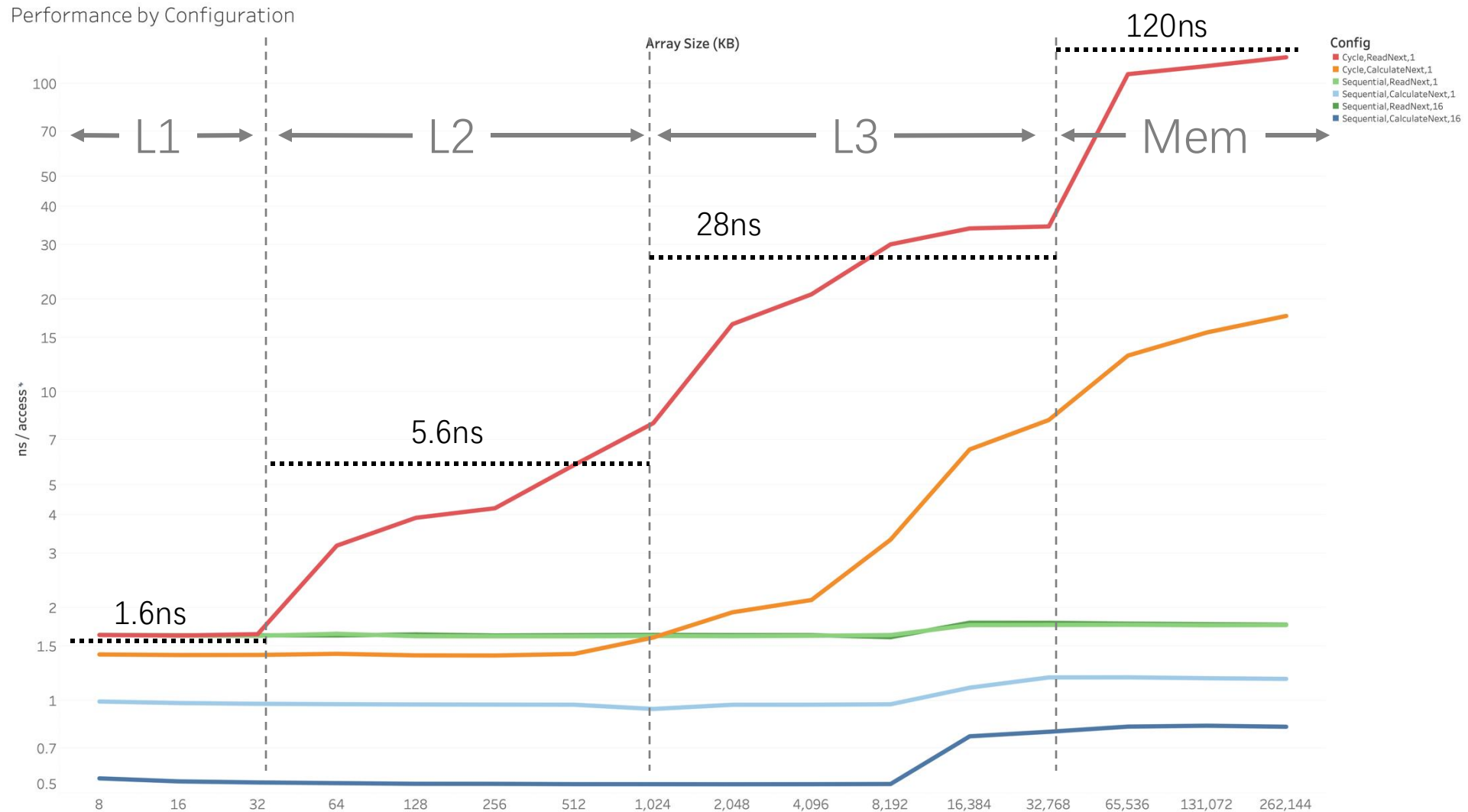
Loop-carried dependence
goes through a register

Loop Maintenance

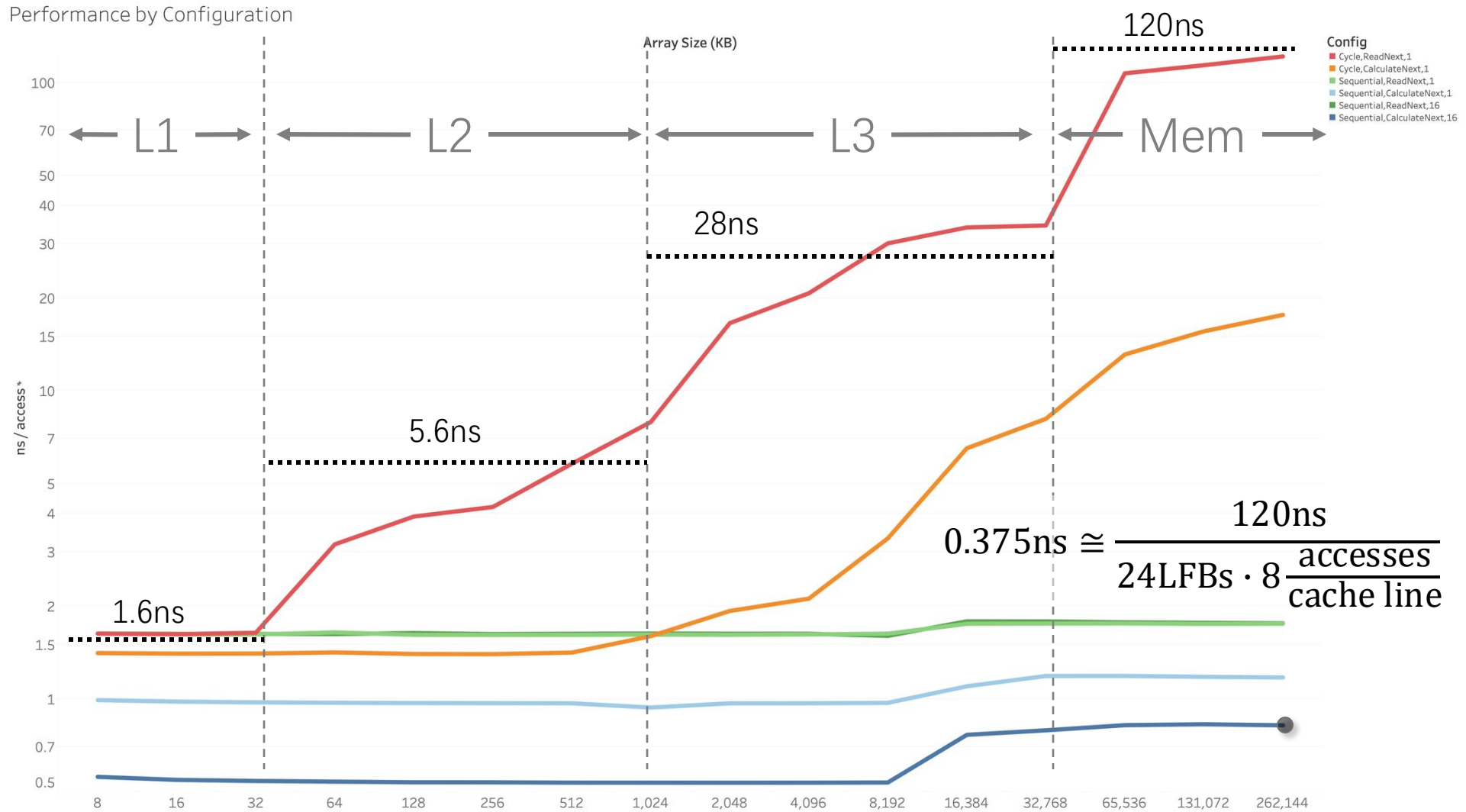
x16

```
sequentialCalcBlocked(int*, int, int):  
    sar edx, 3  
    mov ecx, esi  
    mov r8d, edx  
    test edx, edx  
    jle .L12  
    xor esi, esi  
    mov edx, 1  
    xor eax, eax  
.L11:  
    movsx r9, edx  
    add edx, 1  
    add eax, DWORD PTR [rdi+r9*4]  
    mov r9d, edx  
    sub r9d, ecx  
    cmp ecx, edx  
    cmovl edx, r9d  
    ...  
    add esi, 1  
    cmp r8d, esi  
    jne .L11  
    ret  
.L12:  
    xor eax, eax  
    ret
```

Sequential, Calc, 16x (Dark Blue)



Sequential, Calc, 16x (Dark Blue)



SEQUENTIAL STRIDES



LongStride, CalculateNext

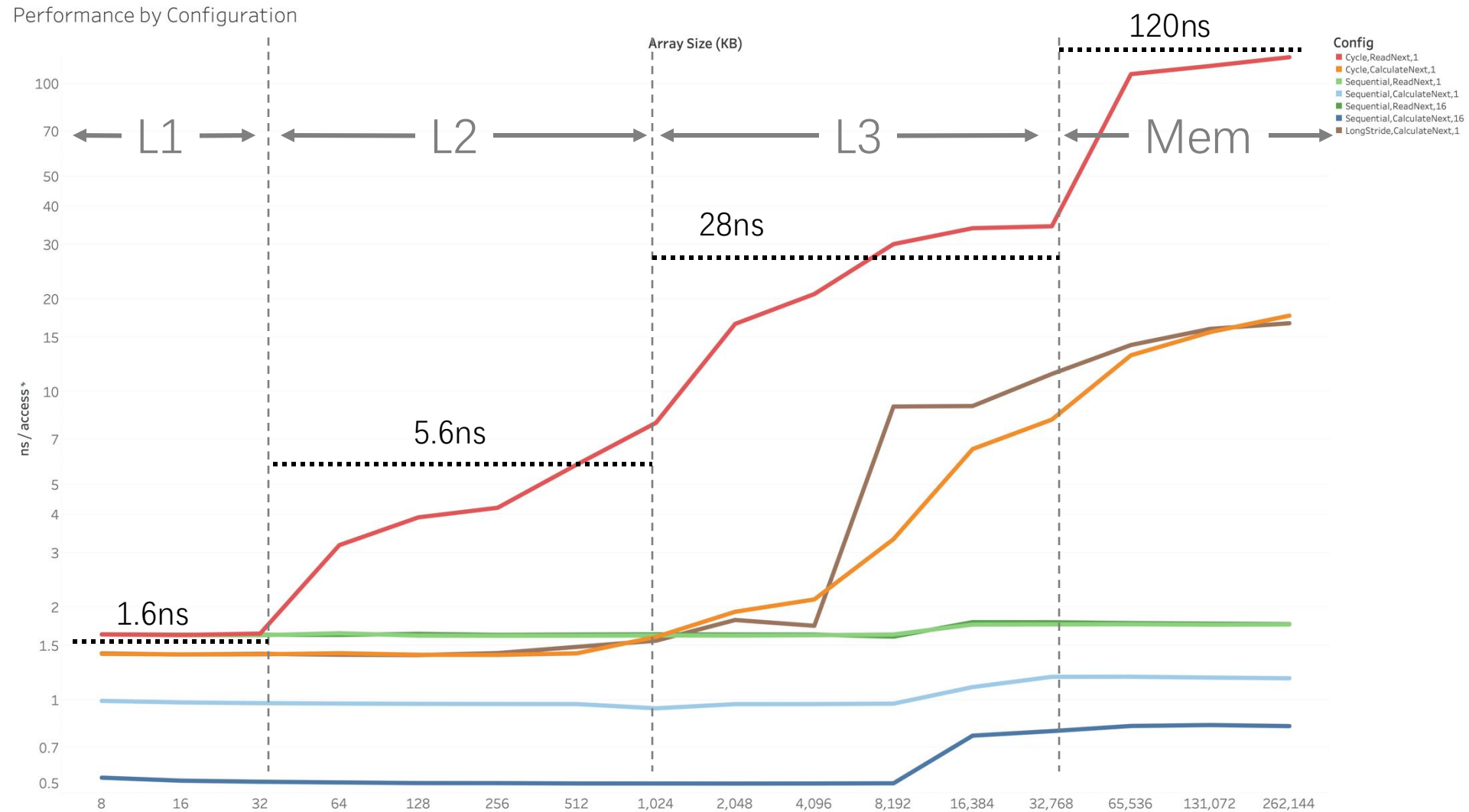
```
int longStrideWithCalcNext(int *A, int p, int iterations) {  
    int sum = 0;  
    int n = 1;  
    for (int i = 0; i < iterations; i++) {  
        sum += A[n];  
        n = n + 512;  
        n = n > p ? n - p : n;  
    }  
    return sum;  
}
```

Only difference with
Sequential: 1 access
per 4KB page

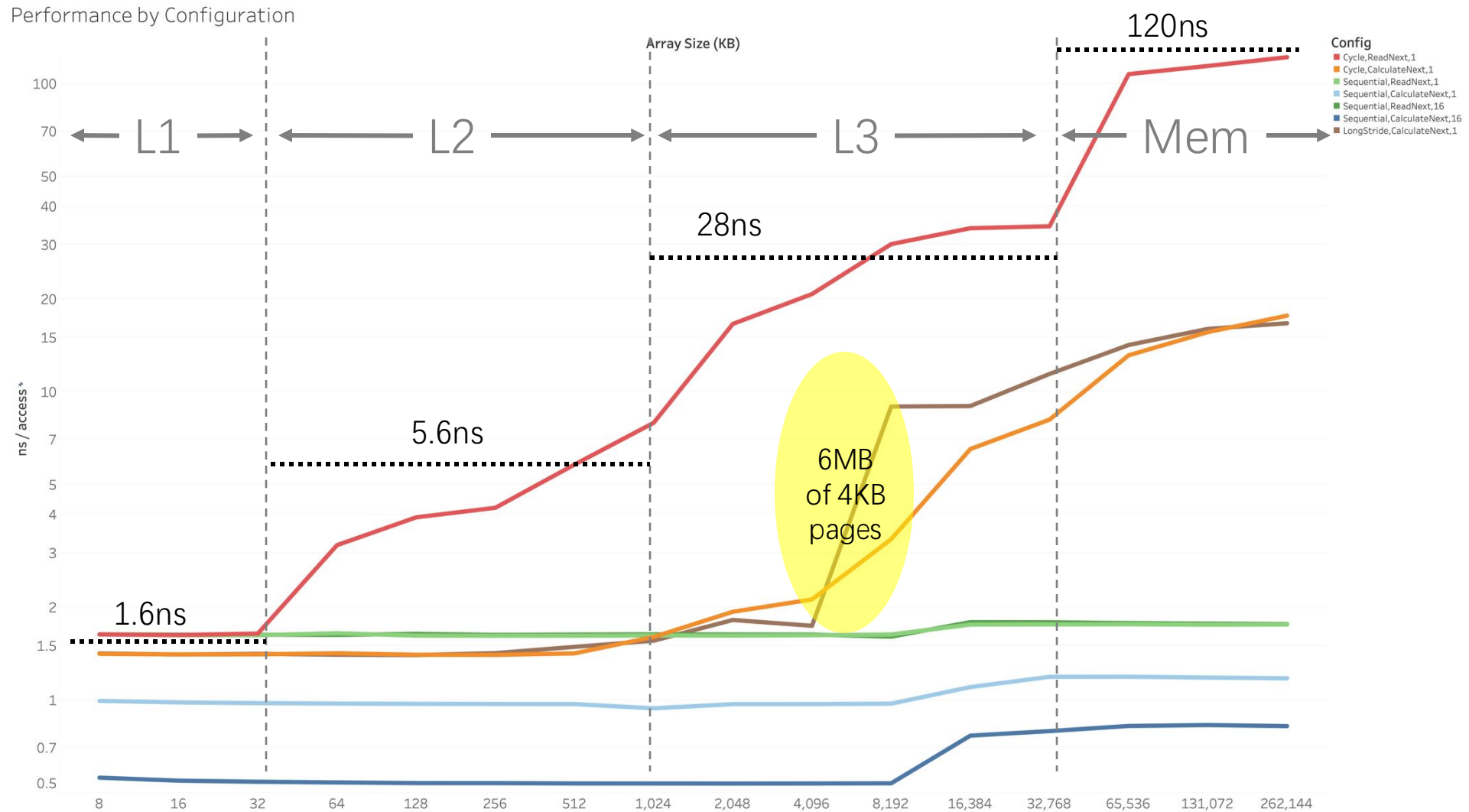
5 more instructions
than ReadNext

```
longStrideWithCalcNext(int*, int, int):  
    test edx, edx  
    jle .L5  
    xor ecx, ecx  
    mov eax, 1  
    xor r8d, r8d  
.L4:  
    movsx r9, eax  
    add eax, 512  
    add r8d, DWORD PTR [rdi+r9*4]  
    mov r9d, eax  
    sub r9d, esi  
    cmp esi, eax  
    cmovl eax, r9d  
    add ecx, 1  
    cmp edx, ecx  
    jne .L4  
    mov eax, r8d  
    ret  
.L5:  
    xor r8d, r8d  
    mov eax, r8d  
    ret
```

LongStride, Calc (Brown)



LongStride, Calc (Brown)



MediumStride, CalculateNext

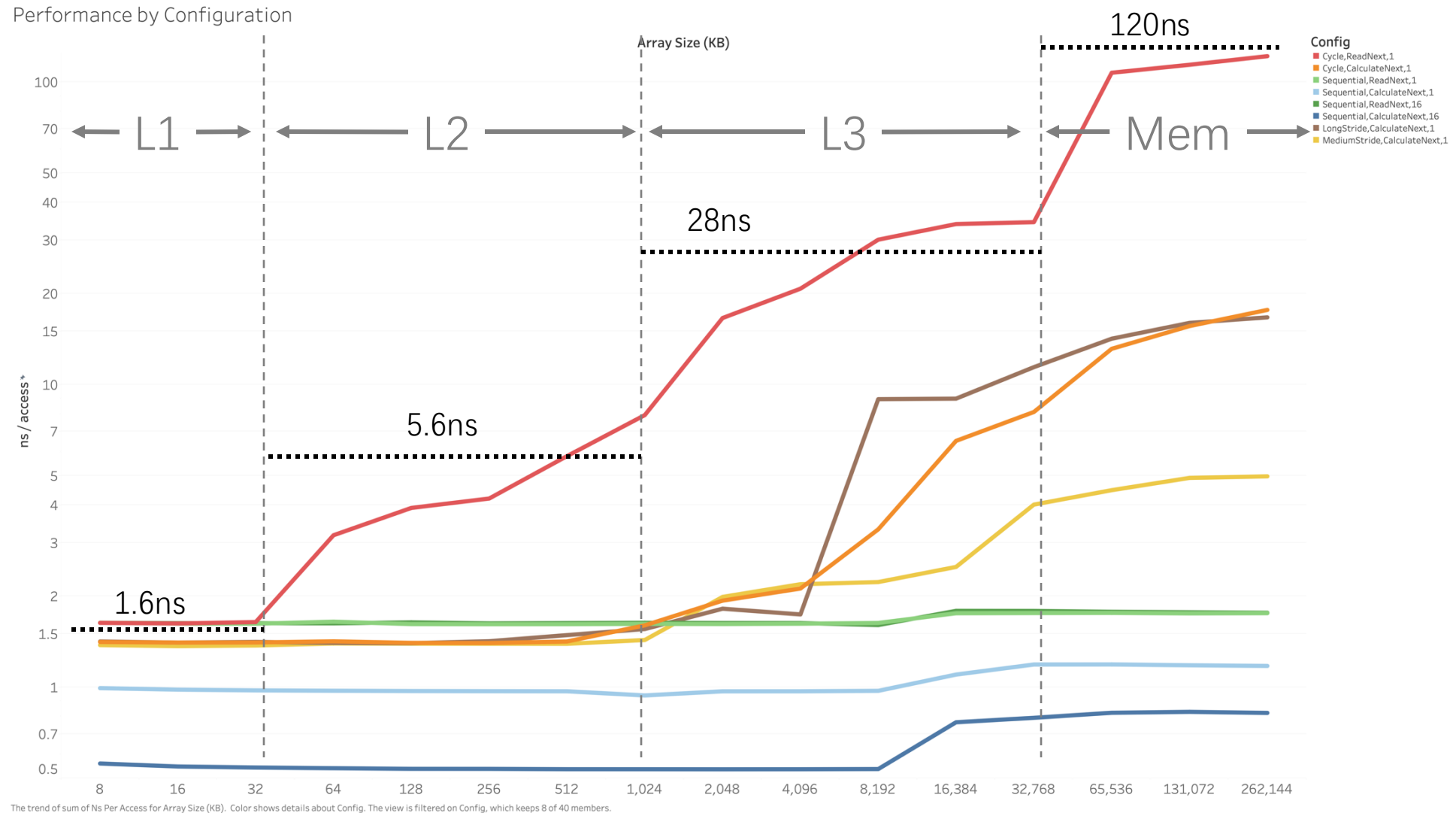
```
int mediumStrideCalcNext(int *A, int p, int iterations) {  
    int sum = 0;  
    int n = 1;  
    for (int i = 0; i < iterations; i++) {  
        sum += A[n];  
        n = n + 8;  
        n = n > p ? n - p : n;  
    }  
    return sum;  
}
```

Only difference with
sequential: 1 access
per cache line

5 more instructions
than ReadNext

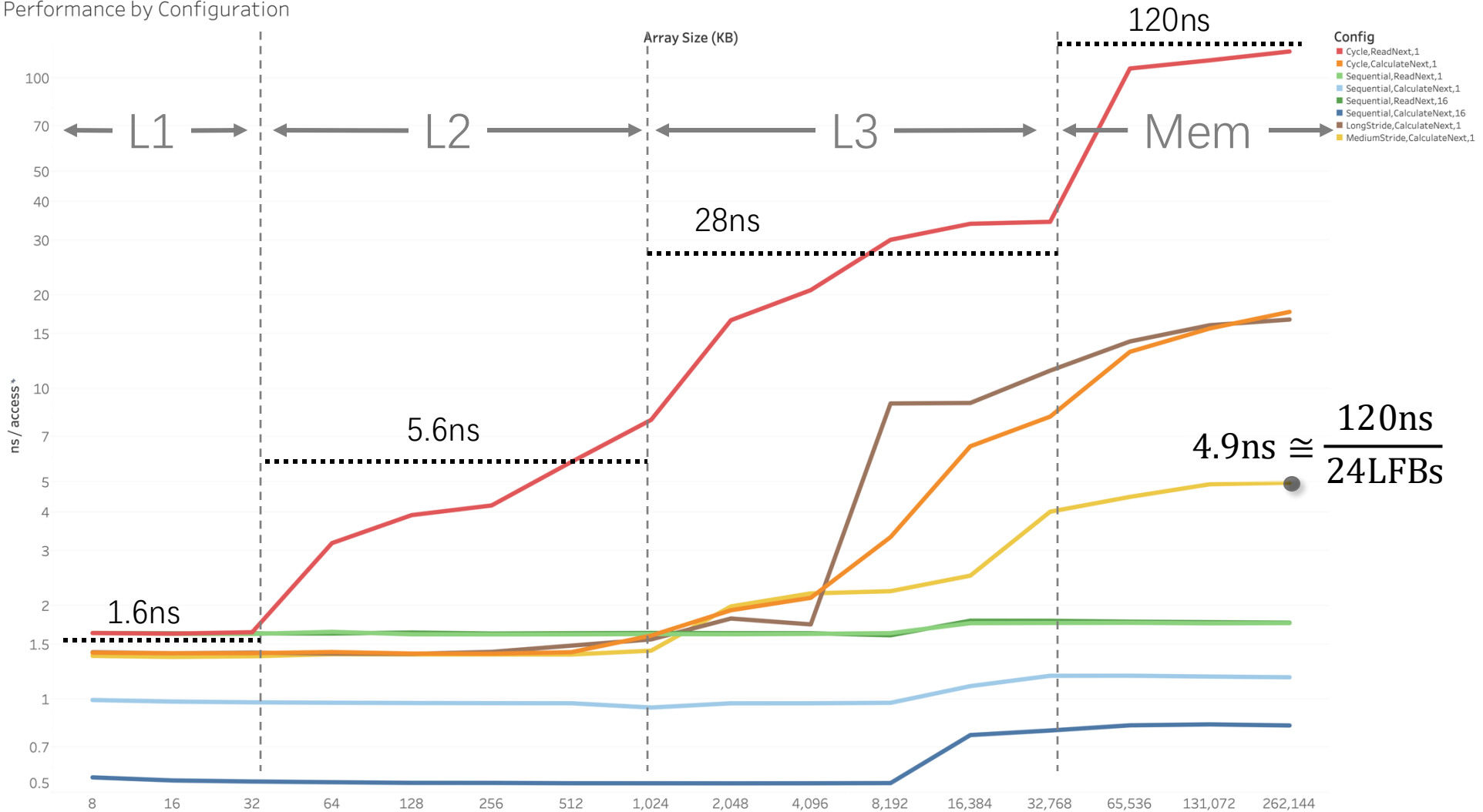
```
mediumStrideCalcNext(int*, int, int):  
    test edx, edx  
    jle .L5  
    xor ecx, ecx  
    mov eax, 1  
    xor r8d, r8d  
.L4:  
    movsx r9, eax  
    add eax, 8  
    add r8d, DWORD PTR [rdi+r9*4]  
    mov r9d, eax  
    sub r9d, esi  
    cmp esi, eax  
    cmovl eax, r9d  
    add ecx, 1  
    cmp edx, ecx  
    jne .L4  
    mov eax, r8d  
    ret  
.L5:  
    xor r8d, r8d  
    mov eax, r8d  
    ret
```

MediumStride, Calc (Yellow)



MediumStride, Calc (Yellow)

Performance by Configuration



Summary

- Garbage Collection
 - ▣ Reference Counting
 - ▣ Mark and Sweep
 - ▣ Stop-and-Copy
- Tradeoff: Ease of use / Performance / Memory Efficiency (Fragmentation)
- Memory Hierarchy & Cache Locality
- Memory Level Parallelism allows serving multiple memory requests simultaneously
 - ▣ Line-fill Buffer, OoO, hardware Prefetching, ...